

Weakly Super-Invertible Matrices and Constant Communication Dishonest Majority MPC

Alexander Bienstock^{1*} and Kevin Yeo²

¹ J.P. Morgan AI Research and J.P. Morgan AlgoCRYPT CoE

² Google and Columbia University

Abstract. In recent years, there has been tremendous progress in improving the concrete communication complexity of dishonest majority MPC. In the sub-optimal corruption threshold setting where $t < (1 - \varepsilon) \cdot n$ for some constant $0 < \varepsilon \leq 1/2$, Sharing Transformation (Goyal *et al.*, CRYPTO'22) and SuperPack (Escudero *et al.*, EUROCRYPT'23) presented protocols with information-theoretic online phases requiring $O(1)$ field elements of total communication per multiplication gate. However, Sharing Transformation assumes that their offline phase is instantiated by a trusted party, while SuperPack instantiates their offline phase with large communication of $\Omega(n)$ per multiplication gate assuming oblivious linear evaluation (OLE) correlations. The main bottleneck in instantiating the offline phases of both protocols is generating random packed beaver triples of the form $[\mathbf{a}], [\mathbf{b}], [\mathbf{c}]$, for random $\mathbf{a}, \mathbf{b} \in \mathbb{F}^k$, and $\mathbf{c} = \mathbf{a} * \mathbf{b} \in \mathbb{F}^k$, where $k = \Omega(n)$ is the *packing parameter*.

To address this bottleneck, our main technical contribution is introducing and constructing *weakly* super-invertible matrices, a relaxation of super-invertible matrices in which sub-matrices have high (but not necessarily full) rank. This relaxation allows for matrices with only $\tilde{O}(n)$ non-zero entries, enabling a first step towards generating packed beaver triples with $\tilde{O}(1)$ total communication per underlying triple, assuming OLE correlations. As the second (and final) step, we use the efficient *triple extraction* protocol of (Choudhury and Patra, Trans. Inform. Theory '17).

We also implement our packed beaver triple protocol and provide experimental results. Our new protocol obtains up to 38% smaller communication and 9% reduction in runtime compared to SuperPack's triple protocol. Additionally, by instantiating SuperPack's offline phase with our new protocol, we obtain up to 16% communication reductions.

Finally, we use our packed beaver triple protocol to instantiate the offline phase of Sharing Transformation, yielding a dishonest majority MPC protocol with $\tilde{O}(|C|)$ total communication across both the offline and online phases.

An extended abstract [BY25] of this article appears in the proceedings of Eurocrypt 2025.

* Work partially done while at NYU and Google.

1 Introduction

Secure Multi-Party Computation (MPC) is a widely studied area of cryptography [GMW87, Yao82, CCD88, BGW88]. In MPC, there are n parties with respective private inputs $x_1, \dots, x_n$, and some functionality $\mathcal{F}$. The parties wish to interact with each other to compute and output $\mathcal{F}(x_1, \dots, x_n)$ in such a way that their inputs still remain hidden, beyond what can be inferred by the output. In fact, this should still hold even if there is an adversary controlling t out of n of the parties, who we call *corrupted*. The adversary sees all of the messages that the other parties, who we call *honest*, send these corrupted parties. Adversaries can either be *static* or *adaptive*. In the former case, the adversary must choose which parties to corrupt before the protocol begins. In the latter case, the adversary chooses which parties to corrupt during the protocol, possibly based on the observed messages.

In this work, we focus on the so-called *dishonest majority* setting in which the number of corrupted parties is $t \geq n/2$. In this setting, it is known that protocols require computational assumptions to be proved secure [RB89], unlike in the case where $t < n/2$ [BGW88, CCD88, RB89]. However, many dishonest majority protocols operate in the *offline-online* model, in which there is first an offline phase that is typically proved computationally-secure and then an online phase that can be proved information-theoretically secure [Bea92]. This model has the benefit that expensive cryptographic operations can be performed in the offline phase (at any point before the functionality and inputs are determined), meaning that the online phase is usually quite efficient. Typically, the offline phase provides the parties with some correlated randomness, which they consume in the online phase. This is the model which we focus on in this work.

In this setting, the main goal has been to reduce the communication complexity of protocols. The seminal works of BeDOZa [BDOZ11] and SPDZ [DPSZ12, DKL⁺13] achieve linear communication complexity in the number of parties per multiplication gate, in both the offline phase and online phase. The offline phases of these protocols produce simple correlated randomness in the form of oblivious linear evaluation (OLE) [IPS09, AIK11] for every pair of parties. An OLE correlation between parties P_A and P_B samples consists of random x, u, v such that P_A receives x and v , and P_B receives u and $w = u \cdot x - v$.³ The above works consider an *optimal* corruption threshold, $t = n - 1$, where all but one parties could be corrupted.

The tremendous recent works of Sharing Transformation [GPS22] and SuperPack [EGP⁺23] instead study a *sub-optimal* corruption threshold, in which $t < (1 - \varepsilon) \cdot n$, for some constant $0 < \varepsilon \leq 1/2$. Here, they are able to obtain online phases with total communicated field elements $58/\varepsilon + 96/\varepsilon^2$ and $6/\varepsilon$, respectively. Both of these protocols crucially take advantage of *packed* Shamir secret sharing [FY92], which allows parties to secret share many (proportional to

³ Such OLEs can be generated with amortized *sub-linear* communication (in the number of correlations), using pseudo-correlation generators [BCG⁺19b, BCG⁺20, BCGI18, WYKW21, BCG⁺19a].

n) values at once. However, the Sharing Transformation work does not instantiate their offline phase, which provides the parties with highly non-trivial correlated randomness. Any currently-available generic protocol would unfortunately have *linear* communication in the number of parties per gate to instantiate this correlated randomness. This is much worse than the $O(1)$ per gate during the online phase. SuperPack does instantiate their offline phase, which still assumes some simple OLE correlations, and requires large communication dependent on n per gate, even assuming this correlated randomness. Again, this is much worse than the $O(1)$ per gate during the online phase. We next introduce the main barrier for better efficiency in instantiating these offline phases.

Packed Beaver Multiplication Triples. A common tool used in dishonest majority MPC is beaver multiplication [Bea92]. In the $t < (1 - \varepsilon) \cdot n$ setting, both of the above works [GPS22, EGP⁺23] utilize this technique through packed Shamir secret sharing [FY92]. For $\Theta(n)$ -dimensional *vectors* $\mathbf{x}$ and $\mathbf{y}$, the parties wish to compute a packed Shamir sharing of the vector of multiplications $\mathbf{x} * \mathbf{y}$ on input packed Shamir sharings $[\mathbf{x}]$ and $[\mathbf{y}]$, where $*$ denotes component-wise multiplication. They do so using packed Shamir sharings $[\mathbf{a}]$, $[\mathbf{b}]$, and $[\mathbf{c}]$, such that $\mathbf{a}$ and $\mathbf{b}$ are fully random $\Theta(n)$ dimensional vectors, and $\mathbf{c} = \mathbf{a} * \mathbf{b}$. These so-called packed beaver triples allow the parties to compute $[\mathbf{x} * \mathbf{y}]$ with communication linear in n . Since this is for $\Theta(n)$ gates at once, the communication per gate is therefore $O(1)$. Indeed, the protocols of [GPS22, EGP⁺23] require that these packed beaver triples $[\mathbf{a}]$, $[\mathbf{b}]$, $[\mathbf{c}]$ be generated during the offline phase. This turns out to be the main bottleneck for both offline phases. In particular, generating these packed triples alone requires $\Omega(n^2)$ communication per packed triple, and thus $\Omega(n)$ per underlying triple, using known techniques. This leaves the following question, which will be addressed in our work:

*Can we generate packed beaver triples
with $\tilde{O}(1)$ communication per underlying triple?*

In this work, we answer this question affirmatively by constructing such a protocol. We next introduce the core technical tool for our solution.

(Weakly) Super-Invertible Matrices. Another common tool used in efficient MPC protocols are super-invertible $m \times n$ matrices $\mathbf{M}$, where $m \leq n$. These matrices satisfy that, for all column subsets $C \subseteq [n]$ with $|C| = m$, the sub-matrix $\mathbf{M}^C$ with columns whose indices appear in C is invertible. These matrices are typically used to efficiently sample secret sharings of random values [DN07]. In the $t < (1 - \varepsilon) \cdot n$ setting, both [GPS22, EGP⁺23] utilize super-invertible matrices to generate packed Shamir sharings $[\mathbf{r}]$ of random vectors with $O(1)$ communication per individual random value. The standard instantiation of super-invertible matrices is the transpose of a Vandermonde matrix [DN07]. Such matrices are *dense* where almost all entries are non-zero. In our work, we introduce a relaxation of super-invertible matrices, which we call *weakly* super-invertible matrices, that will actually allow us to generate packed beaver triples with $\tilde{O}(n)$ communication per packed triple and thus $\tilde{O}(1)$ per underlying triple. The main reason is that we

are able to construct weakly super-invertible matrices that are *sparse*; i.e., most entries are zero. Weak super-invertibility requires that sub-matrices consisting of sufficiently many columns have large (but not necessarily full) rank. We will show that this property is sufficient for efficiently generating packed beaver triples. We also believe that weakly super-invertible matrices may be of independent interest.

1.1 Our Contributions

Weakly Super-Invertible Matrices. As mentioned above, we aim to generate packed beaver triples with $\tilde{O}(n)$ communication per triple. The main way we do this is through a weakening of super-invertible matrices that we develop, called *weakly super-invertible matrices* requiring that sub-matrices have sufficiently high (but not full) rank. We say a $m \times n$ matrix is weakly super-invertible if every $m \times \Theta(m)$ sub-matrix has rank at least $\gamma \cdot m$ for some constant $\gamma > 1/2$. This is a weaker property than standard super-invertibility requiring full rank. Nevertheless, we later show that this property is still sufficient for our MPC applications. We construct weakly super-invertible matrices that are sparse:

Theorem 1 (Informal). *For any constants $1/2 < \gamma < \eta < 1$, any $n = \Theta(m)$ and any finite field $\mathbb{F}$, there exists a matrix generation algorithm that outputs random $m \times n$ matrices satisfying the following except with probability $2^{-\lambda}$:*

1. *Every sub-matrix of dimension $m \times (\eta \cdot m)$ will have rank at least $\gamma \cdot m$.*
2. *Every row will have at most $O(\lambda + \log m)$ non-zero entries.*

Additionally, the whole matrix always has at most $O(n \cdot \log |\mathbb{F}|)$ non-zero entries.

We also present concrete matrix instantiations with only $O(n)$ non-zero entries whose weakly super-invertible guarantees may be related to experimental analysis on three-way cuckoo hashing failures [CLR17, PSZ18].

Packed Beaver Triples with $\tilde{O}(1)$ Communication. Using weakly super-invertible matrices, we provide a protocol that generates $\Theta(n)$ random packed beaver triples with $\tilde{O}(n)$ communication per packed triple or $\tilde{O}(1)$ communication per underlying triple. Our protocol assumes some $\tilde{O}(n)$ pairs of parties each have $O(n)$ OLE correlations (note, this is not all $O(n^2)$ pairs of parties as required by SuperPack, c.f. [EGP⁺23, Remark 1]).

Theorem 2 (Informal). *Let λ be the statistical security parameter. There exists an information-theoretically secure protocol in the OLE-model which securely generates $\Theta(n)$ packed beaver triples in the presence of a malicious, adaptive adversary controlling up to $t < (1 - \varepsilon) \cdot n$ parties. The cost of the protocol is $O(n)$ OLE correlations between each of $O(n \cdot \lambda^2)$ pairs of parties and $\tilde{O}(n^2)$ elements of communication, which is $\tilde{O}(n)$ per packed triple and $\tilde{O}(1)$ per underlying triple.*

We also implement our packed beaver triple protocol and provide experimental results, using the above concrete matrix instantiations. The concrete

communication cost of our protocol is as low as $84/\varepsilon$ field elements per underlying multiplication triple. In contrast, SuperPack requires $n + 14/\varepsilon$ field elements of communication per underlying multiplication triple. Our protocol obtains up to a 38% decrease for $n = 256$ with even better improvements for larger n . For $n = 256$, our new protocol also obtains up to 9% reduction in runtime compared to SuperPack’s triple generation. We believe the improvements will only be better for larger n and $|C|$.

We use the above result to provide a more efficient instantiation of SuperPack’s offline phase. We first note that SuperPack’s offline phase inherently requires linear communication per multiplication gate, that is irrelevant to the packed beaver triple generation. Indeed, it requires a unique random secret sharing per individual multiplication gate in the circuit, whose shares need to be communicated to each party. By plugging in our new triple protocol, we obtain up to a 16% reduction in total communication in the offline phase.

Towards Concretely-Efficient $\tilde{O}(|C|)$ -Communication Dishonest Majority MPC. Our new packed beaver triple protocol can also instantiate the offline phase of Sharing Transformation with $\tilde{O}(1)$ communication per multiplication gate, concretely as low as $814/\varepsilon$. Indeed, efficient packed beaver triple generation was the main barrier for this. This implies an MPC protocol in the $t < (1 - \varepsilon) \cdot n$ setting with $\tilde{O}(1)$ total communication per multiplication gate. While Sharing Transformation’s online phase is concretely more expensive compared to SuperPack, we believe that this result is a step in the direction of concretely efficient $\tilde{O}(|C|)$ total communication protocols for dishonest majority MPC. This result is presented in Section F.

Theorem 3 (Informal). *Let λ be the statistical security parameter. For an arithmetic circuit C over a finite field $\mathbb{F}$, there exists an information-theoretically secure MPC protocol in the OLE-model which securely computes the arithmetic circuit C in the presence of a malicious, adaptive adversary controlling up to $t < (1 - \varepsilon) \cdot n$ parties. The cost of the protocol is $O(|C|/n)$ OLE correlations between each of $O(n \cdot \lambda^2)$ pairs of parties and $\tilde{O}(|C| + n^2)$ elements of communication. This is $\tilde{O}(1)$ per gate when the circuit is sufficiently large, $|C| = \Omega(n^2)$.*

One may compare with the player virtualization techniques of Bracha [Bra87] that could also be used to obtain $\tilde{O}(|C|)$ communication protocols. Bracha’s approach requires black-box emulation of MPC protocols (among other issues) that is highly impractical due to very large concrete costs. In contrast, we show that our algorithms result in both small asymptotic and concrete communication that allow us to obtain concrete improvements to current state-of-the-art protocols such as SuperPack. One can view our work as obtaining the first $\tilde{O}(|C|)$ communication dishonest majority MPC protocol with reasonable practical efficiency. We note that we do not implement this protocol as there is no publicly available implementation of Sharing Transformation to our knowledge. See Section 1.2 for more details about Bracha’s technique and Section 2.1 for further comparison.

1.2 Related Works

Committee-Based MPC. For static security, there is a simple MPC protocol with $\tilde{O}(1)$ total communication in the number of parties in the $t < (1 - \varepsilon) \cdot n$ setting [HIK07]. This protocol works by selecting a random committee of parties of size proportional to the computational security parameter, κ , to essentially perform the MPC for all of the parties. However, this simple protocol is trivially broken by an adaptive adversary who gets to observe and corrupt the members of the committee. Our work only considers security against adaptive adversaries.

One can indeed use committees in a more sophisticated manner, using the *player virtualization* technique of Bracha [Bra87], to achieve communication-efficient adaptively-secure dishonest majority MPC. In this technique, $N = \Theta(n)$ random committees of size $K = \Theta(1/\varepsilon)$ are first sampled. The N committees each use some inner MPC protocol that is secure against all but one corruption to emulate the corresponding party in some outer MPC protocol that is secure against a small constant fraction (less than $1/2$) of corruptions, and which carries out the actual computation. The committees can be chosen such that if there are only $t < (1 - \varepsilon)n$ total corruptions, then the number of committees (and hence parties in the outer protocol) that are corrupted is small enough for security. However, there are two main issues with this approach: (i) since the committees need to emulate parties in the outer protocol, one has to use an outer protocol with $\tilde{O}(|C|)$ communication *and computation*, and thus one must resort to the protocols of [DIK10, GIP15, IKP⁺16], which introduce $\text{poly}(\log n)$ and $\log |C|$ multiplicative factors that hinder concrete communication efficiency [GPS21]; (ii) MPC protocol emulation itself is not a concretely efficient endeavor—one must emulate the parties of the outer protocol with another MPC protocol, introducing to the complexity (at least) multiplicative factors proportional to the committee size K and other constant multiplicative factors.

As we explain in Section 2.1, one can view the part of our packed beaver triple protocol that uses weakly super-invertible matrices as a concretely efficient version of Bracha committees for dishonest majority MPC. We show in Section 8 that our protocol is indeed concretely efficient.

FHE-Based MPC. If one were allowed to use FHE, then MPC can be instantiated with $O(1)$ communication per gate [AJL⁺12, GHS12, MSS11]. However, FHE is a heavy cryptographic tool that is prohibitively expensive [DPSZ12, EGP⁺23] and faces significant barriers in being proved adaptively-secure [KTZ13, CSW23].

Dishonest Majority MPC in the Offline-Online Model. There are many dishonest majority MPC protocols operating in the offline-online mode. For some notable examples, we point readers to BeDOZa [BDOZ11], SPDZ [DPSZ12, DKL⁺13], MASCOT [KOS16], Overdrive [KPR18], TopGear [BCS19] and references therein.

Honest Majority MPC with Sub-Optimal Threshold. In the setting of honest majority information-theoretic MPC with sub-optimal threshold, where the number of corrupted parties is $t < (1/2 - \varepsilon) \cdot n$, there has been a fruitful line of works achieving sub-linear communication per multiplication gate such

as [FY92, DIK10, GPS21, GIP15, GIOZ17, BGJK21, GSY21]. Indeed, the work of [GPS21] shows how to get $O(1)$ communication per multiplication gate in this setting.

Invertible Matrices. Invertible matrices have been an important tool in the construction of MPC. Hirt and Nielsen [HN06] introduced super-invertible matrices and Beerliová-Trubíniová and Hirt [BTH08] extended this to hyper-invertibility. These matrices have been used in MPC in several ways including for randomness extraction and error correction [DN07, BTH08]. To our knowledge, the main instantiation for either matrix definition is Vandermonde matrices.

2 Technical Overview

The main goal of this paper is to generate packed beaver triples with $\tilde{O}(1)$ communication per underlying triple in the OLE-model, an $\tilde{\Omega}(n)$ multiplicative factor improvement over the best previous [EGP⁺23]. This technical overview is dedicated to describing our protocol for this, as well as the rest of our instantiation of SuperPack’s and Sharing Transformation’s offline phases.

Packed Shamir Secret Sharing. Packed Shamir secret sharing, introduced by [FY92] is a generalization of standard Shamir secret sharing [Sha79]. With packed Shamir secret sharing, we can pack a vector $\mathbf{x} \in \mathbb{F}^k$ of k secrets into one degree- d sharing, $[\mathbf{x}]_d$. Reconstructing a degree- d packed Shamir sharing requires $d+1$ shares and can be done using Lagrange interpolation. Moreover, for a random degree- d packed Shamir sharing of $\mathbf{x}$, any $d - k + 1$ shares are independent of the secret $\mathbf{x}$. Packed Shamir sharings have the following properties:

- Linearity: For any $n > d \geq k - 1$ and $\mathbf{x}, \mathbf{y} \in \mathbb{F}^k$, $[\mathbf{x} + \mathbf{y}]_d = [\mathbf{x}]_d + [\mathbf{y}]_d$.
- Multiplicative: For any $d_1, d_2 \geq k - 1$ such that $d_1 + d_2 < n$ and $\mathbf{x}, \mathbf{y} \in \mathbb{F}^k$, $[\mathbf{x} * \mathbf{y}]_{d_1+d_2} = [\mathbf{x}]_{d_1} * [\mathbf{y}]_{d_2}$ where $*$ is component-wise multiplication.

Note that the second property implies that for all $k - 1 \leq d \leq n - k$, a degree- d packed Shamir secret sharing is *multiplication-friendly*. What we mean by this is that for all $\mathbf{x}, \mathbf{c} \in \mathbb{F}^k$, all parties can locally compute $[\mathbf{c} * \mathbf{x}]_{d+k-1}$ from $[\mathbf{x}]_d$ and public vector $\mathbf{c}$. To do this, all parties just locally transform $\mathbf{c}$ to some canonical degree- $(k - 1)$ packed Shamir sharing $[\mathbf{c}]_{k-1}$ and then use the $*$ operation.

Recall that t is the number of corrupted parties and that a degree- d packed Shamir secret sharing is private against an adversary that holds $d - k + 1$ shares. To ensure that the Shamir secret sharing is both private and multiplication-friendly, we choose k such that $t \leq d - k + 1$ and $d \leq n - k$. When $d = n - k$ and $k = (n - t + 1)/2$, both requirements hold and k is maximal.

2.1 Packed Beaver Triples with $\tilde{O}(1)$ Communication Per Triple

A packed beaver triple consists of the sharings $[\mathbf{a}]_{n-k}$, $[\mathbf{b}]_{n-k}$, and $[\mathbf{c}]_{n-k}$ such that $\mathbf{a}$ and $\mathbf{b}$ are fully random, while $\mathbf{c} = \mathbf{a} * \mathbf{b}$. We now show to generate such packed beaver triples with $\tilde{O}(1)$ communication per underlying triple.

Attempt 1: Packed Beaver Triples using Super-Invertible Matrices.

First, we will attempt to generate such packed beaver triples using *super-invertible* matrices and OLE correlations. Although we will fail in this attempt, it will provide intuition for our ultimate construction.

Recall that an $m \times n$ super-invertible matrix $\mathbf{M}$ satisfies that, for all $C \subseteq [n]$ such that $|C| = m$, the sub-matrix $\mathbf{M}^C$, corresponding to the columns of $\mathbf{M}$ with indices in C , is invertible. Super-invertible matrices are commonly used to generate several random packed Shamir sharings $[\mathbf{a}_1]_{n-k}, \dots, [\mathbf{a}_{n-t}]_{n-k}$ with $O(n)$ amortized communication per sharing, and thus $O(1)$ communication per underlying value. To do so, each party P_i begins by sampling their own random sharing $[\mathbf{u}_i]_{n-k}$ and distributing to the other parties their shares. Then, using a $(n-t) \times n$ super-invertible matrix $\mathbf{M}$, the parties simply compute $([\mathbf{a}_1]_{n-k}, \dots, [\mathbf{a}_{n-t}]_{n-k})^\top \leftarrow \mathbf{M} \cdot ([\mathbf{u}_1]_{n-k}, \dots, [\mathbf{u}_n]_{n-k})^\top$. Intuitively, for any set of honest parties of size $n-t$, $H = \{h_1, \dots, h_{n-t}\} \subseteq \text{Hon}$, letting $C = \{c_1, \dots, c_t\} = [n] \setminus \text{Hon}$, we can write

$$([\mathbf{a}_1]_{n-k}, \dots, [\mathbf{a}_{n-t}]_{n-k})^\top = \mathbf{M}^H \cdot ([\mathbf{u}_{h_1}]_{n-k}, \dots, [\mathbf{u}_{h_{n-t}}]_{n-k})^\top + \mathbf{M}^C \cdot ([\mathbf{u}_{c_1}]_{n-k}, \dots, [\mathbf{u}_{c_t}]_{n-k})^\top.$$

Since $\mathbf{M}$ is super-invertible, $\mathbf{M}^H$ must be invertible, and thus, given any $([\mathbf{u}_{c_1}]_{n-k}, \dots, [\mathbf{u}_{c_t}]_{n-k})$, $([\mathbf{a}_1]_{n-k}, \dots, [\mathbf{a}_{n-t}]_{n-k})$ and $([\mathbf{u}_{h_1}]_{n-k}, \dots, [\mathbf{u}_{h_{n-t}}]_{n-k})$ are distributed equivalently. Since honest parties generated the latter randomly and unknown to the adversary, the former must also be random and unknown to the adversary. The cost of each party sharing their $[\mathbf{u}_i]_{n-k}$ is $O(n)$ for a total cost of $O(n^2)$. Since $n-t = \Omega(n)$, the amortized cost is $O(n)$ per packed sharing.

The parties can repeat the same process as above, sharing random $([\mathbf{v}_1]_{n-k}, \dots, [\mathbf{v}_n]_{n-k})$ and multiplying them by $\mathbf{M}$ to obtain random $([\mathbf{b}_1]_{n-k}, \dots, [\mathbf{b}_{n-t}]_{n-k})$ with the same cost. Now, observe that for any $l \in [n-t]$,

$$\begin{aligned} c_l &:= a_l * b_l = \left(\sum_{i=1}^n M_l[i] \cdot u_i \right) * \left(\sum_{j=1}^n M_l[j] \cdot v_j \right) \\ &= \sum_{i=1}^n M_l[i] \cdot \left(\sum_{j=1}^n M_l[j] \cdot (u_i * v_j) \right). \end{aligned} \tag{1}$$

One good start to getting the sharing $[c_l]_{n-k}$ could be to securely compute $u_i * v_j$, for every pair of ordered parties (P_i, P_j) . Thankfully, this is exactly what the (programmable) OLE functionality $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ gives us [RS22]. $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ takes as input u_i from P_i and v_j from P_j . It next samples random α_i^j , computes $\beta_j^i \leftarrow u_i * v_j - \alpha_i^j$, then gives α_i^j to P_i and β_j^i to P_j .

However, now we are faced with a dilemma. We could have every P_i distribute packed Shamir secret sharings of $[\alpha_i^j]_{n-k}$ and $[\beta_j^i]_{n-k}$, for each other party P_j . Unfortunately, this would cost $\Omega(n^3)$ communication in total, which is too much.

We could also have each party P_i compute

$$\mathbf{c}_l^i \leftarrow (\mathbf{M}_l[i])^2 \cdot \mathbf{u}_i * \mathbf{v}_i + \mathbf{M}_l[i] \cdot \sum_{j=1}^n \mathbf{M}_l[j] \cdot (\alpha_i^j + \beta_i^j)$$

and then share $[\mathbf{c}_l^i]_{n-k}$. However, for every $l \in [n-t]$, this value $\mathbf{c}_l^i$ is different. Thus, P_i needs to distribute such a sharing for each $l \in [n-t]$ costing $\Omega(n^3)$ total communication that is too large. Indeed, although super-invertible matrices are good extractors for purely random sharings, computing packed beaver triples in this way is too expensive. Intuitively, this is because known constructions of super-invertible matrices, such as Vandermonde matrices, are *dense*. For *every* row of $\mathbf{M}$, *every* entry could be non-zero. This means that *every* $\mathbf{c}_l$ is computed from *every* party P_i 's $\mathbf{u}_i$ and $\mathbf{v}_i$. Moreover, each row $\mathbf{M}_l$ is different (which is necessary for super-invertibility) meaning that each $\mathbf{c}_l$ is computed in a different way from the $\mathbf{u}_i$ and $\mathbf{v}_i$ vectors. Thus, if we want to compute packed beaver triples in the above manner, we need *sparse* matrices $\mathbf{M}$. This leads us to our first key technical contribution of weakly super-invertible matrices.

Weakly Super-Invertible Matrices. In the above, the super-invertible matrix $\mathbf{M}$ ensured that $\mathbf{M}^H$ has full rank. So, the distribution of each element in $([\mathbf{a}_1]_{n-k}, \dots, [\mathbf{a}_{n-t}]_{n-k})$ is random and hidden from the adversary. This strong property is quite useful in MPC, but comes at the cost of large communication for our proposed packed beaver triple construction above. The core reason is due to the density (large number of non-zero-entries) of super-invertible matrices.

Instead, we take a different approach. Suppose that $\mathbf{M}^H$ has high (but not necessarily full) rank. For concreteness, say the rank was $\gamma \cdot (n-t)$ for some constant $0 < \gamma < 1$ as opposed to full rank $n-t$. Then, it will be guaranteed that a γ -fraction of $([\mathbf{a}_1]_{n-k}, \dots, [\mathbf{a}_{n-t}]_{n-k})$ will be random from the adversary's view. However, the bad $(1-\gamma)$ -fraction is unknown since the corrupted parties are unknown (and could change later with an adaptive adversary). We will show that this suffices for our protocols using another level of extraction when $\gamma > 1/2$.

For this weaker requirement, we construct $m \times n$ sparse matrices with only a small number of non-zero entries. We present a simple construction where for each column, $\zeta = \Theta(\log |\mathbb{F}|)$ rows are chosen uniformly at random. These ζ entries per column are set to be a uniformly random non-zero entry. The remaining $m - \zeta$ entries are set to 0. The total number of non-zero entries is $O(n \log |\mathbb{F}|)$. For our MPC constructions, we use fields of size $|\mathbb{F}| = O(n + 2^\lambda)$. So, each column and row has $O(\lambda + \log n)$ non-zero entries (except with negligible probability). To prove weakly super-invertibility, we compute the number of vectors in the null space of the corresponding sub-matrix. We show that the null space will not contain any vectors with more than $O(\lambda + \log m)$ non-zero entries except with negligible probability implying that the sub-matrix has large rank.

Finally, we present concrete matrix instantiations with $3 \leq \zeta \leq 8$ and $\gamma \in (1/2, 2/3]$ using relations to prior cuckoo hashing experiments [CLR17, PSZ18] and robust cuckoo hashing [Yeo23a] that we conjecture are weakly super-invertible. We also run additional experiments providing further evidence of weak super-

invertibility (see Section 5.3 and Appendix B.5). This gives us total number of non-zero entries at most $8n$.

Two-level Extraction using Weakly Super-Invertible Matrices and [CP17].

Equipped with *weakly* super-invertible matrices, we can revisit our attempt to generate packed beaver triples from above. Indeed, we can still try to extract $[\mathbf{a}_1]_{n-k}, \dots, [\mathbf{a}_{n-t}]_m$ and $[\mathbf{b}_1]_{n-k}, \dots, [\mathbf{b}_m]_{n-k}$ from $[\mathbf{u}_1]_{n-k}, \dots, [\mathbf{u}_n]_{n-k}$ and $[\mathbf{v}_1]_{n-k}, \dots, [\mathbf{v}_n]_{n-k}$, respectively, using *weakly* super-invertible $\mathbf{M}$, where $m = \Theta(n)$ (this does not fully work, as mentioned above). Now, for each $l \in [m]$, Equation 1 becomes

$$\begin{aligned} \mathbf{c}_l &:= \mathbf{a}_l * \mathbf{b}_l = \left(\sum_{i \in [n]: \mathbf{M}_l[i] \neq 0} \mathbf{M}_l[i] \cdot \mathbf{u}_i \right) * \left(\sum_{j \in [n]: \mathbf{M}_l[j] \neq 0} \mathbf{M}_l[j] \cdot \mathbf{v}_j \right) \\ &= \sum_{i \in [n]: \mathbf{M}_l[i] \neq 0} \mathbf{M}_l[i] \cdot \left(\sum_{j \in [n]: \mathbf{M}_l[j] \neq 0} \mathbf{M}_l[j] \cdot (\mathbf{u}_i * \mathbf{v}_j) \right). \end{aligned}$$

Therefore, for each $l \in [m]$ we only need to invoke $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ between every ordered pair of parties (P_i, P_j) such that $\mathbf{M}_l[i] \neq 0$ and $\mathbf{M}_l[j] \neq 0$ to receive α_i^j and β_j^i , respectively, such that $\alpha_i^j + \beta_j^i = \mathbf{u}_i * \mathbf{v}_j$. Based on this, each P_i such that $\mathbf{M}_l[i] \neq 0$ can compute

$$\mathbf{c}_l^i \leftarrow (\mathbf{M}_l[i])^2 \cdot \mathbf{u}_i * \mathbf{v}_i + \mathbf{M}_l[i] \cdot \sum_{j \in [n] \setminus \{i\}: \mathbf{M}_l[j] \neq 0} \mathbf{M}_l[j] \cdot (\alpha_i^j + \beta_j^i)$$

and then share $[\mathbf{c}_l^i]_{n-k}$. Then, all parties compute $[\mathbf{c}_l]_{n-k} \leftarrow \sum_{i \in [n]: \mathbf{M}_l[i] \neq 0} [\mathbf{c}_l^i]_{n-k}$.

Since the row norm of $\mathbf{M}$ is $O(\lambda)$ with all-but-negligible probability, we get the following efficiency benefit. First, we only need $O(\lambda^2 \cdot m)$ invocations of $\mathcal{F}_{\text{OLE}}^{\text{prog}}$. Second, each party P_i 's column of $\mathbf{M}$ only has ζ non-zeros, and thus P_i only needs to share $[\mathbf{c}_l^i]_{n-k}$ for ζ such l . Thus, this only costs $O(n^2 \cdot \zeta)$ total communication. Therefore, in total this is $\tilde{O}(n^2)$ for $O(n)$ packed triples, or $\tilde{O}(1)$ per underlying triple, which was our goal!

However, as noted above, since $\mathbf{M}$ is only *weakly* super-invertible, the extracted values $\mathbf{a}_l, \mathbf{b}_l$ may not be fully random. Indeed, we are only guaranteed that some $\gamma \cdot m$ of them are fully random, but we do not know which ones. Furthermore, we, of course, do not know the identities of the honest parties. Thus, the protocol up until this point is only our *first level* of extraction.

To obtain fully random packed beaver triples, we need a *second level* of extraction. We base this on the “triple extraction” protocol of [CP17]. This will allow us to extract $\mu := \gamma \cdot m - (m + 1)/2$ random triples via the m triples obtained from the first level of extraction, some unknown $\gamma \cdot m$ of which were random. Since $\gamma > 1/2$, we have $\mu = \Omega(m) = \Omega(n)$. Briefly speaking, in this technique, the parties convert their current packed triples into m evaluations on vectors of polynomials $\mathbf{A}(\cdot), \mathbf{B}(\cdot), \mathbf{C}(\cdot)$, such that $\mathbf{A}(\cdot) * \mathbf{B}(\cdot) = \mathbf{C}(\cdot)$. A large fraction of these evaluations are random, corresponding to the randomness of the

triples from the first level of extraction, and thus using the degrees of freedom of the polynomials, we can use μ other evaluations as our output packed triples. See section 6 for more details. The only communication required by the second level of extraction is from $O(n)$ invocations of the standard packed beaver multiplication procedure, each of which costs $O(n)$. So, the whole second level costs $O(n^2)$. Therefore, our full packed beaver triple protocol costs $\tilde{O}(n^2)$ communication for $\mu = \Omega(n)$ packed beaver triples. This is $\tilde{O}(1)$ communication per underlying triple, and thus we have achieved our goal.

Comparison with Bracha Committees. One can view the first level of extraction using weakly super-invertible matrices as a concretely-efficient version of using Bracha committees for packed triple generation. Importantly, we avoid the impractical MPC protocol emulation required by the use of Bracha committees. As each of the columns of the matrix $\mathbf{M}$ correspond to a real party, one can view the rows of $\mathbf{M}$ as committees. Each party is assigned to ζ random committees. In the Bracha committee approach, it is only required that a constant fraction greater than $1/2$ of the committees contain at least one honest party. Weakly super-invertible matrices have the stricter requirement that the sub-matrix $\mathbf{M}^H$ corresponding to honest parties' columns has rank $\gamma \cdot m$, for $\gamma > 1/2$.⁴

2.2 More Efficient Dishonest Majority MPC Offline Phase

As mentioned earlier, the main bottleneck in instantiating the offline phase of SuperPack (and Sharing Transformation) is generating packed beaver triples. Another challenge is efficiently generating *authenticated* packed sharings required for malicious security in the online phase. Once we have this, the rest of the offline phase can be instantiated using standard techniques (presented in Section D for completeness); the same holds for Sharing Transformation. Next, we will show that authenticating packed sharings can be done using packed beaver triples.

Authenticated Sharings with $\tilde{O}(1)$ Communication per Value. In the online phase of SuperPack, all circuit values will be shared using degree- $(n - k)$ *authenticated* packed Shamir sharings $\llbracket \mathbf{v} \rrbracket_{n-k} := ([\mathbf{v}]_{n-k}, [\Delta \cdot \mathbf{v}]_{n-k})$, where Δ is uniformly random. This Δ is referred to as the “MAC key”, while the sharing $[\Delta \cdot \mathbf{v}]_{n-k}$ is referred to as the (information-theoretic) “MAC” on $\mathbf{v}$. Intuitively, if $\mathbf{v}$ is opened then it will be checked by opening $\Delta \cdot \mathbf{v}$ —since Δ is random, with all-but-negligible probability, the adversary will be unable to force an opening to some $\mathbf{v}' \neq \mathbf{v}$.⁵

In the offline phase, we have to generate such authenticated packed Shamir sharings. First, we show that generating authenticated packed Shamir sharings of *random* vectors can be done using packed beaver triples. We again assume a semi-honest adversary in the offline phase, for now.

Assume that we have the sharing $[\Delta \cdot \mathbf{1}]_{n-k}$, where $\mathbf{1} = (1, \dots, 1) \in \mathbb{F}^k$, with some random Δ (this can be generated using standard techniques) and some

⁴ As an extreme example, the sub-matrix $\mathbf{M}^H$ of all ones would suffice for Bracha committees, but is clearly insufficient for weakly super-invertible matrices.

⁵ This is done once for many such openings at a time using random linear combinations.

packed beaver triple $([r]_{n-k}, [s]_{n-k}, [r * s]_{n-k})$. First, the parties can compute $[\Delta \cdot \mathbf{1} + s]_{n-k} \leftarrow [\Delta \cdot \mathbf{1}]_{n-k} + [s]_{n-k}$ and open their shares to P_1 . Since s is random, nothing about Δ is revealed. P_1 can then reconstruct $\Delta \cdot \mathbf{1} + s$, compute the (unique) packed Shamir sharing $[\Delta \cdot \mathbf{1} + s]_{k-1}$, and distribute to the rest of the parties their shares. Finally, all parties can compute $[\Delta \cdot r]_{n-1} \leftarrow [r]_{n-k} * [\Delta \cdot \mathbf{1} + s]_{k-1} - [r * s]_{n-k}$. Observe that the parties opening their shares of $[\Delta \cdot \mathbf{1} + s]_{n-k}$ to P_1 , and then P_1 distributing the shares of $[\Delta \cdot \mathbf{1} + s]_{k-1}$ each cost $O(n)$ communication. So, producing $[\Delta \cdot r]_{n-1}$ costs additional $O(1)$ communication per underlying value. Using standard techniques, the parties can obtain a sharing $[\Delta * r]_{n-k}$ with reduced degree for the same cost (see Section D).

Once we have authenticated packed Shamir sharings $\llbracket r \rrbracket_{n-k}$ of random vectors r , we can produce authenticated packed Shamir sharings $\llbracket x \rrbracket_{n-k}$ of any vector x (including components of packed beaver triples) efficiently using standard techniques. We refer the reader to Section D for more details.

Malicious Security. For malicious security of the offline phase, we can use several standard techniques that we will instantiate in Section D. One primary opportunity for the corrupted parties to deviate from the protocol is in generating the packed beaver triples. For example, a corrupt party P_j can input to $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ different vectors $\bar{u}_j, \bar{v}_j$ than those underlying their sharings $[u]_{n-k}, [v]_{n-k}$. The adversary can use this and other methods so that the output authenticated packed beaver triples will actually be of the form $(\llbracket a \rrbracket_{n-k}, \llbracket b \rrbracket_{n-k}, \llbracket a * b + \delta \rrbracket_{n-k})$, for some δ . To handle this, the parties can use the standard *triple sacrificing* technique [DPSZ12] which takes as input two packed beaver triples $(\llbracket a_1 \rrbracket_{n-k}, \llbracket b_1 \rrbracket_{n-k}, \llbracket a_1 * b_1 + \delta_1 \rrbracket_{n-k})$ and $(\llbracket a_2 \rrbracket_{n-k}, \llbracket b_2 \rrbracket_{n-k}, \llbracket a_2 * b_2 + \delta_2 \rrbracket_{n-k})$, then outputs the first if $\delta_1 = \delta_2 = \mathbf{0}$. This standard procedure uses only $O(1)$ overhead per underlying triple.

3 Preliminaries

We now present some important preliminaries. For additional preliminaries on additive secret sharing and basic ideal functionalities $\mathcal{F}_{\text{commit}}$ and $\mathcal{F}_{\text{coin}}$ that we will use in our MPC protocol, see Appendix A of the Supplementary Material.

Linear Algebra. We denote a k -dimensional vector as $v = (v_1, \dots, v_k)^\top \in \mathbb{F}^k$. Given two k -dimensional vectors u, v , we use $u * v$ to denote component-wise multiplication of the vectors.

For any $m \times n$ matrix M , we denote its i -th row vector as M_i for any $i \in [m]$. The entry in the i -th row and j -th column of M is denoted by $M_i[j]$ for any $i \in [m]$ and $j \in [n]$. We will denote the j -th column vector of M as M^j for any $j \in [n]$. For convenience, we will overload these operators to consider subsets of rows or columns of a matrix. For example, let $C = \{c_1, \dots, c_\ell\} \subseteq [n]$ be any subset of columns and we define the following $m \times |S|$ sub-matrix $M^C = [M^{c_1}, \dots, M^{c_\ell}]$ consisting of all column vectors of M in the subset of columns C . Finally, we can consider sub-matrices of M that are defined by arbitrary subsets of both rows and columns. For any subset of rows $R = \{r_1, \dots, r_k\} \subseteq [m]$ and subset of

columns $C = \{c_1, \dots, c_\ell\} \subseteq [n]$, we define the following $k \times \ell$ sub-matrix

$$\mathbf{M}_R^C = \begin{bmatrix} \mathbf{M}_{r_1}[c_1] & \dots & \mathbf{M}_{r_1}[c_\ell] \\ \dots & \dots & \dots \\ \mathbf{M}_{r_k}[c_1] & \dots & \mathbf{M}_{r_k}[c_\ell] \end{bmatrix}$$

consisting all entries that appear in one of the rows denoted by R and one of the columns denoted by C . We will use $\mathbf{M}^\top$ to denote the transpose matrix of $\mathbf{M}$.

MPC Preliminaries. In this paper, we focus on using MPC with n parties $P_1, \dots, P_n$ to compute functionalities $\mathcal{F}$ that can be represented as arithmetic circuits over a finite field $\mathbb{F}$ with input, addition, multiplication, and output gates. We use κ to denote the computational security parameter, λ to denote the statistical security parameter, C to denote the circuit, and $|C|$ for the number of multiplication gates in the circuit. In this work, we assume that the field size is $|\mathbb{F}| \geq 2^\lambda$. Note that this implies that $|F| \geq |C| + n$, since both the number of parties and circuit size are bounded by $\text{poly}(\lambda)$.

We consider an adversary $\mathcal{A}$ that can corrupt at most t parties adaptively and maliciously. We study the $t < (1 - \varepsilon) \cdot n$ setting, where ε is any constant $0 < \varepsilon \leq 1/2$. Such an adversary receives all messages sent to currently corrupted parties and based on these can *adaptively* corrupt more parties, as long as the number of corrupted parties does not exceed t . In addition, the adversary can *maliciously* arbitrarily deviate from the protocol.

Ultimately, we will study MPC protocols in the offline-online setting in which there is an offline phase that is *circuit-independent* and is *computationally-secure* with respect to κ , followed by an online setting that can be *statistically-secure* with respect to λ . In this paper, we instantiate the offline phase of an MPC protocol. However, we will assume the presence of an ideal functionality which can prepare circuit-independent correlated randomness ((V)OLE correlations). Therefore, we can still prove our protocols *statistically-secure*.

We use the Universal Composability (UC) framework of [Can01] to formally prove security of our protocol. See Section A for more details on this framework.

Packed Shamir Secret Sharings. In our work, we will use *packed Shamir secret sharing*, introduced by [FY92] and explained in Section 2. For more details on packed Shamir secret sharings, see Section A. In Section C, we adapt the common approach (also used by SuperPack) of using pseudorandom generators (PRGs) to reduce the communication complexity when distributing secret sharings [LN17, NV18, BBC⁺19, GLO⁺21, GSY21, RS22, EGP⁺23]. Of particular note to our packed triple protocol, distributing random packed sharings of the form $[\mathbf{r}]_{n-k}$ only costs $k - 1 \leq (n - t)/2$ elements with a PRG and distributing packed sharings of the form $[\mathbf{x}]_{n-k}$ for fixed $\mathbf{x}$ only costs $n - t$ elements.

4 Revisiting Invertible Matrices

Many prior MPC works have utilized matrices that emit certain strong invertibility properties. These matrices have been found in applications as sub-routines of

MPC protocols such as critically being able to extract many random sharings from some base random sharings (some of which are adversarially generated).

Super-Invertible Matrices. First defined by Hirt and Nielsen [HN06], super-invertible matrices are any matrix $\mathbf{M}$ of dimension $m \times n$ where the number of columns is at least as large as the number of rows (that is, $n \geq m$). The matrix $\mathbf{M}$ is super-invertible if every sub-matrix of dimension $m \times m$ is invertible. In other words, for any subset of m columns, the sub-matrix consisting of the m chosen columns should have full rank. We present the formal definition below:

Definition 1 (Super-Invertible Matrices [HN06]). *Let $\mathbf{M}$ be a matrix of $m \times n$ dimension with $n \geq m$. $\mathbf{M}$ is a super-invertible matrix if, for any subset $C = \{c_1, \dots, c_m\} \subseteq [n]$ of size exactly m , the $m \times m$ sub-matrix $\mathbf{M}^C$, defined as consisting of the m column vectors of $\mathbf{M}$ in the subset C , must be invertible.*

Hyper-Invertible Matrices. Hyper-invertible matrices were introduced by Beerliová-Trubíniová and Hirt [BTH08]. In super-invertible matrices, it was only required that every $m \times m$ sub-matrix was full rank m . For hyper-invertible matrices, the invertibility requirement is extended to apply for every square sub-matrix. In particular, for a $m \times n$ matrix $\mathbf{M}$ to be hyper-invertible, it must be that every $z \times z$ sub-matrix of $\mathbf{M}$ must be invertible for every choice of $1 \leq z \leq m$. In other words, for every $1 \leq z \leq m$ and every choice of subsets $R \subseteq [m]$ and $C \subseteq [n]$ both of exactly size z , the sub-matrix $\mathbf{M}_R^C$ consisting of the rows in R and columns in C must be invertible (have full rank z). See Appendix B.3 for the definition.

Existing Constructions. Both super-invertible and hyper-invertible matrices have been used extensively in multiple MPC protocols. In general, both of these matrices have been instantiated using Vandermonde matrices that can be shown to satisfy the requirements of being hyper-invertible (and, thus, also being super-invertible). We refer readers to prior works (such as [HN06, BTH08]) for more details on Vandermonde matrices being super- and hyper-invertible. However, we point out that Vandermonde matrices of dimension $m \times n$ will consist of $\Omega(m \cdot n)$ non-zero entries that has a direct impact on the communication efficiency of MPC protocols. To our knowledge, we are unaware of any super-invertible or hyper-invertible matrices with $o(m \cdot n)$ non-zero entries. In fact, we show hyper-invertible matrices with $o(m \cdot n)$ non-zero entries cannot exist later (see Appendix B.4).

5 Weakly Super-Invertible Matrices

In this section, we present new, weaker notions of super-invertible matrices with less restrictive requirements compared to super-invertible matrices. We later show that the benefit of considering weaker invertibility notions is that we obtain constructions with a significantly smaller number of non-zero entries that can improve the efficiency when used in MPC applications. We also present concrete instantiations of these matrices using relations to three-way cuckoo hashing.

5.1 Definition

Our new definitions of $m \times n$ *weakly super-invertible matrices* will weaken definitions in two ways. First, we will no longer require $m \times m$ sub-matrices to be invertible (that is, equivalent to being full rank). Instead, we will only require that sub-matrices of interest will have sufficiently high rank (but not necessarily full rank). We will desire that $m \times h$ sub-matrices should have rank sufficiently close to m for sufficiently large $h = \Omega(m)$. In particular, we will require that the rank of each $m \times h$ sub-matrix should have rank $\gamma \cdot m$ for some constant $\gamma > 1/2$. The requirement of $\gamma > 1/2$ will be important for our MPC applications later.

Secondly, we will consider probabilistic guarantees for matrices that are randomly generated. To do this, we will consider random matrix families $\mathcal{M}$ equipped with an algorithm to randomly generate some matrix from the family. At a high level, weakly super-invertible matrix families will require that, for any randomly generated matrix $m \times n$, $\mathbf{M}$, from the family $\mathcal{M}$, every $m \times h$ sub-matrix of $\mathbf{M}$ for sufficiently large $h = \Omega(m)$ should have rank at least $\gamma \cdot m$ for some constant $\gamma > 1/2$ except with negligible probability where the randomness is over the choice of the original generated matrix from the family $\mathcal{M}$.

Matrix Families. To define weakly super-invertible matrix families, we start by defining the notion of a matrix family $\mathcal{M}$ that is equipped with a randomized algorithm to generate random matrices from the family $\mathcal{M}$. All matrices in the family $\mathcal{M}$ will have the same dimension of $m \times n$ and every family $\mathcal{M}$ will be defined with respect to some fixed field $\mathbb{F}$. We define a matrix family as follows:

Definition 2 (Matrix Family). *Let $\mathbb{F}$ be some field. A matrix family of $m \times n$ matrices is the tuple $\mathcal{M} = (\mathcal{S}, \text{Gen})$ satisfying the following:*

- $\mathcal{S} \subseteq \mathbb{F}^{m \times n}$ is a subset of all $m \times n$ matrices.
- $\mathbf{M} \leftarrow \text{Gen}(R)$ takes a random string R as input and outputs a matrix $\mathbf{M} \in \mathcal{S}$.

Weakly Super-Invertible Matrix Families. Using matrix families, we can now define our new notion of weakly super-invertible matrix families that extends the super-invertibility notion by Hirt and Nielsen [HN06].

In our definition, we consider the random process where a random matrix $\mathbf{M} \leftarrow \text{Gen}(R)$ is generated from a matrix family $\mathcal{M}$ for a uniformly random chosen R . Afterwards, pick any subset of m columns $C \subseteq [n]$ and consider the sub-matrix $\mathbf{M}^C$ consisting of the m column vectors of $\mathbf{M}$ denoted by C . We say that a matrix family $\mathcal{M}$ is weakly super-invertible if every choice of $h = \Omega(m)$ columns C results in a sub-matrix $\mathbf{M}^C$ that has rank at least $\Omega(m)$ except with negligible probability over the random choice of the matrix $\mathbf{M}$.

Definition 3 (Weakly Super-Invertible Matrix Family). *Fix constants $0 < \delta, \gamma, \eta < 1$ such that $\eta \geq \gamma$. Let $\mathcal{M} = (\mathcal{S}, \text{Gen})$ be a matrix family over $m \times n$ matrices where $n \geq m$. $\mathcal{M}$ is (δ, γ, η) -weakly super-invertible if*

$$\Pr_{\mathbf{R}}[\exists C \subseteq [n] \mid \text{rank}(\mathbf{M}^C) < \gamma \cdot m, \mathbf{M} \leftarrow \text{Gen}(\mathbf{R}), |C| = \eta \cdot m] \leq \delta.$$

In other words, a weakly super-invertible matrix guarantees that no $m \times (\eta \cdot m)$ sub-matrix will have rank strictly less than $\gamma \cdot m$ except with probability δ . We note that the restriction of $\eta \geq \gamma$ is necessary as we want the sub-matrix to have rank at least $\gamma \cdot m$. Therefore, the number of columns in the sub-matrix must satisfy $\eta \cdot m \geq \gamma \cdot m$ implying $\eta \geq \gamma$.

Finally, we define the main notions of efficiency for matrix families that will be important in our MPC applications. In particular, we will desire that all the matrices in some matrix family $\mathcal{M}$ have a small number of non-zero entries. In particular, we want the guarantee that each row of $\mathcal{M}$ has a small number of non-zero entries (typically, much less than n). For the i -th row vector of $\mathbf{M}$, we define its L_0 norm as $L_0(\mathbf{M}_i) = |\{j \mid j \in [n], \mathbf{M}_i[j] \neq 0\}|$. We formally define this efficiency measure for weakly super-invertible matrix families as follows:

Definition 4 (($\delta, \gamma, \eta, \ell$)-Weakly Super-Invertible Matrix Family). *Let $\mathcal{M} = (\mathcal{S}, \text{Gen})$ be a matrix family over $m \times n$ matrices. The matrix family $\mathcal{M}$ is $(\delta, \gamma, \eta, \ell)$ -weakly super-invertible if the following holds:*

1. $\mathcal{M}$ is (δ, γ, η) -weakly super-invertible.
2. The probability that there exists any row vector with L_0 norm larger than ℓ is at most δ . In other words, $\Pr[\exists i \in [m] \mid L_0(\mathbf{M}_i) > \ell] \leq \delta$.

5.2 Sparse Weakly Super-Invertible Matrices

We present sparse weakly super-invertible $m \times n$ matrices over any finite field $\mathbb{F}$ with $O(n \log |\mathbb{F}|)$ non-zero entries. For constant-sized fields $|\mathbb{F}| = O(1)$, $\mathcal{M}_{\zeta \text{col}}$ has $O(n)$ non-zero entries that we will later show is asymptotically optimal. For our MPC applications, we will consider larger fields such that the L_0 norm of $\mathcal{M}_{\zeta \text{col}}$ is optimal except for the $O(\log |\mathbb{F}|)$ factor. To our knowledge, all previously known super-invertible matrices have $\Theta(m \cdot n)$ non-zero entries (see Section 4).

Construction. First, we will fix any finite field $\mathbb{F}$. We start by presenting our construction of the matrix family $\mathcal{M}_{\zeta \text{col}} = (\mathcal{S}_{\zeta \text{col}}, \text{Gen}_{\zeta \text{col}})$ whose matrix generation algorithm is defined as follows:

Gen $_{\zeta \text{col}}(\mathbf{R})$:

1. Initialize $m \times n$ matrix $\mathbf{M}$ to be all zeroes.
2. For each $i = 1, \dots, n$:
 - (a) Use $\mathbf{R}$ to generate ζ integers $x_1^i, x_2^i, \dots, x_\zeta^i$ uniformly at random from $[m]$ and ζ non-zero integers $y_1^i, \dots, y_\zeta^i$ uniformly at random from $\mathbb{F} \setminus \{0\}$. Set $\mathbf{M}_{x_j^i}[i] = y_j^i$ for every $j \in \{1, 2, \dots, \zeta\}$.
3. Return $\mathbf{M}$.

In other words, $\mathcal{M}_{\zeta \text{col}}$ generates matrices by picking a uniformly random subset of ζ entries for each column vector and setting them to be a uniformly random non-zero element in $\mathbb{F}$. The subset $\mathcal{S}_{\zeta \text{col}}$ consists of all matrices with at most ζ non-zero entries in each of the columns and zero entries in all the remaining entries of each column. Note, it is possible that a column picks the same row twice. Therefore, there are at most ζ non-zero entries in each column. Next, we show that our construction, $\mathcal{M}_{\zeta \text{col}}$, is weakly super-invertible:

Theorem 4. *Pick any choice of constants $c \geq 1$, $0 < \gamma < 1$ and $\gamma < \eta \leq c$ as well as finite field $\mathbb{F}$ where $|\mathbb{F}| \geq 3$. Let the number of columns be $n = c \cdot m$ and the number of non-zero entries in each column be $\zeta = \Theta(\log |\mathbb{F}|)$. The $m \times n$ matrix family $\mathcal{M}_{\zeta \text{col}}$ is $(2^{-\lambda}, \gamma, \eta, O(\lambda + \log m))$ -weakly super-invertible over $\mathbb{F}$ for sufficiently large $m = \Omega(\lambda \log m)$.*

In this section, we will focus on fields where $|\mathbb{F}| \geq 3$. Note, the requirements of the field size $|\mathbb{F}|$ is less restrictive than standard MPC protocol requirements. For example, it is commonly required that $|\mathbb{F}| \geq n$ if n is the number of parties to enable (packed) Shamir secret sharing. Indeed, the number of columns n will correspond exactly to the number of parties in our MPC construction. Similarly, malicious MPC security commonly requires that each element of $\mathbb{F}$ is λ bits meaning that $|\mathbb{F}| \geq 2^\lambda$. While we use such large fields for our malicious MPC protocol, we show that constant field sizes $|\mathbb{F}| = O(1)$ are sufficient for weakly super-invertibility and fields of size $|\mathbb{F}| = O(n)$ are sufficient for semi-honest MPC. All deferred proofs (due to lack of space) may be found in Appendix B.1.

In Appendix B.2, we show our constructions in fact hold for all finite fields by showing that $\mathcal{M}_{\zeta \text{col}}$ remains weakly super-invertible when $|\mathbb{F}| = 2$. In this case, the proof differs and we rely on prior work for the XORSAT problem [PS16].

We start with efficiency proving the expected and worst case number of non-zero entries (L_0 norm) of each row by applying Chernoff bounds:

Theorem 5. *Let $\mathbf{M}$ be a random $m \times n$ matrix generated by $\mathcal{M}_{\zeta \text{col}}$ with ζ non-zero entries in each column. For all rows $i \in [m]$, $\mathbb{E}[L_0(\mathbf{M}_i)] = O(\zeta)$. Also, for some $\ell = O(\max(\zeta, \lambda + \log m))$, $\Pr[\exists i \in [m] \mid L_0(\mathbf{M}_i) > \ell] \leq 2^{-\lambda}$.*

Note, if one plugs in $\zeta = \Theta(\log |\mathbb{F}|)$, then we see that each row has expected $O(\log |\mathbb{F}|)$ non-zero entries. In the worst case, each row has at most $\ell = O(\max(\log |\mathbb{F}|, \lambda + \log m))$ non-zero entries. As discussed earlier, it is typically assumed that $|F| \geq n = \Omega(m)$ and $|F| \geq 2^\lambda$ for malicious security. Therefore, this means that $\ell = O(\lambda + \log m)$. The remainder of this section will be dedicated to proving that for $h = \eta \cdot m$, all $m \times h$ sub-matrices have rank at least $\gamma \cdot m$ except with probability $\delta \leq 2^{-\lambda}$.

Proof Overview of High Rank Sub-Matrices. We first present a high-level overview of our proof that will show that a randomly generated matrix from $\mathcal{M}_{\zeta \text{col}}$ will satisfy the necessary requirements of being weakly super-invertible except with probability $2^{-\lambda}$. Our proof will use the following steps. In the following, we will use “size” of a vector to refer to its number of non-zero entries (that is, the L_0 norm of the vector).

1. First, we consider vectors in the null space and show that the size of the null space can be used to lower bound the rank of sub-matrices.
2. We consider the case where $m \times h$ sub-matrices may contain null space vectors of small sizes. To analyze this case, we consider a combinatorial argument showing that null space vectors of size x satisfying $x = \Omega(\lambda + \log m)$ and $x = O(m/\zeta)$ do not exist except with negligibly small probability.

3. To show that larger null space vectors do not exist with all-but-negligible probability, we show that larger null space vectors are unlikely to emit linear combinations summing to the zero vector.
4. We put everything together to prove that $\mathcal{M}_{\zeta\text{col}}$ is weakly super-invertible.

Step 1: Null Space Vectors and Rank. To analyze our matrix construction, we will consider null space vectors. Consider any random $m \times n$ matrix $\mathbf{M}$ generated by $\mathcal{M}_{\zeta\text{col}}$. We say that a vector $(a_1, \dots, a_n) \in \mathbb{F}^n$ is in the null space of $\mathbf{M}$ if $a_1\mathbf{M}^1 + a_2\mathbf{M}^2 + \dots + a_n\mathbf{M}^n = \mathbf{0}$. For convenience, we will focus on the null space vectors' non-zero entries and their corresponding non-zero coefficients. Consider any subset of columns $C = \{i_1, \dots, i_x\}$ such that there exists some linear combination consisting of non-zero coefficients $a_1, \dots, a_x \in \mathbb{F} \setminus \{0\}$ such that their component-wise sum is the all-zero vector in the field $\mathbb{F}$; i.e., $a_1\mathbf{M}^{i_1} + a_2\mathbf{M}^{i_2} + \dots + a_x\mathbf{M}^{i_x} = \mathbf{0}$. In this phrasing, the corresponding null space vector consists of the subset of x columns C and the corresponding coefficients $a_1, \dots, a_x$. Therefore, the size (L_0 norm) of a null space vector is equal to the size of the column subset. The number of null space vectors can be used to directly bound the rank of the matrix $\mathbf{A}$ using the rank-nullity theorem:

Lemma 1. *Consider any matrix $\mathbf{A} \in \mathbb{F}^{m \times n}$ with $h \leq m$ and the number of null space vectors is Y . Then, the rank of $\mathbf{A}$ over finite field $\mathbb{F}$ is exactly $h - \log_{|\mathbb{F}|}(Y)$.*

We use Y^x to denote the number of null space vectors of size x with exactly x non-zero entries. We will break down the analysis into considering the number of null space vectors of size x , Y^x , for different sizes: $x \leq \beta \cdot m/\zeta = O(m/\zeta)$ and any $x = \Omega(m/\zeta)$. Each of the next two steps will handle each of these cases.

Step 2: Small Null Space Vectors. We will consider null space vectors of size at most $x \leq \beta \cdot m/\zeta$ for some constant $\beta > 0$ we will pick later. Consider any matrix $\mathbf{M}$ generated by the matrix family $\mathcal{M}_{\zeta\text{col}}$. We start by showing that $\mathbf{M}$ will not contain many null space vectors of size at most $x \leq \beta \cdot m/\zeta$. To do this, consider any fixed subset of x columns, $C \subseteq [n]$. Each column picks at most ζ rows to place non-zero entries. Consider all t rows that are chosen at least once across the x columns. If C is part of a null space vector, we show that each of the t rows must be chosen at least twice. Therefore, we immediately see that $t \leq \zeta x/2$ as a total of ζx rows are chosen across the x columns. Since $x \leq \beta \cdot m/\zeta$, it seems unlikely that each row is chosen at least twice as we show below:

Lemma 2. *Pick any constant $c \geq 1$. Let $n = c \cdot m$. Let $\mathbf{M}$ be a $m \times n$ matrix generated by $\mathcal{M}_{\zeta\text{col}}$. Then, there exists constant $0 < \beta < 1$ and $c_x = \Omega(\lambda + \log m)$ such that the probability there exists at least one null space vector of any size $c_x \leq x \leq \beta \cdot m/\zeta$ in $\mathbf{M}$ is at most $2^{-2\lambda}$ for sufficiently large $m = \Omega(\lambda)$.*

To summarize, if $\mathbf{M}$ is generated by $\mathcal{M}_{\zeta\text{col}}$, we show that $\mathbf{M}$ will not contain any null space vectors of size $c_x \leq x \leq \beta \cdot m/\zeta$ for some choice of $c_x = \Omega(\lambda + \log m)$ and constant $0 < \beta < 1$. Recall that our goal was to prove that any sub-matrix, $\mathbf{M}^C$, consisting of $h = \eta \cdot m$ columns $C \subseteq [n]$ of $\mathbf{M}$ has high rank. The above shows that, for any choice of h columns, the sub-matrix $\mathbf{M}^C$ also has no null

space vectors of size $c_x \leq x \leq \beta \cdot m/\zeta$. In particular, if $\mathbf{M}^C$ has a null space vector of size x , it is easy to see that it would also be a null space vector of the transpose of the originally generated matrix $\mathbf{M}$.

Step 3: Large Null Space Vectors. Next, we will consider the case of large null space vectors of size $x > \beta \cdot m/\zeta$. Note, the prior argument cannot scale to such large null space vectors with many columns. For example, if we consider $O(m)$ columns, it is likely that all m rows will be picked by at least two columns. So, the prior argument is not sufficient for large null space vectors.

Instead, we will use the following observation. Suppose that $\mathcal{M}_{\zeta\text{col}}$ generates a random $m \times n$ matrix $\mathbf{M}$. Consider any subset of x columns $C = \{i_1, \dots, i_x\} \subseteq [n]$ and pick arbitrary non-zero coefficients $a_1, \dots, a_x \in \mathbb{F} \setminus \{0\}$. Let $\mathbf{M}^C$ be the sub-matrix corresponding to the columns C and suppose there are t non-zero rows in $\mathbf{M}^C$. Furthermore, suppose that each row contains at least two non-zero entries. Consider the probability that C and $a_1, \dots, a_x$ will be a null space vector over the random choices of the generation of $\mathbf{M}$. Suppose the j -th row is a non-zero row vector, then we can consider the j -th entry of the linear combination: $a_1 \mathbf{M}^{i_1} + a_2 \mathbf{M}^{i_2} + \dots + a_x \mathbf{M}^{i_x}$. It is not hard to see that, since $a_1, \dots, a_x$ are non-zero and there are at least two non-zero entries in the j -th row, the probability that the j -th entry of the linear combination is 0 should be small. At first, one would guess that this probability would be exactly $1/(|\mathbb{F}| - 1)$. Surprisingly, we note that this probability changes and depends on the value of x (that is, the number of summands). As an example, consider $|\mathbb{F}| = 3$. For $x = 2$, the probability can easily be seen as $1/(|\mathbb{F}| - 1) = 1/2$. When $x = 3$, we note the probability becomes $1/4$.⁶ For $x = 4$, the probability becomes $3/8$. In other words, the probability changes with x and can either increase or decrease. Nevertheless, we prove the following lemma formalizing the above and showing the probability is never larger than $1/(|\mathbb{F}| - 1)$:

Lemma 3. *Consider any finite field $|\mathbb{F}| \geq 3$. For any $y \geq 1$, consider y random variables $X_1, \dots, X_y$ where each X_i are drawn uniformly at random from $\mathbb{F} \setminus \{0\}$. Then, $\Pr[X_1 + \dots + X_y = 0] \leq 1/(|\mathbb{F}| - 1)$.*

Indeed, the above is not true for $|\mathbb{F}| = 2$ as the sum of any even number of non-zero entries in such fields always sums to zero. In Appendix B, we show our results do extend to fields of size $|\mathbb{F}| = 2$. Now, we use the above to prove a result about null space vectors of large size relying upon Lemma 3:

Lemma 4. *Pick any constant $c \geq 1$ as well as any finite field $\mathbb{F}$ such that $|\mathbb{F}| \geq 3$ and parameter $\zeta = \Theta(\log |\mathbb{F}|)$. Let $n = c \cdot m$ and suppose $\mathbf{M}$ is a $m \times n$ matrix generated by $\mathcal{M}_{\zeta\text{col}}$. Then, for every constant $0 < \eta < 1$ and $0 < \beta < 1$, the probability there exists at least one null space vector of any size $\beta \cdot m/\zeta < x \leq \eta \cdot m$ in $\mathbf{M}$ is at most $2^{-2\lambda}$ for sufficiently large $m = \Omega(\lambda)$.*

⁶ Indeed, the only possibilities for terms $a_1 \mathbf{M}^{i_1}, a_2 \mathbf{M}^{i_2}, a_3 \mathbf{M}^{i_3}$ to sum to 0 in $\mathbb{F}_3$ are $(1, 1, 1)$ and $(2, 2, 2)$, out of 8 total (equally likely) possibilities.

To prove this, we start by upper bound probability that a single choice of subset of columns C and non-zero coefficients will be a null space vector (see Lemma 7 in Appendix B.1) before generalizing to all column subsets.

Step 4: Putting it All Together. Finally, we utilize all the results from the prior three steps to prove that $\mathcal{M}_{\zeta\text{col}}$ is a weakly super-invertible matrix family of dimension $m \times n$ where the number of non-zero entries in each columns is $\zeta = \Theta(\log |\mathbb{F}|)$. We note the prior steps have shown that there are no null space vectors of size more than $c_x = \Omega(\lambda + \log m)$. The last step essentially assumes the worst-case that all vectors of size at most c_x are null vectors. This gives us an upper bound on the null space. Finally, we apply Lemma 1 to obtain the rank. The proof of Theorem 4 may be found in Appendix B.1.

Optimality of $\mathcal{M}_{\zeta\text{col}}$. Our construction $\mathcal{M}_{\zeta\text{col}}$ is asymptotically optimal in terms of L_0 norm as each row vector of any weakly super-invertible matrix must have at least $\Omega(1)$ L_0 norm in expectation. This can be easily be seen by picking any $m \times h$ sub-matrix where $h = \Omega(m)$ and noting that this sub-matrix must have rank at least $\gamma \cdot m = \Omega(m)$. So, $\Omega(m)$ rows must have at least one non-zero entry. Our construction $\mathcal{M}_{\zeta\text{col}}$ with $|\mathbb{F}| = 3$ and $\zeta = O(1)$ has $O(m)$ non-zero entries matching this lower bound. Our MPC applications utilize $\mathcal{M}_{\zeta\text{col}}$ with larger fields $|\mathbb{F}| = O(n)$ and $|\mathbb{F}| = O(n + 2^\lambda)$ with $O(m \cdot (\log n))$ and $O(m \cdot (\lambda + \log n))$ non-zero entries respectively, that is optimal up to λ and $\log n$ factors.

Weakly Hyper-Invertible Matrices. One could also try to weaken the requirements for hyper-invertibility to obtain sparse matrices. In Appendix B.4, we show that a similar weakening for hyper-invertible matrices still requires $\Omega(n)$ L_0 norm for each row. In other words, there must be $\Omega(n \cdot m)$ non-zero entries in the entire matrix. So, Vandermonde matrices have optimal sparsity.

5.3 Practical Instantiations

In the previous section, we showed constructions that are asymptotically efficient with formally provable guarantees of weak super-invertibility. However, we are unaware of how to derive reasonable constants from the proofs for practical parameter settings. Instead, we show a connection between the weak super-invertibility guarantees of our matrix constructions and cuckoo hashing failure probabilities. Afterwards, we use prior experimental analysis of cuckoo hashing to obtain practical weakly super-invertible matrices from $\mathcal{M}_{\zeta\text{col}}$.

Connection with Robust Cuckoo Hashing. Consider our construction $\mathcal{M}_{\zeta\text{col}}$ resulting in a $m \times n$ matrix with ζ non-zero entries in each column. One can interpret this as a ζ -way robust cuckoo hashing instance [Yeo23a] with a universe of n items (each uniquely corresponding to a column) where $\eta \cdot n$ items may be chosen to be allocated into a table with m entries (each uniquely corresponding to a row). Each of the n possible items may be placed into ζ entries corresponding to the ζ non-zero entries per column. A cuckoo hashing instance is robust if there exists no subset of $\eta \cdot n$ items that cannot be allocated into the m entries except with negligible probability over the choice of the random ζ entry allocations

for each of the n items.⁷ From a ζ -way robust cuckoo hashing scheme, we show $\mathcal{M}_{\zeta\text{col}}$ will be weakly super-invertible where $m \times (\eta \cdot n)$ sub-matrices will have rank at least $\eta \cdot n$ for sufficiently large fields. See the proof in Appendix B.5.

Theorem 6. *Suppose there exists a robust cuckoo hashing scheme for a universe of n items and a table with m entries such that no subset of $\eta \cdot n$ items exists that cannot be allocated with failure probability p . Then, $\mathcal{M}_{\zeta\text{col}}$ resulting in a $m \times n$ matrix with ζ non-zero entries in each column will be weakly super-invertible with $\gamma = \eta n/m$ except with probability $p + 2^{-\Omega(\eta n - \log |\mathbb{F}|)}$.*

When the field size is $|\mathbb{F}| = 2^{\Omega(\eta \cdot n)}$, then the failure probability is negligible. While this seems large, it turns out to be a somewhat reasonable assumption for MPC applications where field sizes are $|\mathbb{F}| = \Omega(2^\lambda)$, but $\lambda \approx n$ in many practical settings. We conjecture that the failure probability is small even for smaller fields such as $|\mathbb{F}| \approx n$ (see empirical evidence in Appendix B.5).

Practical Instantiations. Using the above ideas, we will conjecture that $\mathcal{M}_{\zeta\text{col}}$ with small ζ results in a weakly super-invertible matrix with strong concrete guarantees and efficiency. Prior extensive experimental analysis [CLR17, PSZ18] has shown that three-way cuckoo hashing for n items and $m = 1.5n$ entries does not result in any observable allocation failures. This parameterization has been used in practical constructions of several cryptographic primitives including PSI [CLR17, PSZ18, CHLR18, GPR⁺21, CMdG⁺21] and batch PIR [ACLS18, MR23, Yeo23b, BPSY24]. As pointed out in [Yeo23a], many of these prior works implicitly assumed that three-way cuckoo hashing is robust where there exists no computationally efficient adversary that can find n items from a $\text{poly}(n)$ universe that cannot be allocated into a table with $m = 1.5n$ entries.

We present our concrete instantiations with small ζ . We consider $\mathcal{M}_{\zeta\text{col}}$ with parameters such that $m = \eta n$ and we want all $m \times (\eta \cdot n)$ square sub-matrices to have high rank. Using the same ideas from Theorem 6, we can conjecture that each of these square sub-matrices will have rank at least $2m/3$ such that $\gamma = 2/3$. In fact, we could do this formally for large fields $|\mathbb{F}| = 2^{\Omega(\epsilon n)}$ that is somewhat compatible with our MPC applications where $|\mathbb{F}| = \Omega(2^\lambda)$ and λ and n are comparable. However, we conjecture that $\mathcal{M}_{\zeta\text{col}}$ is weakly super-invertible for much smaller fields. In Appendix B.5, we present empirical evidence that this holds for fields as small as $|\mathbb{F}| \approx n$ and we also show how to choose ζ and γ to avoid robust cuckoo hashing attacks [Yeo23a].

Weakness of Cuckoo Hashing Reduction. In the above, we showed that one can relate weakly super-invertibility to hashing allocations. One may wonder why our analysis of our theoretical constructions do not reduce directly to cuckoo hashing. In Appendix B.6, we show that current, state-of-the-art cuckoo hashing analysis with provable guarantees are unable to obtain weakly super-invertible matrices with $\gamma > 1/2$ as needed for our MPC applications.

⁷ The original robust cuckoo hashing [Yeo23a] considers computational adversaries. For universes of n items, the statistical formulation is equivalent.

6 Our Packed Beaver Triple Protocol

We now present our efficient generation of random packed beaver triples. First, we present the ideal functionality $\mathcal{F}_{\text{triple}}$. Let $m \in \mathbb{N}$, $1/2 < \gamma < 1$, $\mu \leftarrow \gamma \cdot m - (m+1)/2$, and $\sigma \geq k$. $\mathcal{F}_{\text{triple}}$ generates random packed triple sharings with error, $([a_{l,w}]_{n-k}, [b_{l,w}]_{n-k}, [a_{l,w} * b_{l,w} + \delta_{l,w}]_{n-1})$, for $l \in [\mu]$, $w \in [\lceil \sigma/k \rceil]$, where the errors $\delta_{1,w}, \dots, \delta_{\mu,w}$ are input by the adversary. We explain the need to allow the adversary to input such errors below. Note that in the semi-honest case, these errors do not exist. These sharings will be consistent with shares for corrupted parties input by the adversary.

Functionality 1: $\mathcal{F}_{\text{triple}}$

Parameters: Let $m \in \mathbb{N}$, $1/2 < \gamma < 1$, and $\mu \leftarrow \gamma \cdot m - (m+1)/2$.

1. $\mathcal{F}_{\text{triple}}$ receives from the adversary the corrupted parties' shares, $\{(a_{1,w}^j, b_{1,w}^j, c_{1,w}^j), \dots, (a_{\mu,w}^j, b_{\mu,w}^j, c_{\mu,w}^j)\}_{j \in \text{Corr}, w \in [\lceil \sigma/k \rceil]}$, and errors $(\delta_{1,w}, \dots, \delta_{\mu,w})_{w \in [\lceil \sigma/k \rceil]}$.
2. $\mathcal{F}_{\text{triple}}$ samples random $((a_{1,w}, b_{1,w}), \dots, (a_{\mu,w}, b_{\mu,w}))_{w \in [\lceil \sigma/k \rceil]}$.
3. $\mathcal{F}_{\text{triple}}$ samples random sharings $(([a_{1,w}]_{n-k}, [b_{1,w}]_{n-k}, [a_{1,w} * b_{1,w} + \delta_{1,w}]_{n-1}), \dots, ([a_{\mu,w}]_{n-k}, [b_{\mu,w}]_{n-k}, [a_{\mu,w} * b_{\mu,w} + \delta_{\mu,w}]_{n-1}))_{w \in [\lceil \sigma/k \rceil]}$ such that for all $j \in \text{Corr}, w \in [\lceil \sigma/k \rceil]$, the j -th shares are $(a_{1,w}^j, b_{1,w}^j, c_{1,w}^j), \dots, (a_{\mu,w}^j, b_{\mu,w}^j, c_{\mu,w}^j)$.
4. $\mathcal{F}_{\text{triple}}$ then distributes to the honest parties their shares.

Required Sub-Functionalities and Procedures. Before presenting Π_{triple} below, we introduce some sub-functionalities and procedures that we will use. $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ is a two party functionality that generates several random correlations between the two parties, known as oblivious linear evaluation (OLE). More specifically, the two parties P_A and P_B input $\mathbf{u} \in \mathbb{F}^\sigma$ and $\mathbf{x} \in \mathbb{F}^\sigma$, respectively, to $\mathcal{F}_{\text{OLE}}^{\text{prog}}$, which samples random $\mathbf{v} \leftarrow_{\$} \mathbb{F}^\sigma$, and outputs $\mathbf{w} \leftarrow \mathbf{u} * \mathbf{x} - \mathbf{v}$ to P_A and $\mathbf{v}$ to P_B . Observe that $\mathbf{x}$ remains random to P_A and $\mathbf{u}$ remains random to P_B . See Section D for the formal definition. We will assume that $\sigma \geq k$.

$\mathcal{F}_{\text{verify-deg}}$ takes in several packed Shamir sharings which are supposed to be of degree- $(k-1)$. If any are not, it sends **abort** to the honest parties. This is accomplished in the usual way, by taking a random linear combination of the sharings, and then opening the resulting sharing to all parties, who check its degree. See Section D for more details.

Finally, we will actually need to perform packed beaver multiplication in Π_{triple} (using packed triples that may not be uniformly random). We do this in the standard procedure π_{beaver} , presented in Section D.

6.1 Packed Beaver Triples from Weakly Super-Invertible Matrices

Now we present Π_{triple} below that is parameterized by a family of $(\delta, \gamma, \eta, \ell)$ -weakly super-invertible $m \times n$ matrices, $\mathcal{F} = (\mathcal{S}, \text{Gen})$, where $\delta = \text{negl}(\lambda)$, $1/2 < \gamma < 1$, $2\epsilon n > m = \Omega(n)$, $1/(2\epsilon) > \eta = n - t > \epsilon \cdot n$, and $\ell = O(\lambda)$.

As discussed in Section 2, Π_{triple} proceeds via two ‘levels’ of extraction. In the first level, the parties extract random sharings $[\mathbf{a}]_{n-k}$ and $[\mathbf{b}]_{n-k}$ using a sampled weakly super-invertible matrix $\mathbf{M}$ on sharings $[\mathbf{u}_i]_{n-k}, [\mathbf{v}_i]_{n-k}$ sampled by each P_i . Then, in order to get the sharings $[\mathbf{c}]_{n-k}$, the ordered pairs (P_i, P_j) of parties such that $\mathbf{M}_l[i], \mathbf{M}_l[j] \neq 0$ for some row l invoke $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ to obtain α_i^j, β_j^i , respectively, such that $\alpha_i^j + \beta_j^i = \mathbf{u}_i * \mathbf{v}_j$. Using these values, each such P_i such that $\mathbf{M}_l[i] \neq 0$ can distribute a new sharing $[\mathbf{c}_l^i]_{n-k}$, from which the parties can compute $[\mathbf{c}]_{n-k}$. Since $\mathbf{M}$ is sparse, each party only has to do this $\tilde{O}(1)$ times.

Since $\mathbf{M}$ is only *weakly* super-invertible, the extracted values $\mathbf{a}_{l,w}, \mathbf{b}_{l,w}$ may not be fully random. Indeed, for each $w \in [\lceil \sigma/k \rceil]$, we are only guaranteed that some $\gamma \cdot m$ of them are fully random, but we do not know which ones. This leads us to our second level of extraction, which is based on the ‘triple extraction’ protocol of [CP17]. In this level of extraction, for every $w \in [\lceil \sigma/k \rceil]$, we are able to extract $\mu = \gamma \cdot m - (m+1)/2$ random packed beaver triples from the m packed triples output by the first level of extraction.

Protocol 1: Π_{triple}

Parameters: This protocol is parameterized by a $(\delta, \gamma, \eta, \ell)$ -weakly super-invertible $m \times n$ matrix, $\mathcal{F} = (\mathcal{S}, \text{Gen})$, where $\delta = \text{negl}(\lambda)$, $1/2 < \gamma < 1$, $2\varepsilon n > m = \Omega(n)$, $1/(2\varepsilon) > \eta = n - t > \varepsilon \cdot n$, and $\ell = O(\lambda)$. Let $\mu \leftarrow \gamma \cdot m - (m+1)/2$. We assume w.l.o.g. that m is odd.

Extraction Level 1:

1. Each party P_i first samples random $\mathbf{u}_i \leftarrow_{\$} \mathbb{F}^\sigma$ and $\mathbf{v}_i \leftarrow_{\$} \mathbb{F}^\sigma$.
2. Then all parties invoke $\mathcal{F}_{\text{coin}}$ to obtain random value $r \in \mathbb{F}$, interpret it as bit string $\mathbf{R}$, and locally compute $m \times n$ matrix, $\mathbf{M} \leftarrow \text{Gen}(\mathbf{R})$.
3. Then, for each $l \in [m]$: for every ordered pair (P_i, P_j) such that $\mathbf{M}_l[i], \mathbf{M}_l[j] \neq 0$, parties P_i and P_j invoke $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ on input seeds $\mathbf{u}_i$ and $\mathbf{v}_j$, respectively, and receive back α_i^j and β_j^i , respectively, such that $\alpha_i^j + \beta_j^i = \mathbf{u}_i * \mathbf{v}_j$.
4. Next, each party splits $\mathbf{u}_i, \mathbf{v}_i$ and each α_i^j, β_j^i into length- k vectors $\{\mathbf{u}_{i,w}\}_{w \in [\sigma/k]}, \{\mathbf{v}_{i,w}\}_{w \in [\sigma/k]}, \{\alpha_{i,w}^j\}_{w \in [\sigma/k]}, \{\beta_{i,w}^j\}_{w \in [\sigma/k]}$, respectively.
5. Then, for $w \in [\lceil \sigma/k \rceil]$:
 - (a) Each party P_i computes and distributes random packed Shamir sharings $[\mathbf{u}_{i,w}]_{n-k}$ and $[\mathbf{v}_{i,w}]_{n-k}$.
 - (b) Next, the parties locally compute $([\mathbf{a}_{1,w}]_{n-k}, \dots, [\mathbf{a}_{m,w}]_{n-k})^\top \leftarrow \mathbf{M} \cdot ([\mathbf{u}_{1,w}]_{n-k}, \dots, [\mathbf{u}_{m,w}]_{n-k})^\top$ and $([\mathbf{b}_{1,w}]_{n-k}, \dots, [\mathbf{b}_{m,w}]_{n-k})^\top \leftarrow \mathbf{M} \cdot ([\mathbf{v}_{1,w}]_{n-k}, \dots, [\mathbf{v}_{m,w}]_{n-k})^\top$.
 - (c) Then for each $l \in [m]$:
 - i. Each P_i such that $\mathbf{M}_l[i] \neq 0$ computes:

$$\mathbf{c}_{l,w}^i \leftarrow (\mathbf{M}_l[i])^2 \cdot \mathbf{u}_{i,w} * \mathbf{v}_{i,w} + \mathbf{M}_l[i] \cdot \sum_{j \in [n] \setminus \{i\} : \mathbf{M}_l[j] \neq 0} \mathbf{M}_l[j] \cdot (\alpha_{i,w}^j + \beta_{i,w}^j),$$

then computes and distributes degree- $(n-k)$ packed Shamir sharing $[\mathbf{c}_{l,w}^i]_{n-k}$.

ii. Then, all of the parties set $[c_{l,w}]_{n-k} \leftarrow \sum_{i: \mathcal{M}_l[i] \neq 0} [c_{l,w}^i]_{n-k}$.

Extraction Level 2: Let $N = (m-1)/2$. The parties do the following for each $w \in [\lceil \sigma/k \rceil]$:

1. For $l \in [N+1]$, $\tau \in [k]$, the parties implicitly view $a_{l,w}^\tau, b_{l,w}^\tau, c_{l,w}^\tau$ as the l -th evaluation points of polynomials $A_{\tau,w}(\cdot)$ of degree N , $B_{\tau,w}(\cdot)$ of degree N , $C_{\tau,w}(\cdot)$ of degree $m-1$ ($= 2N$), respectively; i.e., $A_{\tau,w}(l) = a_{l,w}^\tau$, $B_{\tau,w}(l) = b_{l,w}^\tau$, and $C_{\tau,w}(l) = c_{l,w}^\tau$.
Note that the $N+1$ values $a_{l,w}^\tau, b_{l,w}^\tau$ define degree N polynomials $A_{\tau,w}(\cdot)$, $B_{\tau,w}(\cdot)$, respectively.
2. Letting $\mathbf{A}_w(\cdot) \leftarrow (A_{1,w}(\cdot), \dots, A_{k,w}(\cdot))$, $\mathbf{B}_w(\cdot) \leftarrow (B_{1,w}(\cdot), \dots, B_{k,w}(\cdot))$, and $\mathbf{C}_w(\cdot) \leftarrow (C_{1,w}(\cdot), \dots, C_{k,w}(\cdot))$, the parties have

$$([\mathbf{A}_w(1)]_{n-k}, \dots, [\mathbf{A}_w(N+1)]_{n-k}) = ([a_{1,w}]_{n-k}, \dots, [a_{N+1,w}]_{n-k});$$

$$([\mathbf{B}_w(1)]_{n-k}, \dots, [\mathbf{B}_w(N+1)]_{n-k}) = ([b_{1,w}]_{n-k}, \dots, [b_{N+1,w}]_{n-k});$$

$$([\mathbf{C}_w(1)]_{n-k}, \dots, [\mathbf{C}_w(N+1)]_{n-k}) = ([c_{1,w}]_{n-k}, \dots, [c_{N+1,w}]_{n-k}).$$

They use the former two, respectively, to locally compute $[\mathbf{A}_w(l)]_{n-k}$ and $[\mathbf{B}_w(l)]_{n-k}$ for $l \in [N+2, m]$ (using Lagrange interpolation).

3. Then, for $l \in [N+2, m]$, the parties execute π_{beaver} on input $[\mathbf{A}_w(l)]_{n-k}$ and $[\mathbf{B}_w(l)]_{n-k}$, using packed beaver triples $[a_{l,w}]_{n-k}, [b_{l,w}]_{n-k}, [c_{l,w}]_{n-k}$, to receive $[\mathbf{C}_w(l)]_{n-1}$.
4. Note that the sharings $([\mathbf{C}_w(1)]_{n-1}, \dots, [\mathbf{C}_w(m)]_{n-1})$ implicitly define the polynomials $\mathbf{C}_w(\cdot) = \mathbf{A}_w(\cdot) * \mathbf{B}_w(\cdot)$. Thus, for $l \in [m+1, m+\mu]$, the parties locally compute and output $([a^{l,w}]_{n-k}, [b^{l,w}]_{n-k}, [c^{l,w}]_{n-1}) \leftarrow ([\mathbf{A}_w(m+l)]_{n-k}, [\mathbf{B}_w(m+l)]_{n-k}, [\mathbf{C}_w(m+l)]_{n-1})$ (using Lagrange interpolation).

Checking the degree of P_1 's sharings: The parties invoke $\mathcal{F}_{\text{verify-deg}}$ on all sharings sent by P_1 in π_{beaver} .

Note that for semi-honest security, we can omit the use of $\mathcal{F}_{\text{verify-deg}}$.

Errors. Note that there are two places where corrupted parties P_i could inject errors δ such that $\mathbf{c} = \mathbf{a} * \mathbf{b} + \delta$ for our output triples in this protocol:

- P_i could input to $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ different seeds than those corresponding to the $[u_{i,w}]_{n-k}, [v_{i,w}]_{n-k}$ that they share.
- A corrupt party P_1 could distribute incorrect sharings $[\bar{\mathbf{d}}]_{k-1}, [\bar{\mathbf{e}}]_{k-1}$ in π_{beaver} .

Complexity of Π_{triple} . In extraction level 1: Step 1 requires $O(n \cdot \sigma)$ total computation. For each $l \in [m]$, step 3 uses $\ell^2 \cdot \sigma$ correlations with all-but-negligible probability, since the L0 row norm of $\mathbf{M}$ is ℓ with all-but-negligible probability. For each $w \in [\lceil \sigma/k \rceil]$, Step 5a costs $2n(n-t)$ total communication and $\tilde{O}(n^2)$ computation. For each $w \in [\lceil \sigma/k \rceil]$, Step 5b costs $O(n^2 \zeta)$ total computation, across all $l \in [m]$. For each $w \in [\lceil \sigma/k \rceil]$, Step 5(c)i costs $n(n-t) \cdot \zeta$ total communication and $\tilde{O}(n \cdot \zeta \cdot (\ell + n))$ computation, across all $l \in [m]$. For each $w \in [\lceil \sigma/k \rceil]$, Step 5(c)ii costs $O(n \cdot m \cdot \ell)$ computation, across all $l \in [m]$.

In extraction level 2: For each $w \in [\lceil \sigma/k \rceil]$, Step 2 costs $\tilde{O}(n \cdot m)$ total computation. For each $w \in [\lceil \sigma/k \rceil]$, Step 3 runs $\pi_{\text{beaver}} \leq m/2$ times. Each run of π_{beaver} costs $2(n-1)$ total communication and computation for Step 2 as well as $2(n-1)$ total elements of communication and $\tilde{O}(n)$ total computation for Steps 3 and 4. Thus, in total, the runs of π_{beaver} cost $\leq m/2 \cdot 4(n-1)$ communication and $\tilde{O}(mn)$ computation. For each $w \in [\lceil \sigma/k \rceil]$, Step 4 costs $\tilde{O}(n \cdot m)$ total computation.

The above totals at most $m \cdot \ell^2 \cdot \sigma$ OLE correlations, $(\sigma/k) \cdot (2n(n-t) + n(n-t)\zeta + 2(n-1)m)$ communication and $\tilde{O}(m \cdot \ell^2 \cdot \sigma + (\sigma/k) \cdot (n^2 + n^2\zeta + nm\ell))$ computation. Using our weakly super-invertible matrix family $\mathcal{M}_{\zeta \text{col}}$ from Section 5.2, we have that $m = \varepsilon n$ and $\ell = O(\lambda)$. So the above is at most $\tilde{O}(n \cdot \sigma)$ correlations, $\sigma n(n-t)/k \cdot (4 + \zeta - 2/n)$ communication and $\tilde{O}(\sigma \cdot n)$ computation. This cost is for $\mu \cdot \lceil \sigma/k \rceil = \Omega(\sigma)$ packed beaver triples, where $\mu = (\gamma - 1/2)m = (\gamma - 1/2)\varepsilon n$. So, we have $\tilde{O}(n)$ correlations, $n/(\gamma - 1/2) \cdot (4 + \zeta - 2/n)$ communication and $\tilde{O}(n)$ computation per packed triple, or at most $\tilde{O}(1)$ correlations, $2/(\varepsilon(\gamma - 1/2)) \cdot (4 + \zeta - 2/n)$ communication and $\tilde{O}(1)$ computation per individual triple. With concrete constants $\zeta > 0, \gamma > 1/2$ from Section B.5, the communication is constant.

We prove the following Lemma in Appendix G.

Lemma 5. Π_{triple} UC-realizes $\mathcal{F}_{\text{triple}}$ in the $(\mathcal{F}_{\text{OLE}}^{\text{prog}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{verify-deg}})$ -hybrid model, for $2\varepsilon n > m = \Omega(n)$ and any $1/2 < \gamma < 1$.

7 Authenticated Packed Shamir Sharings

Now we present and instantiate the functionality and procedure that are used to generate authenticated packed Shamir sharings. We can simply plug in this procedure, along with Π_{triple} , to construct the rest of the SuperPack and Sharing Transformation preprocessing. We do so in Appendices D and F, respectively.

7.1 Random Authenticated Packed Shamir Sharings

First, we introduce $\mathcal{F}_{\text{auth-rand}}$ which is used to generate authenticated packed Shamir sharings of random values. Let $m \in \mathbb{N}, 1/2 < \gamma < 1, \mu \leftarrow \gamma \cdot m - (m+1)/2$, and $t \geq k$. $\mathcal{F}_{\text{auth-rand}}$ takes in the parties' shares of the MAC key $[\Delta \cdot \mathbf{1}]_{n-k}$, reconstructs $\Delta \cdot \mathbf{1}$, and finally samples $\mu \cdot \lfloor t/k \rfloor$ random authenticated packed sharings $\llbracket \mathbf{r}_{l,w} \rrbracket_{n-k} := ([\mathbf{r}_{l,w}]_{n-k}, [\Delta \cdot \mathbf{r}_{l,w}]_{n-k})$ consistent with shares input by the adversary.

Functionality 2: $\mathcal{F}_{\text{auth-rand}}$

Parameters: Let $m \in \mathbb{N}, 1/2 < \gamma < 1, \mu \leftarrow \gamma \cdot m - (m+1)/2$, and $t \geq k$.

1. $\mathcal{F}_{\text{auth-rand}}$ first receives from all parties their shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ and reconstructs $\Delta \cdot \mathbf{1}$.

2. $\mathcal{F}_{\text{auth-rand}}$ then receives from the adversary either **abort** or the corrupted parties' shares $(r_{l,w}^j, s_{l,w}^j)_{l \in [\mu], w \in [\lfloor t/k \rfloor], j \in \text{Corr}}$.
3. Finally, $\mathcal{F}_{\text{auth-rand}}$ samples random sharings $([r_{l,w}]_{n-k}, [\Delta \cdot r_{l,w}]_{n-k})_{l \in [\mu], w \in [\lfloor t/k \rfloor]}$ based on the corrupted parties' shares and distributes to the honest parties their shares.

We now present our protocol $\Pi_{\text{auth-rand}}$ which instantiates $\mathcal{F}_{\text{auth-rand}}$. $\Pi_{\text{auth-rand}}$ first invokes $\mathcal{F}_{\text{triple}}$ to get $\mu \cdot \lfloor t/k \rfloor$ random triples $([r]_{n-k}, [s]_{n-k}, [r * s + \delta]_{n-1})$. For each such triple, the parties first compute $[\Delta \cdot \mathbf{1} + s]_{n-k}$ and open it to P_1 . P_1 then reconstructs $\Delta \cdot \mathbf{1} + s_{l,w}$ and computes then distributes packed Shamir sharing $[\Delta \cdot \mathbf{1} + s_{l,w}]_{k-1}$. Finally, the parties compute $[\Delta \cdot r_{l,w} - \delta_{l,w}]_{n-1} \leftarrow [r_{l,w}]_{n-k} * [\Delta \cdot \mathbf{1} + s_{l,w}]_{k-1} - [r_{l,w} * s_{l,w} + \delta_{l,w}]_{n-1}$ and invoke $\mathcal{F}_{\text{deg-reduce}}$ on these obtained values to receive back $[\Delta \cdot r_{l,w} - \delta_{l,w}]_{n-k}$. The parties then output each such $[\mathbf{r}]_{n-k} \leftarrow ([r]_{n-k}, [\Delta \cdot \mathbf{r} - \delta]_{n-k})$.

Note that since $n - k > n - t$, the honest parties' shares do not define any given sharing, and thus the adversary can always locally change the corrupted parties' shares to change the underlying secret. We may equivalently think that the shares of corrupted parties are changed so that the secret of the MAC sharing is $[\Delta \cdot \mathbf{r}]_{n-k}$ (and they can then change their shares to any arbitrary values). Thus, the output authenticated sharings satisfy those demanded by the functionality.

Protocol 2: $\Pi_{\text{auth-rand}}$

1. The parties take as input $[\Delta \cdot \mathbf{1}]_{n-k}$.
2. The parties invoke $\mathcal{F}_{\text{triple}}$ and receive random sharings $(([r_{1,w}]_{n-k}, [s_{1,w}]_{n-k}, [r_{1,w} * s_{1,w} + \delta_{1,w}]_{n-1}), \dots, ([r_{\mu,w}]_{n-k}, [s_{\mu,w}]_{n-k}, [r_{\mu,w} * s_{\mu,w} + \delta_{\mu,w}]_{n-1}))_{w \in [\lfloor t/k \rfloor]}$.
3. For $w \in [\lfloor t/k \rfloor], l \in [\mu]$:
 - (a) The parties first compute $[\Delta \cdot \mathbf{1} + s_{l,w}]_{n-k}$ and open it to P_1 .
 - (b) P_1 next reconstructs $\Delta \cdot \mathbf{1} + s_{l,w}$ and computes then distributes packed Shamir sharing $[\Delta \cdot \mathbf{1} + s_{l,w}]_{k-1}$.
 - (c) Then, the parties compute $[\Delta \cdot r_{l,w} - \delta_{l,w}]_{n-1} \leftarrow [r_{l,w}]_{n-k} * [\Delta \cdot \mathbf{1} + s_{l,w}]_{k-1} - [r_{l,w} * s_{l,w} + \delta_{l,w}]_{n-1}$ and invoke $\mathcal{F}_{\text{deg-reduce}}$ on it and receive back $[\Delta \cdot r_{l,w} - \delta_{l,w}]_{n-k}$.^a
4. Next, the parties invoke $\mathcal{F}_{\text{verify-deg}}$ on the $[\Delta \cdot \mathbf{1} + s_{l,w}]_{k-1}$ sharings.
5. Finally, the parties output $[\mathbf{r}_{l,w}]_{n-k} \leftarrow ([r_{l,w}]_{n-k}, [\Delta \cdot r_{l,w} - \delta_{l,w}]_{n-k})$, for $l \in [\mu], w \in [\lfloor t/k \rfloor]$.

^a Really, the parties input $n - t$ sharings in parallel to $\mathcal{F}_{\text{deg-reduce}}$ at a time.

Complexity of $\Pi_{\text{auth-rand}}$. Step 2 costs $2/(\varepsilon(\gamma - 1/2)) \cdot (4 + \zeta - 2/n)$ communication and $\tilde{O}(1)$ computation and OLE correlations per triple, Step 3a costs $n - 1$ elements per sharing, Step 3b costs $n - 1$ elements and $\tilde{O}(n)$ computation per sharing, and Step 3c costs $3n - k$ elements and $\tilde{O}(n)$ computation per sharing. This is $2/(\varepsilon(\gamma - 1/2)) \cdot (4 + \zeta - 2/n) + 10/\varepsilon - 4/(\varepsilon n) - 1$ communication and $\tilde{O}(1)$ OLE correlations and computation per slot.

Lemma 6. $\Pi_{\text{auth-rand}}$ UC-realizes $\mathcal{F}_{\text{auth-rand}}$ in the $(\mathcal{F}_{\text{triple}}, \mathcal{F}_{\text{deg-reduce}})$ -hybrid model.

We provide the proof of this Lemma in Appendix G.

7.2 Authenticating Packed Shamir Sharings

Now we present procedure π_{auth} which authenticates input packed sharings. It does so by generating random authenticated packed sharings using $\mathcal{F}_{\text{auth-rand}}$, and then using the common mask-and-open technique to authenticate the input sharings using the authentication of those from $\mathcal{F}_{\text{auth-rand}}$.

Procedure 3: π_{auth}

1. The parties take as input $([u_1]_{n-k}, \dots, [u_{n-t}]_{n-k})$, and $[\Delta \cdot \mathbf{1}]_{n-k}$.
2. First, the parties invoke $\mathcal{F}_{\text{auth-rand}}$ on input $[\Delta \cdot \mathbf{1}]_{n-k}$ to prepare random authenticated sharings $([r_1]_{n-k}, \dots, [r_{n-t}]_{n-k})$.
3. Then for $l \in [n-t]$:
 - (a) They first compute $[u_l + r_l]_{n-k}$ and open it to P_1 .
 - (b) P_1 next reconstructs $u_l + r_l$ and computes then distributes packed Shamir sharing $[u_l + r_l]_{k-1}$.
 - (c) Then, the parties compute $[\Delta \cdot u_l]_{n-1} \leftarrow [\Delta \cdot \mathbf{1}]_{n-k} * [u_l + r_l]_{k-1} - [\Delta \cdot r_l]_{n-k}$.
4. Then they invoke $\mathcal{F}_{\text{deg-reduce}}$ on input $([\Delta \cdot u_1]_{n-1}, \dots, [\Delta \cdot u_{n-t}]_{n-1})$ and receive back $([\Delta \cdot u_1]_{n-k}, \dots, [\Delta \cdot u_{n-t}]_{n-k})$.
5. Next, they invoke $\mathcal{F}_{\text{verify-deg}}$ on all of the $[u_l + r_l]_{k-1}$ sharings.
6. Finally, the parties output $[u_l]_{n-k} \leftarrow ([u_l]_{n-k}, [\Delta \cdot u_l]_{n-k})$, for $l \in [n-t]$.

Complexity of π_{auth} .

1. Step 2 costs $2/(\varepsilon(\gamma - 1/2)) \cdot (4 + \zeta - 2/n) + 10/\varepsilon - 4/(\varepsilon n) - 1$ communication and $\tilde{O}(1)$ computation and OLE correlations per slot.
2. Step 3a costs $n - 1$ elements per sharing.
3. Step 3b costs $n - 1$ elements and $\tilde{O}(n)$ computation per sharing.
4. Step 4 costs $3n - k$ elements and $\tilde{O}(n)$ computation per sharing.

The above totals $2/(\varepsilon(\gamma - 1/2)) \cdot (4 + \zeta - 2/n) + 20/\varepsilon - 8/(\varepsilon n) - 2$ communication and $\tilde{O}(1)$ computation and OLE correlations per individual slot.

8 Concrete Communication Costs and Experiments

We have implemented our packed triple protocol, only ignoring the calls to $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ (as in the implementation of SuperPack). Here we discuss the concrete communication costs and runtimes of our protocols with specific instantiations of our weakly super-invertible matrices using different values of ζ and γ (see Section 5.3 and Appendix B.5). We compare to the communication and runtime of the public implementation of SuperPack's packed triple protocol.⁸

⁸ Available at <https://github.com/ckweng/SuperPack>.

	$\varepsilon = 1/2$		$\varepsilon = 1/4$		$\varepsilon = 1/8$	
	SuperPack	$\Pi_{\text{triple}} (\zeta = 3)$	SuperPack	$\Pi_{\text{triple}} (\zeta = 5)$	SuperPack	$\Pi_{\text{triple}} (\zeta = 7)$
$n = 256$	284.0	174.8 (0.660)	312.0	570.9 (0.626)	368	1832 (0.596)
$n = 512$	540.0	167.6 (0.667)	568.0	431.0 (0.667)	624	1222 (0.644)
$n = 1024$	1052	167.6 (0.667)	1080	431.0 (0.667)	1136	1054 (0.667)
$n = 2048$	2076	167.6 (0.667)	2104	431.1 (0.667)	2160	1054 (0.667)
$n = 4096$	4124	167.7 (0.667)	4152	431.1 (0.667)	4208	1054 (0.667)
$n = 8192$	8220	167.7 (0.667)	8248	431.1 (0.667)	8304	1054 (0.667)

Fig. 1. Number of communicated field elements per multiplication triple generated. We report for n starting at 256 and $\varepsilon \in \{1/2, 1/4, 1/8\}$. We specify the chosen ζ and γ in parentheses taken from Appendix B.5. **Bold** values indicate settings where our protocol has lower communication compared to Superpack’s protocol.

Our Implementation. We implement Π_{triple} by using the implementation of SuperPack [EGP⁺23] as a baseline. As with SuperPack, our code is written in C++ with no dependencies beyond the standard library and includes fully functional network code. However, we also deploy the protocol as multiple processes in a single machine, and emulate real network conditions using the package `netem`,⁹ which allows us to set bandwidth and latency constraints. For our protocols we use a 64-bit finite field $\mathbb{F}$ and weakly super-invertible matrix construction parameterized by values of ζ and γ provided in Appendix B.5.

Experimental Setup. We execute our experiments on an AWS c5.12xlarge instance with 48 vCPUs, 3.6 GHz Intel Xeon processors, and 96 GiB of memory. We explore the performance of our protocol and SuperPack [EGP⁺23] using network settings of 10 or 50 megabits per second (mbps). All reported results are the average of at least 5 trials with standard deviation less than 10% of the average.

8.1 Packed Beaver Triple Generation

Communication. In Figure 1, we report the concrete communication costs of our triple generation protocol Π_{triple} . We choose parameters ζ and γ for our weakly super-invertible matrix construction from Appendix B.5 that minimize communication. Additional communication costs for other choices of ζ and γ in Appendix E. We compare the concrete communication costs of Π_{triple} with the triple generation in SuperPack [EGP⁺23] that requires communicating $n + 6/\varepsilon - 2$ field elements per triple. For $\varepsilon = 1/2$, our Π_{triple} protocol with $\zeta = 3$ has at least 37% less communication than that of SuperPack. Furthermore, the improvements only grow more substantial for increasing values of n . The fact that our protocol has increasingly better communication for larger n makes sense as its communication is completely independent of n . In contrast, the communication of SuperPack is dependent on n . We see similar improvements for $\varepsilon = 1/4$ and $\varepsilon = 1/8$ but only starting at larger values of $n = 512$ and $n = 1024$ respectively. For smaller ε , we note that it makes sense that communication increases for both protocols as lower ε means that each packed sharing holds smaller batches.

⁹ <https://wiki.linuxfoundation.org/networking/netem>

Computation. In Figure 2, we report the computational costs of our triple generation protocol Π_{triple} for $n = 256$, again choosing parameters ζ and γ from Appendix B.5 to minimize running times. For circuit sizes $|C| = 100\text{K}$, our protocol Π_{triple} seems far superior to SuperPack in either network setting. In particular, SuperPack’s triple generation algorithm does not complete without crashing due to its large computational costs. In contrast, our Π_{triple} always successfully terminates. We take this as evidence that Π_{triple} is more efficient than SuperPack for larger circuits.

For smaller circuit sizes $|C| = 10\text{K}$, Π_{triple} is 8.6% faster than SuperPack in the 10mbps network setting when $\varepsilon = 1/2$. For faster network settings of 50mbps, SuperPack ends up having smaller runtimes. This makes sense as our Π_{triple} protocol involves two levels of extraction, which seems to result in larger computational costs than the packed triple generation protocol of SuperPack. In contrast, Π_{triple} has lower smaller communication. So, SuperPack being faster in certain higher throughput network settings is reasonable. For $\varepsilon = 1/4$, SuperPack is faster for both 10mbps and 50mbps since its communication is also lower in this setting.

8.2 Instantiating SuperPack’s Offline Phase

We also present comparisons for instantiating the offline phase of SuperPack using our protocol $\Pi_{\text{prep-sup-mal}}$. As a reminder, the whole preprocessing protocol for SuperPack has communication of $6n + 35/\varepsilon$ field elements per game. In Figure 3, we provide the concrete communication costs for certain choices of ζ and γ that minimize communication. In Appendix E, we provide communication costs for other settings of ζ and γ . For $\varepsilon = 1/2$, one can see $\Pi_{\text{prep-sup-mal}}$ with $\zeta = 3$ has at least 16% less communication than that of SuperPack starting at $n = 512$. Furthermore, with the reduction only gets better for larger n . For $n = 256$, SuperPack’s preprocessing protocol has lower communication. We see similar improvements for $\Pi_{\text{prep-sup-mal}}$ over SuperPack for $\varepsilon = 1/4$ and $\varepsilon = 1/8$, albeit starting at larger values of $n = 1024$ and $n = 2048$, respectively. For $\varepsilon = 1/4$, we see that $\zeta = 5$ in $\Pi_{\text{prep-sup-mal}}$ gives the best results and for $\varepsilon = 1/8$, we see that $\zeta = 6$ in $\Pi_{\text{prep-sup-mal}}$ gives the best results. The value of γ is typically 0.667 for all of these settings. That our $\Pi_{\text{prep-sup-mal}}$ has increasingly better communication for larger n makes sense, as its communication dependence on n is 3x lower than that of SuperPack. Also, as mentioned above, it makes sense that lower ε yields higher communication.

We were unable to reliably execute the implementation of SuperPack to measure communication costs as it crashed for relevant choices of n and $|C|$. In general, we believe that this is an artifact of the implementation as opposed to the protocols. We believe that the running times for Superpack’s offline phase instantiated with Π_{triple} should also lead to faster running times and an implementation that was built to scale to larger parties would showcase this fact. We leave such an implementation to future work.

	$\varepsilon = 1/2$		$\varepsilon = 1/4$	
	SuperPack	$\Pi_{\text{triple}} (\zeta = 3)$	SuperPack	$\Pi_{\text{triple}} (\zeta = 6)$
$ C =10K$				
10 mbps	56.89	52.02 (0.660)	73.71	152.8 (0.667)
50 mbps	23.35	36.01 (0.660)	22.72	61.32 (0.667)
$ C =100K$				
10 mbps	-	288.9 (0.660)	-	1524 (0.667)
50 mbps	-	233.0 (0.660)	-	544.9 (0.667)

Fig. 2. Running time in seconds of SuperPack’s and our packed beaver triple protocol. We report for $n = 256$ and $\varepsilon \in \{1/2, 1/4\}$. We specify the chosen ζ and γ in parentheses taken from Appendix B.5. We denote by ‘-’ the settings where experiments crashed due to large computational costs. **Bold** values indicate settings where our protocol has lower computation compared to SuperPack’s protocol.

	$\varepsilon = 1/2$		$\varepsilon = 1/4$		$\varepsilon = 1/8$	
	SuperPack	TwoLevel($\zeta = 3$)	SuperPack	TwoLevel($\zeta = 5$)	SuperPack	TwoLevel($\zeta = 6$)
$n = 256$	1606	2185 (0.660)	1676	5644 (0.626)	1826	28870 (0.547)
$n = 512$	3142	2639 (0.667)	3212	5036 (0.667)	3352	15360 (0.597)
$n = 1024$	6214	3664 (0.667)	6284	6061 (0.667)	6424	12890 (0.532)
$n = 2048$	12358	5712 (0.667)	12428	8110 (0.667)	12568	12300 (0.665)
$n = 4096$	24650	9808 (0.667)	24716	12210 (0.667)	24856	17001 (0.667)
$n = 8192$	49220	18000 (0.667)	49290	20400 (0.667)	49430	25190 (0.667)

Fig. 3. Number of communicated field elements per individual multiplication gate. We report for various different n starting at 256 and $\varepsilon \in \{1/2, 1/4, 1/8\}$. We specify the chosen ζ and γ in parentheses taken from Appendix B.5. **Bold** values indicate settings where our protocol has lower communication compared to SuperPack’s protocol.

Disclaimer

Disclaimer: This paper was prepared for informational purposes in part by the Artificial Intelligence Research group of JPMorgan Chase & Co. and its affiliates ("JP Morgan") and is not a product of the Research Department of JP Morgan. JP Morgan makes no representation and warranty whatsoever and disclaims all liability, for the completeness, accuracy or reliability of the information contained herein. This document is not intended as investment research or investment advice, or a recommendation, offer or solicitation for the purchase or sale of any security, financial instrument, financial product or service, or to be used in any way for evaluating the merits of participating in any transaction, and shall not constitute a solicitation under any jurisdiction or to any person, if such solicitation under such jurisdiction or to such person would be unlawful.

© 2025 JPMorgan Chase & Co. All rights reserved.

References

- ACLS18. Sebastian Angel, Hao Chen, Kim Laine, and Srinath T. V. Setty. PIR with compressed queries and amortized query processing. In *2018 IEEE Symposium on Security and Privacy*, pages 962–979. IEEE Computer Society Press, May 2018.
- AIK11. Benny Applebaum, Yuval Ishai, and Eyal Kushilevitz. How to garble arithmetic circuits. In Rafail Ostrovsky, editor, *52nd FOCS*, pages 120–129. IEEE Computer Society Press, October 2011.

- AJL⁺12. Gilad Asharov, Abhishek Jain, Adriana López-Alt, Eran Tromer, Vinod Vaikuntanathan, and Daniel Wichs. Multiparty computation with low communication, computation and interaction via threshold FHE. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 483–501. Springer, Berlin, Heidelberg, April 2012.
- BBC⁺19. Dan Boneh, Elette Boyle, Henry Corrigan-Gibbs, Niv Gilboa, and Yuval Ishai. Zero-knowledge proofs on secret-shared data via fully linear PCPs. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 67–97. Springer, Cham, August 2019.
- BCG⁺19a. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, Peter Rindal, and Peter Scholl. Efficient two-round ot extension and silent non-interactive secure computation. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS '19*, page 291–308, New York, NY, USA, 2019. Association for Computing Machinery.
- BCG⁺19b. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent OT extension and more. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 489–518. Springer, Cham, August 2019.
- BCG⁺20. Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators from ring-LPN. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 387–416. Springer, Cham, August 2020.
- BCGI18. Elette Boyle, Geoffroy Couteau, Niv Gilboa, and Yuval Ishai. Compressing vector OLE. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 896–912. ACM Press, October 2018.
- BCS19. Carsten Baum, Daniele Cozzo, and Nigel P. Smart. Using TopGear in overdrive: A more efficient ZKPoK for SPDZ. In Kenneth G. Paterson and Douglas Stebila, editors, *SAC 2019*, volume 11959 of *LNCS*, pages 274–302. Springer, Cham, August 2019.
- BDOZ11. Rikke Bendlin, Ivan Damgård, Claudio Orlandi, and Sarah Zakarias. Semi-homomorphic encryption and multiparty computation. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 169–188. Springer, Berlin, Heidelberg, May 2011.
- Bea92. Donald Beaver. Efficient multiparty protocols using circuit randomization. In Joan Feigenbaum, editor, *CRYPTO'91*, volume 576 of *LNCS*, pages 420–432. Springer, Berlin, Heidelberg, August 1992.
- BGJK21. Gabrielle Beck, Aarushi Goel, Abhishek Jain, and Gabriel Kaptchuk. Order-C secure multiparty computation for highly repetitive circuits. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 663–693. Springer, Cham, October 2021.
- BGW88. Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation (extended abstract). In *20th ACM STOC*, pages 1–10. ACM Press, May 1988.

- BPSY24. Alexander Bienstock, Sarvar Patel, Joon Young Seo, and Kevin Yeo. Batch PIR and labeled PSI with oblivious ciphertext compression. In Davide Balzarotti and Wenyuan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.
- Bra87. Gabriel Bracha. An $o(\log n)$ expected rounds randomized byzantine generals protocol. *J. ACM*, 34(4):910–920, October 1987.
- BTH08. Zuzana Beerliová-Trubíniová and Martin Hirt. Perfectly-secure MPC with linear communication complexity. In Ran Canetti, editor, *TCC 2008*, volume 4948 of *LNCS*, pages 213–230. Springer, Berlin, Heidelberg, March 2008.
- BY25. Alexander Bienstock and Kevin Yeo. Weakly super-invertible matrices and constant communication dishonest majority mpc. In *Eurocrypt 2025*. Springer, 2025.
- Can01. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.
- CCD88. David Chaum, Claude Crépeau, and Ivan Damgård. Multiparty unconditionally secure protocols (extended abstract). In *20th ACM STOC*, pages 11–19. ACM Press, May 1988.
- CHLR18. Hao Chen, Zhicong Huang, Kim Laine, and Peter Rindal. Labeled PSI from fully homomorphic encryption with malicious security. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 1223–1237. ACM Press, October 2018.
- CLR17. Hao Chen, Kim Laine, and Peter Rindal. Fast private set intersection from homomorphic encryption. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1243–1255. ACM Press, October / November 2017.
- CMdG⁺21. Kelong Cong, Radames Cruz Moreno, Mariana Botelho da Gama, Wei Dai, Iliia Iliashenko, Kim Laine, and Michael Rosenberg. Labeled psi from homomorphic encryption with reduced computation and communication. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 1135–1150, 2021.
- CP17. Ashish Choudhury and Arpita Patra. An efficient framework for unconditionally secure multiparty computation. *IEEE Transactions on Information Theory*, 63(1):428–468, 2017.
- CSW23. Ran Cohen, Abhi Shelat, and Daniel Wichs. Adaptively secure mpc with sublinear communication complexity. *J. Cryptol.*, 36(2), March 2023.
- DIK10. Ivan Damgård, Yuval Ishai, and Mikkel Krøigaard. Perfectly secure multiparty computation and the computational overhead of cryptography. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 445–465. Springer, Berlin, Heidelberg, May / June 2010.
- DKL⁺13. Ivan Damgård, Marcel Keller, Enrique Larraia, Valerio Pastro, Peter Scholl, and Nigel P. Smart. Practical covertly secure MPC for dishonest majority - or: Breaking the SPDZ limits. In Jason Crampton, Sushil Jajodia, and Keith Mayes, editors, *ESORICS 2013*, volume 8134 of *LNCS*, pages 1–18. Springer, Berlin, Heidelberg, September 2013.
- DM02. Olivier Dubois and Jacques Mandler. The 3-xorsat threshold. *Comptes Rendus Mathématique*, 335(11):963–966, 2002.
- DN07. Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In Alfred Menezes, editor, *CRYPTO 2007*,

- volume 4622 of *LNCS*, pages 572–590. Springer, Berlin, Heidelberg, August 2007.
- DPSZ12. Ivan Damgård, Valerio Pastro, Nigel P. Smart, and Sarah Zakarias. Multi-party computation from somewhat homomorphic encryption. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 643–662. Springer, Berlin, Heidelberg, August 2012.
- EGP⁺23. Daniel Escudero, Vipul Goyal, Antigoni Polychroniadou, Yifan Song, and Chenkai Weng. SuperPack: Dishonest majority MPC with constant online communication. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 220–250. Springer, Cham, April 2023.
- FY92. Matthew K. Franklin and Moti Yung. Communication complexity of secure computation (extended abstract). In *24th ACM STOC*, pages 699–710. ACM Press, May 1992.
- GHS12. Craig Gentry, Shai Halevi, and Nigel P. Smart. Fully homomorphic encryption with polylog overhead. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 465–482. Springer, Berlin, Heidelberg, April 2012.
- GIOZ17. Juan A. Garay, Yuval Ishai, Rafail Ostrovsky, and Vassilis Zikas. The price of low communication in secure multi-party computation. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part I*, volume 10401 of *LNCS*, pages 420–446. Springer, Cham, August 2017.
- GIP15. Daniel Genkin, Yuval Ishai, and Antigoni Polychroniadou. Efficient multi-party computation: From passive to active security via secure SIMD circuits. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *CRYPTO 2015, Part II*, volume 9216 of *LNCS*, pages 721–741. Springer, Berlin, Heidelberg, August 2015.
- GLO⁺21. Vipul Goyal, Hanjun Li, Rafail Ostrovsky, Antigoni Polychroniadou, and Yifan Song. ATLAS: Efficient and scalable MPC in the honest majority setting. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 244–274, Virtual Event, August 2021. Springer, Cham.
- GMW87. Oded Goldreich, Silvio Micali, and Avi Wigderson. How to play any mental game or A completeness theorem for protocols with honest majority. In Alfred Aho, editor, *19th ACM STOC*, pages 218–229. ACM Press, May 1987.
- Goe. Michel Goemans. Chernoff bounds, and some applications. <https://math.mit.edu/~goemans/18310S15/chernoff-notes.pdf>. Accessed: 2024-02-02.
- GPR⁺21. Gayathri Garimella, Benny Pinkas, Mike Rosulek, Ni Trieu, and Avishay Yanai. Oblivious key-value stores and amplification for private set intersection. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 395–425, Virtual Event, August 2021. Springer, Cham.
- GPS21. Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. Unconditional communication-efficient MPC via hall’s marriage theorem. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 275–304, Virtual Event, August 2021. Springer, Cham.
- GPS22. Vipul Goyal, Antigoni Polychroniadou, and Yifan Song. Sharing transformation and dishonest majority MPC with packed secret sharing. In

- Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 3–32. Springer, Cham, August 2022.
- GSY21. S. Dov Gordon, Daniel Starin, and Arkady Yerukhimovich. The more the merrier: Reducing the cost of large scale MPC. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 694–723. Springer, Cham, October 2021.
- HIK07. Danny Harnik, Yuval Ishai, and Eyal Kushilevitz. How many oblivious transfers are needed for secure multiparty computation? In Alfred Menezes, editor, *CRYPTO 2007*, volume 4622 of *LNCS*, pages 284–302. Springer, Berlin, Heidelberg, August 2007.
- HN06. Martin Hirt and Jesper Buus Nielsen. Robust multiparty computation with linear communication complexity. In Cynthia Dwork, editor, *CRYPTO 2006*, volume 4117 of *LNCS*, pages 463–482. Springer, Berlin, Heidelberg, August 2006.
- IKP⁺16. Yuval Ishai, Eyal Kushilevitz, Manoj Prabhakaran, Amit Sahai, and Ching-Hua Yu. Secure protocol transformations. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 430–458. Springer, Berlin, Heidelberg, August 2016.
- IPS09. Yuval Ishai, Manoj Prabhakaran, and Amit Sahai. Secure arithmetic computation with no honest majority. In Omer Reingold, editor, *TCC 2009*, volume 5444 of *LNCS*, pages 294–314. Springer, Berlin, Heidelberg, March 2009.
- KOS16. Marcel Keller, Emmanuela Orsini, and Peter Scholl. MASCOT: Faster malicious arithmetic secure computation with oblivious transfer. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 830–842. ACM Press, October 2016.
- KPR18. Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making SPDZ great again. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 158–189. Springer, Cham, April / May 2018.
- KTZ13. Jonathan Katz, Aishwarya Thiruvengadam, and Hong-Sheng Zhou. Feasibility and infeasibility of adaptively secure fully homomorphic encryption. In Kaoru Kurosawa and Goichiro Hanaoka, editors, *PKC 2013*, volume 7778 of *LNCS*, pages 14–31. Springer, Berlin, Heidelberg, February / March 2013.
- LN17. Yehuda Lindell and Ariel Nof. A framework for constructing fast MPC over arithmetic circuits with malicious adversaries and an honest-majority. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 259–276. ACM Press, October / November 2017.
- MP20. Brice Minaud and Charalampos Papamanthou. Note on generalized cuckoo hashing with a stash. *arXiv preprint arXiv:2010.01890*, 2020.
- MR23. Muhammad Haris Mughees and Ling Ren. Vectorized batch private information retrieval. In *2023 IEEE Symposium on Security and Privacy*, pages 437–452. IEEE Computer Society Press, May 2023.
- MSS11. Steven Myers, Mona Sergi, and Abhi Shelat. Threshold fully homomorphic encryption and secure computation. *IACR Cryptol. ePrint Arch.*, 2011:454, 2011.
- Nob21. Daniel Noble. An intimate analysis of cuckoo hashing with a stash. Cryptology ePrint Archive, Report 2021/447, 2021.

- NV18. Peter Sebastian Nordholt and Meilof Veeningen. Minimising communication in honest-majority MPC by batchwise multiplication verification. In Bart Preneel and Frederik Vercauteren, editors, *ACNS 18 International Conference on Applied Cryptography and Network Security*, volume 10892 of *LNCS*, pages 321–339. Springer, Cham, July 2018.
- PS16. Boris Pittel and Gregory B Sorkin. The satisfiability threshold for k-xorsat. *Combinatorics, Probability and Computing*, 25(2):236–268, 2016.
- PSZ18. Benny Pinkas, Thomas Schneider, and Michael Zohner. Scalable private set intersection based on ot extension. *ACM Transactions on Privacy and Security (TOPS)*, 21(2):1–35, 2018.
- RB89. Tal Rabin and Michael Ben-Or. Verifiable secret sharing and multiparty protocols with honest majority (extended abstract). In *21st ACM STOC*, pages 73–85. ACM Press, May 1989.
- RS22. Rahul Rachuri and Peter Scholl. Le mans: Dynamic and fluid MPC for dishonest majority. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part I*, volume 13507 of *LNCS*, pages 719–749. Springer, Cham, August 2022.
- Sha79. Adi Shamir. How to share a secret. *Commun. ACM*, 22(11):612–613, nov 1979.
- WYKW21. Chenkai Weng, Kang Yang, Jonathan Katz, and Xiao Wang. Wolverine: Fast, scalable, and communication-efficient zero-knowledge proofs for boolean and arithmetic circuits. In *2021 IEEE Symposium on Security and Privacy*, pages 1074–1091. IEEE Computer Society Press, May 2021.
- Yao82. Andrew C. Yao. Protocols for secure computations. In *Proceedings of the 23rd Annual Symposium on Foundations of Computer Science*, SFCS '82, page 160–164, USA, 1982. IEEE Computer Society.
- Yeo23a. Kevin Yeo. Cuckoo hashing in cryptography: Optimal parameters, robustness and applications. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 197–230. Springer, Cham, August 2023.
- Yeo23b. Kevin Yeo. Lower bounds for (batch) PIR with private preprocessing. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part I*, volume 14004 of *LNCS*, pages 518–550. Springer, Cham, April 2023.

Supplementary Material

A Additional Preliminaries

Universal Composability (UC) framework of [Can01]. In this framework, we have the following two worlds:

- **Real World Execution.** In the real world, the adversary $\mathcal{A}$ interacts with honest parties and can adaptively corrupt up to t of them. At the end of the protocol, the output of the real-world execution includes the inputs and outputs of honest parties and the view of the adversary.
- **Ideal World Execution.** In the ideal world, a simulator $\mathcal{S}$, simulates honest parties and interacts with the adversary $\mathcal{A}$. Furthermore, $\mathcal{S}$ has one-time access to $\mathcal{F}$, which includes providing inputs of corrupted parties to $\mathcal{F}$, receiving the outputs of corrupted parties, and sending instructions specified in $\mathcal{F}$ (e.g., asking $\mathcal{F}$ to abort). The output of the ideal world execution includes the inputs and outputs of honest parties and the view of the adversary.

In more detail, we say that a protocol Π UC-realizes $\mathcal{F}$ if there exists a simulator $\mathcal{S}$, such that for every adversary $\mathcal{A}$, the distribution of the output of the real world execution is (in our case, statistically) indistinguishable from the distribution of the output of the ideal world execution.

More on Packed Shamir Secret Sharings. In our work, we will use *packed Shamir secret sharing*, introduced by [FY92]. This is a generalization of standard Shamir secret sharing [Sha79]. With packed Shamir secret sharing, we will pack k secrets into one sharing. A *degree- d* ($d \geq k - 1$) packed Shamir sharing of $\mathbf{x} = (x_1, \dots, x_k)^\top \in \mathbb{F}^k$ is a vector $(w_1, \dots, w_n)$ for which there is some polynomial $f(x)$ of degree at most d such that $f(-i + 1) = x_i$ for all $i \in [k]$ and $f(i) = w_i$ for all $i \in [n]$. We denote such a sharing $[\mathbf{x}]_d$. The i -th share w_i is held by P_i . Reconstructing a degree- d packed Shamir sharing requires $d + 1$ shares and can be done using Lagrange interpolation. For a random degree- d packed Shamir sharing of $\mathbf{x}$, any $d - k + 1$ shares are independent of the secret $\mathbf{x}$. We therefore say that a degree- d packed Shamir sharing is *private* against an adversary that holds $d - k + 1$ shares. For some $C \subseteq [n]$, we say that some sharing $[\mathbf{x}]_d$ is *consistent* with set $\{v_i\}_{i \in C}$ if $w_i = v_i$ for all $i \in C$.

Packed Shamir sharings have the following properties, which follow directly from the computation of the underlying polynomials:

- **Linearity:** For any $n > d \geq k - 1$ and $\mathbf{x}, \mathbf{y} \in \mathbb{F}^k$, $[\mathbf{x} + \mathbf{y}]_d = [\mathbf{x}]_d + [\mathbf{y}]_d$.
- **Multiplicative:** For any $d_1, d_2 \geq k - 1$ such that $d_1 + d_2 < n$ and any $\mathbf{x}, \mathbf{y} \in \mathbb{F}^k$, $[\mathbf{x} * \mathbf{y}]_{d_1 + d_2} = [\mathbf{x}]_{d_1} * [\mathbf{y}]_{d_2}$, where $*$ is component-wise multiplication.

Note that the second property implies that for all $k - 1 \leq d \leq n - k$, a degree- d packed Shamir secret sharing is *multiplication-friendly*. What we mean by this is that for all $\mathbf{x}, \mathbf{c} \in \mathbb{F}^k$, all parties can locally compute $[\mathbf{c} * \mathbf{x}]_{d+k-1}$ from $[\mathbf{x}]_d$ and public vector $\mathbf{c}$. To do this, all parties just locally transform $\mathbf{c}$ to the (unique) degree- $(k - 1)$ packed Shamir sharing $[\mathbf{c}]_{k-1}$ and then use the $*$ operation.

When $k = 1$, a packed Shamir sharing is simply a Shamir sharing. Usually, Shamir sharing uses the default evaluation point 0 to store the secret. In our work, we also use different evaluation points in different Shamir secret sharings. Concretely, for all $i \in [k]$, we use $[x]_i$ to represent a degree- d Shamir sharing of x such that the secret is stored at the evaluation point $-i + 1$. If we use f to denote the degree- d polynomial corresponding to $[x]_i$, then $f(-i + 1) = x$.

Recall that t is the maximal number of corrupted parties and that a degree- d packed Shamir secret sharing is private against an adversary that holds $d - k + 1$ shares. To ensure that the Shamir secret sharing is both private and multiplication-friendly, we choose k such that $t \leq d - k + 1$ and $d \leq n - k$. When $d = n - k$ and $k = (n - t + 1)/2$, both requirements hold and k is maximal.

Additive Secret Sharings. An additive sharing of some value x is a vector $(w_1, \dots, w_n)$ such that $\sum_{i=1}^n w_i = x$. We denote such a sharing $\langle x \rangle$. The i -th share w_i is held by P_i . Reconstruction is done by summing the shares as above. For a random additive sharing $\langle x \rangle$, any $n - 1$ shares are independent of x . Additive sharings $\langle x_1 \rangle$ and $\langle x_2 \rangle$ can be added together to get $\langle x_1 + x_2 \rangle$ by having parties simply add their shares of the two sharings together.

Basic MPC Functionalities. For our protocol, we will use two basic functionalities that are common in the literature. One is $\mathcal{F}_{\text{commit}}$, which enables a party to commit to some values of their choice towards a set of receivers, without revealing the value. Later, the party can open their committed value with the guarantee that this value is exactly the same one that was committed to earlier. One way to instantiate this is to use a random oracle H (as in [DKL⁺13]): a party commits to m by sampling random $r \leftarrow_{\$} \{0, 1\}^\kappa$ and sending $c \leftarrow H(m, r) \in \{0, 1\}^\lambda$; then the party opens m by sending the pair (m, r) to the receivers, who check that $H(m, r) = c$. The communication complexity is λ bits for committing and $|m| + \lambda$ bits for opening, both per receiver. The second functionality is $\mathcal{F}_{\text{coin}}$, which provides parties with a uniformly random value $r \in \mathbb{F}$ when invoked. The standard way to instantiate this is by having the parties (i) compute some random sharing unknown to the adversary, $\langle r \rangle$, for example by independently sharing random values $\langle r_1 \rangle, \dots, \langle r_n \rangle$, then computing $\langle r \rangle \leftarrow \sum_{i=1}^n \langle r_i \rangle$; (ii) commit to their individual shares; and (iii) finally open $\langle r \rangle$. If more coins are needed at the same time, $\langle r \rangle$ can be expanded using a PRG.

Chernoff's Bound. Throughout our paper, we will use Chernoff's bound that provides tail bounds for the sum of multiple independent binary variables. We provide the statement of the theorem below (see [Goe] for example).

Theorem 7. *Let $X = X_1 + \dots + X_n$ where $X_i = 1$ with probability p and $X_i = 0$ with probability $1 - p$ and all X_i are independent. Let $\mu = \mathbb{E}[X] = pn$. Then,*

1. $\Pr[X > (1 + \delta)\mu] \leq e^{-\frac{\delta^2}{2+\delta} \cdot \mu}$ for any $\delta > 0$.
2. $\Pr[X < (1 - \delta)\mu] \leq e^{-\frac{\delta^2}{2} \cdot \mu}$ for any $0 < \delta < 1$.

In other words, $\Pr[|X - \mu| > \delta \cdot \mu] \leq e^{-\Theta(\mu)}$ for any constant $0 < \delta < 1$.

B Supplementary Material for Invertible Matrices

B.1 Missing Proofs for Section 5.2

Proof of Theorem 5. Each of the n column vectors has exactly ζ non-zero entries. So, the expected L_0 norm for any row is $\zeta \cdot (n/m) = O(\zeta)$ as $n = O(m)$.

For the worst case bound, fix any row $i \in [m]$. For any $j \in [n]$, let X_j be the random binary variable indicating whether the j -th column has a non-zero entry in the i -th row. Then, we see that $\Pr[X_j] = \zeta/m$. So, $\mu = \mathbb{E}[X_1 + \dots + X_n] = n\zeta/m = c\zeta$ for the chosen constant $c \geq 1$ that is the sum of n independent binary variables. By Chernoff's bound, we get that the probability that the i -th row vector has more than $\ell = O(\max(\zeta, \lambda + \log m))$ non-zero entries is at most $2^{-\lambda - \log m}$ for sufficiently large ℓ . Finally, we apply a Union bound over all m rows to obtain the probability upper bound of $2^{-\lambda}$. $\square$

Proof of Lemma 1. Note that the null space is a subspace of $\mathbb{F}^h$. Therefore, the dimension of the null space is exactly $\log_{|\mathbb{F}|}(Y)$. By the rank-nullity theorem, we know that rank of $\mathbf{A}$ and dimension of the null space of $\mathbf{A}$ must sum to the number of columns of $\mathbf{A}$. Therefore, the rank of $\mathbf{A}$ over $\mathbb{F}$ is exactly $r = h - \log_{|\mathbb{F}|}(Y)$. $\square$

Proof of Lemma 2. We first fix any $c_x \leq x \leq \beta \cdot m/\zeta$ number of columns and will show that the probability that any subset of x columns forms a null space vector is at most $2^{-\Omega(\zeta x)}$. We can conclude that for every number $\beta \cdot m/\zeta \geq x \geq c_x$ of rows, for sufficiently large $c_x = \Omega(\lambda + \log m)$ such that $2^{-\Omega(\zeta x)} \leq 2^{-\Omega(\zeta c_x)} \leq 2^{-2\lambda - \log m}$, the probability that there exists a null space vector of any size x is at most

$$\sum_{x=c_x}^{\beta \cdot m/\zeta} 2^{-\Omega(\zeta x)} \leq (\beta \cdot m/\zeta) \cdot 2^{-2\lambda - \log m} \leq 2^{-2\lambda}.$$

Note, we can choose such $c_x = \Omega(\lambda + \log m)$ since $m = \Omega(\lambda)$ is sufficiently large.

To show this bound on the above probability, fix any subset of x columns. Let t be the number of rows chosen by at least one of the x columns. For this subset of columns to be part of a null space vector, it must be that each row is chosen at least twice. If a row is chosen exactly once, then we can immediately see that it is impossible to obtain the zero vector as the product of two non-zero elements in $\mathbb{F}$ will always be non-zero since $\mathbb{F}$ is a field. Therefore, we know that $t \leq \zeta x/2$. Fix any subset of t rows chosen by the x columns and we will calculate an upper bound on the probability that these x columns form a null space vector over these t rows. This is upper bounded by the probability that each column only chooses from these t variables. Therefore, we get that this probability is at most $(t/m)^\zeta$ for each of the x columns independently and the probability is $(t/m)^{\zeta x}$ for all x columns. Next, we perform a Union bound over all possible choices of the x columns and the t rows to get

$$\binom{n}{x} \binom{m}{t} \left(\frac{t}{m}\right)^{\zeta x} \leq \left(\frac{en}{x}\right)^x \cdot \left(\frac{em}{t}\right)^t \cdot \left(\frac{t}{m}\right)^{\zeta x}$$

by applying the upper bound that $\binom{a}{b} \leq (ea/b)^b$. The above probability is strictly increasing in t and since $t \leq \zeta x/2$, the probability is at most

$$\left(\frac{en}{x}\right)^x \left(\frac{2em}{\zeta x}\right)^{\zeta x/2} \left(\frac{\zeta x}{2m}\right)^{\zeta x} \leq \left(\frac{e^{\zeta/2+1} n x^{\zeta/2-1} \zeta^{\zeta/2}}{2^{\zeta/2} m^{\zeta/2}}\right)^x = O\left(\frac{x \zeta^{\zeta/(\zeta-2)}}{m}\right)^{(\zeta/2-1) \cdot x}.$$

Note that we used the fact that $n \leq cm$ in the second equality. Also, in the last step, we use that $\zeta/(\zeta-2) \cdot (\zeta/2-1) = \zeta/(\zeta-2) \cdot (\zeta-2)/2 = \zeta/2$. Recall that our goal is to show that the above probability is bounded by $2^{-\Omega(\zeta x)}$. Therefore, it is sufficient to pick $x \leq m/(2 \cdot \zeta^{\zeta/(\zeta-2)}) = m/(2 \cdot \zeta^{1+O(1/\zeta)})$. Note that $\zeta^{1/\zeta} = 2^{\log \zeta / \zeta} = O(1)$. So, we can pick some constant $0 < \beta < 1$ so that $x \leq \beta \cdot m/\zeta$ such that the above probability is at most $2^{-\Omega(\zeta x)}$ as required. $\square$

Proof of Lemma 3. The probability is clearly zero for $y = 1$. Pick any $y \geq 2$ and consider the probability distribution of the sum of $y - 1$ random variables. We can see that

$$\begin{aligned} \Pr[X_1 + \dots + X_y = 0] &= \sum_{i \in \mathbb{F} \setminus \{0\}} \Pr[X_y = |\mathbb{F}| - i] \cdot \Pr[X_1 + \dots + X_{y-1} = i] \\ &= \frac{1}{|\mathbb{F}| - 1} \sum_{i \in \mathbb{F} \setminus \{0\}} \Pr[X_1 + \dots + X_{y-1} = i] \leq \frac{1}{|\mathbb{F}| - 1} \end{aligned}$$

completing the proof. $\square$

Lemma 7. *Pick any constant $0 < \eta < 1$. Suppose that $\mathbb{F}$ is a finite field. Let $\mathbf{M}$ be a $m \times n$ matrix generated by $\mathcal{M}_{\zeta \text{col}}$. Consider any subset of columns $C = \{i_1, \dots, i_x\} \subseteq [n]$ such that $\beta m/\zeta < x \leq \eta \cdot m$ and any set of non-zero coefficients $a_1, \dots, a_x \in \mathbb{F} \setminus \{0\}$. Then, the probability that C and $a_1, \dots, a_x$ is a null space vector is at most $(|\mathbb{F}| - 1)^{-\nu x} + 2^{-\Omega(\zeta x)}$ for some constant $\nu > 1$.*

Proof. First, pick any constant $\nu > 1$ satisfying $\nu x < m$. Note, this is possible since $x \leq \eta \cdot m$ and η is a constant strictly smaller than 1. Therefore, some choice of $\nu > 1$ always exists. We split the analysis into two parts. First, we will consider the probability conditioned on $\mathbf{M}^C$ containing $t \geq \nu x$ non-zero rows. Afterwards, we show that the probability there are less than νx non-zero rows is also small.

Suppose that $\mathbf{M}^C$ has $t \geq \nu x$ non-zero row vectors. Note, if any of the t non-zero row vectors contains exactly one non-zero entry, then it is impossible for C and $a_1, \dots, a_x$ to be a null space vector. Suppose that each of the t non-zero row vectors contains at least two non-zero entries. Suppose the j -th row of $\mathbf{M}^C$ is a non-zero row vector with $y \geq 2$ non-zero entries that are chosen uniformly at random from $\mathbb{F} \setminus \{0\}$. Consider the j -th entry of the linear combination: $a_1 \mathbf{M}^{i_1} + a_2 \mathbf{M}^{i_2} + \dots + a_x \mathbf{M}^{i_x}$. By Lemma 3, the probability that $y \geq 2$ uniformly random non-zero elements in $\mathbb{F}$ sum to zero is at most $1/(|\mathbb{F}| - 1)$. Note, the elements in each row are chosen independently at random. So, the probability that all $t \geq \nu x$ entries of the linear combination (corresponding to the t non-zero row vectors) will be zero is $(|\mathbb{F}| - 1)^{-t} \leq (|\mathbb{F}| - 1)^{-\nu x}$ as $t \geq \nu x$.

Next, we consider the probability that the conditioned event of at least νx non-zero row vectors is false. Fix any subset of $t < \nu x$ rows and we will calculate the probability that any subset of x columns will only pick one of the t rows in the fixed subset. A single column will pick these t rows with probability $(t/m)^\zeta$. So, the probability that all x columns will do this is $(t/m)^{\zeta x}$. Finally, we perform a Union bound over all $\binom{m}{t}$ possible subsets to get

$$\binom{m}{t} \left(\frac{t}{m}\right)^{\zeta x} \leq \left(\frac{em}{t}\right)^t \left(\frac{t}{m}\right)^{\zeta x} \leq \left(\frac{em}{\nu x}\right)^{\nu x} \left(\frac{\nu x}{m}\right)^{\zeta x}$$

where we applied the upper bound $(a/b)^b \leq \binom{a}{b}$ and the fact that the probability is maximized by setting $t = \nu x$. By our choice of ν , we know that $\nu x/m < 1$. Therefore, we know that $(\nu x/m)^{\zeta x} = 2^{-\Omega(\zeta x)}$. On the other hand, we know that $x = \Omega(m/\zeta)$, so we know that $\left(\frac{em}{\nu x}\right)^{\nu x} = O(\zeta)^{\nu x} = 2^{O(x \log \zeta)}$. Therefore, we see that this probability is at most $2^{O(x \log \zeta)} \cdot 2^{-\Omega(\zeta x)} = 2^{-\Omega(x(\zeta - \log \zeta))} = 2^{-\Omega(\zeta x)}$. Finally, the probability that C and $a_1, \dots, a_x$ is a null space vector is at most $(|\mathbb{F}| - 1)^{-\nu x} + 2^{-\Omega(\zeta x)}$ for some constant $\nu > 1$. $\square$

Proof of Lemma 4. Consider any fixed size $\beta \cdot m/\zeta \leq x \leq \eta \cdot m$ for null space vectors. Let Y^x be the random variable denoting the number of null space vectors of size x in $\mathbf{M}$. Now, fix any subset of x columns denoted by $C = \{i_1, \dots, i_x\} \subseteq [n]$ as well as x non-zero coefficients $a_1, \dots, a_x$. By Lemma 7, we know that C and $a_1, \dots, a_x$ is a null space vector with probability at most $(|\mathbb{F}| - 1)^{-\nu x} + 2^{-\Omega(\zeta x)}$ for some constant $\nu > 1$. Next, we can apply a Union bound over all choices of columns and non-zero coefficients to obtain a bound on the number of null space vectors of size x in $\mathbf{M}$ denoted by Y^x . The number of choices of columns is $\binom{n}{x} = \binom{cm}{x}$ and the number of choices of non-zero coefficients is $(|\mathbb{F}| - 1)^x$. So, we get

$$\begin{aligned} \Pr[Y^x \geq 1] &\leq \binom{cm}{x} \cdot (|\mathbb{F}| - 1)^x \cdot ((|\mathbb{F}| - 1)^{-\nu x} + 2^{-\Omega(\zeta x)}) \\ &\leq 2^{O(x \cdot \log \zeta)} \cdot (|\mathbb{F}| - 1)^{-(\nu-1)x} + 2^{-\Omega(x(\zeta - \log |\mathbb{F}|))} \\ &\leq 2^{O(x \cdot \log \zeta)} \cdot (2^{-\Omega(\log(|\mathbb{F}|-1)x)} + 2^{-\Omega(\zeta x)}) \\ &\leq 2^{O(x \cdot \log \zeta)} \cdot 2^{-\Omega(\zeta x)} \\ &= 2^{-\Omega(\zeta x)}. \end{aligned}$$

In the second inequality, we used that $x = \Omega(m/\zeta)$ to see that $\binom{cm}{x} = 2^{O(x \log \zeta)}$. In the third inequality, we use the fact that ν is a constant such that $\nu > 1$ and assume that $\zeta = \Theta(\log |\mathbb{F}|)$. In the fourth inequality, we use that $\zeta = \Theta(\log |\mathbb{F}|)$. We get that the above probability is at most $\Pr[Y^x \geq 1] \leq 2^{-\Omega(\zeta x)} \leq 2^{-2\lambda - \log m}$ as $x \geq \beta \cdot m/\zeta$ and we assume $m = \Omega(\lambda)$. Finally, we apply a Union bound over all $O(m)$ choices of null space vector sizes x to get that

$$\sum_{x=\beta \cdot m/\zeta}^{\alpha \cdot m} \Pr[Y^x \geq 1] \leq m \cdot 2^{-2\lambda - \log m} = 2^{-2\lambda}$$

to complete the proof. $\square$

Proof of Theorem 4. Let $\mathbf{M}$ be a $m \times n$ matrix generated randomly by $\mathcal{M}_{\zeta \text{col}}$. Now, consider any subset of $\eta \cdot m$ columns of $\mathbf{M}$ denoted by $C \subseteq [n]$. Recall that our goal is to show that the corresponding sub-matrix $\mathbf{M}^C$ has rank at least $\gamma \cdot m$ for some chosen constant $\gamma < 1$. We show that the sub-matrix $\mathbf{M}^C$ has rank at least $|C| - O(\lambda \log m + \log^2 m) = \eta \cdot m - O(\lambda \log m + \log^2 m) \geq \gamma \cdot m$ that follows as $\eta > \gamma$ and the fact that $m = \Omega(\lambda \log m)$ is sufficiently large.

It remains to show that $\mathbf{M}^C$ has large rank. First, by combining Lemma 2 and Lemma 4, we know that $\mathbf{M}$ has no null space vectors of size x where $c_x \leq x \leq \eta \cdot m$ for any constant $0 < \eta < 1$ and some $c_x = \Omega(\lambda + \log m)$ except with probability $2 \cdot 2^{-2\lambda} \leq 2^{-\lambda}$. Thus, $\mathbf{M}^C$ has no null space vectors of size more than c_x as $\mathbf{M}^C$ is a sub-matrix of $\mathbf{M}$ with $\eta \cdot m$ columns. Therefore, the total number of null space vectors in $\mathbf{M}^C$ is at most $\sum_{x=0}^{c_x-1} Y^x \leq \sum_{x=0}^{c_x-1} \binom{n}{x} \cdot (|\mathbb{F}| - 1)^x \leq c_x \cdot |\mathbb{F}|^{c_x} \cdot \binom{n}{c_x} \leq c_x \cdot (n \cdot |\mathbb{F}|)^{c_x}$ as we know that there are no null space vectors of size c_x or larger. Next, we apply Lemma 1 to see that the rank of the sub-matrix $\mathbf{M}^C$ is at least $\eta \cdot m - \log_{|\mathbb{F}|}(c_x \cdot (n \cdot |\mathbb{F}|)^{c_x}) \geq \eta \cdot m - c_x(\log_{|\mathbb{F}|} n + 1) - \log_{|\mathbb{F}|} c_x$ as the number of columns was $|C| = \eta \cdot m$. Suppose m is chosen to satisfy $m \geq (c_x(\log_{|\mathbb{F}|} n + 1) + \log_{|\mathbb{F}|} c_x)/(\eta - \gamma) = \Omega((\lambda + \log m) \log m)$. Note, it suffices to choose $m = \Omega(\lambda \log m)$ as $m = \Omega(\log^2 m)$ is already true for sufficiently large constant m . Then, we can see that the rank of the sub-matrix $\mathbf{M}^C$ is at least $\eta \cdot m - c_x(\log_{|\mathbb{F}|} n + 1) - \log_{|\mathbb{F}|} c_x \geq \gamma \cdot m$, except with probability $2^{-\lambda}$ for any subset of $\eta \cdot m$ columns $C \subseteq [n]$, completing the proof. $\square$

B.2 Extending $\mathcal{M}_{\zeta \text{col}}$ to Fields of Size Two

So far, we have shown that $\mathcal{M}_{\zeta \text{col}}$ emits a weakly super-invertible matrix over finite fields satisfying $|\mathbb{F}| \geq 3$. Next, we show that $\mathcal{M}_{\zeta \text{col}}$ is also a weakly super-invertible matrix in the binary field. Although, we note that we require larger choice of $\zeta = \Omega(\lambda + \log m)$. When $|\mathbb{F}| = O(1)$ and $|\mathbb{F}| \geq 3$, we only require that $\zeta = \Theta(\log |\mathbb{F}|) = \Theta(1)$ for our construction $\mathcal{M}_{\zeta \text{col}}$. We leave it as an open problem to obtain the same overhead for $|\mathbb{F}| = 2$. For convenience, we will use $\mathbb{F}_2$ for the binary field going forward.

Theorem 8. *Pick any choice of constants $c \geq 1$, $0 < \gamma < 1$ and $\gamma < \eta \leq c$. Let the number of columns be $n = c \cdot m$ and the number of non-zero entries in each column be $\zeta = \Theta(\lambda + \log m)$. The $m \times n$ matrix family $\mathcal{M}_{\zeta \text{col}}$ is $(2^{-\lambda}, \gamma, \eta, O(\lambda + \log m))$ -weakly super-invertible over $\mathbb{F}_2$ for sufficiently large $m = \Omega(\lambda \log m)$.*

As a reminder, $\mathcal{M}_{\zeta \text{col}}$ places uniformly random non-zero entries into ζ random locations per column. For the binary field $\mathbb{F}_2$, this means that $\mathcal{M}_{\zeta \text{col}}$ will place 1 entries into ζ random locations per column.

To do this, we see that the first two steps of the proof may still be used identically without modification. In particular, the relation between null space vectors and rank remains true for every field. Furthermore, the argument in the second step simply calculates the probability that each row is chosen at least

twice. This remains a requirement for every field. The main challenge occurs in the third step of the proof. Our proofs of Lemma 7 and Lemma 4 utilize the fact that the underlying field is sufficiently large. For the binary field $\mathbb{F}_2$, we cannot make this assumption anymore.

Instead, we will relate the setting of $\mathbb{F}_2$ to the random XORSAT problem. The XORSAT problem consists of m variables and h equations over the binary field $\mathbb{F}_2$. Each of the h equations are generated by picking ζ different variables at random and requiring that the sum of the ζ variables is equal to a uniformly random $b \in \{0, 1\}$. We say a specific XORSAT instance is satisfiable if there exists some assignment to the m variables such that all h equations are satisfied.

Next, we present a connection between $\mathcal{M}_{\zeta\text{col}}$ and the XORSAT problem. Recall that $\mathcal{M}_{\zeta\text{col}}$ generates $m \times n$ matrices $\mathbf{M}$ where each column consists of ζ non-zero entries (in $\mathbb{F}_2$, these will have values 1 and the remaining entries are 0). Consider any sub-matrix $\mathbf{M}^C$ for any subset of h columns $C \subseteq [n]$. We can interpret each of the m rows as variables and each of the $h = \eta \cdot m$ columns as equations. Finally, we will pick a uniformly random bit vector $\mathbf{b} \in \{0, 1\}^h$. Then, we can see that $\mathbf{M}^C$ and $\mathbf{b}$ is an instance of the XORSAT problem by trying to find whether a solution $\mathbf{x}$ exists such that $(\mathbf{M}^C)^\top \mathbf{x} = \mathbf{b}$ where $(\mathbf{M}^C)^\top$ is the transpose of $\mathbf{M}^C$.

We will extend our notion of null space vectors to the XORSAT problem. Consider an instance of the XORSAT problem with m variables and h equations denoted by the $h \times m$ matrix $\mathbf{A} \in \{0, 1\}^{h \times m}$ and the solution vector $\mathbf{b} \in \{0, 1\}^h$. A null space vector is a subset of rows of $\mathbf{A}$ whose component-wise sum is even. In the field $\mathbb{F}_2$, this means that the component-wise sum will be an entirely zero vector. Finally, we note that we can simply ignore the non-zero coefficients for each null space vector. In the binary field $\mathbb{F}_2$, the only non-zero entry is 1 meaning that all the coefficients must be 1.

Going back to the proof, we will use this connection to re-do step 3 of the proof where we will consider the case of large null space vectors of size $x > \beta \cdot m / \zeta$. We rely on known results in a variant of the XORSAT problem known as the constrained model introduced in [DM02] where each variable must appear in at least two equations. The unconstrained model of XORSAT omits this restriction. The unconstrained model can be reduced to the constrained model by simply removing variables that appear in at most one equation, which is the typical approach for analyzing the unconstrained XORSAT problem (see [PS16] as an example).

For our choice of $\zeta = O(\lambda + \log m)$, we show that the probability that there even exists a single variable that appears in at most one equation is exponentially small in λ . In other words, $\mathcal{M}_{\zeta\text{col}}$ will produce an instance of XORSAT in the constrained model except with negligible probability.

Lemma 8. *Let $\mathbf{M}$ be randomly generated by $\mathcal{M}_{\zeta\text{col}}$ with $\zeta = \Theta(\lambda + \log m)$ over the binary field $\mathbb{F}_2$. Every row of $\mathbf{M}$ has at least two non-zero entries except with probability $2^{-4\lambda}$.*

Proof. Consider any of the m rows and we will apply Chernoff's bound. Each of the n columns will choose to place a 1-entry in the fixed row with independent

probability ζ/m . So, the expected number of 1-entries in any row is $n \cdot \zeta/m = \Theta(\zeta)$ that is sum of n independent binary variables. By Chernoff's bounds, we know that the probability that any row has at most a single 1-entry is at most $2^{-2\lambda - \log m}$ for sufficiently large $\zeta = O(\lambda + \log m)$. Applying a Union bound over the m rows obtains the desired probability of $m \cdot 2^{-4\lambda - \log m} \leq 2^{-4\lambda}$. $\square$

As a remark, the reason for the larger choice of ζ is the ability to guarantee that the matrix corresponds to a XORSAT instance in the constrained model. Next, we are ready to apply known results about XORSAT from prior works. In particular, we will use a result of Pittel and Sorkin [PS16] that proves an upper bound on the expected value of the number of null space vectors of size $x = \Omega(m/\zeta)$:

Lemma 9 (Lemma 9, [PS16]). *Suppose $\zeta \geq 3$ and pick any $2/\zeta < \alpha < 1$ chosen independently of m such that $h = \alpha \cdot m$. Let $\mathbf{A}$ be an $h \times m$ matrix denoting a random XORSAT instance in the constrained model where each variable appears in at least two equations. For any $x = \Omega(m/\zeta)$, let Y^x denote the number of null space vectors in $\mathbf{A}$ of size x . Then, $\mathbb{E}[Y^x] \leq 2^{-\Omega(\zeta \cdot x)}$.*

The above lemma considers a single random XORSAT instance $\mathbf{A}$ of $h = \alpha \cdot m$ equations and m variables such that the number of equations is a constant fraction smaller than the number of variables. Then, the expected number of null space vectors of size x in $\mathbf{A}$ is exponentially small in $\zeta \cdot x$. We show that this is sufficient to show that any matrix generated by our matrix family $\mathcal{M}_{\zeta \text{col}}$ will also not contain null space vectors of size x for any $x = \Omega(m/\zeta)$.

Lemma 10. *Pick any constant $c \geq 1$. Let $n = c \cdot m$ and $\zeta = \Theta(\lambda + \log m)$. Let $\mathbf{M}$ be a $m \times n$ matrix generated by $\mathcal{M}_{\zeta \text{col}}$ in the binary field $\mathbb{F}_2$. Then, for every constant $0 < \eta < 1$ and $0 < \beta < 1$, the probability there exists at least one null space vector of any size $\beta \cdot m/\zeta \leq x \leq \eta \cdot m$ in $\mathbf{M}$ is at most $2^{-2\lambda}$.*

Proof. Consider any fixed size $\beta \cdot m/\zeta \leq x \leq \eta \cdot m$ for null space vectors. Let E be the event that the randomly generated matrix $\mathbf{M}^\top$ satisfies the constrained XORSAT model meaning that each column of $\mathbf{M}^\top$ has at least two non-zero entries. For now, we will condition on the event E holding. Now, fix any subset of $h = \max\{x, 2m/\zeta\}$ rows denoted by $C \subseteq [n]$ and consider the sub-matrix $\mathbf{A} = (\mathbf{M}^C)^\top$. Denote Y_A^x as the number of null space vectors of size x in this sub-matrix $\mathbf{A}$. By Lemma 9, we know that $\mathbb{E}[Y_A^x | E] \leq 2^{-\Omega(\zeta \cdot x)}$. By Markov's inequality, we get that $\Pr[Y_A^x \geq 1 | E] \leq 2^{-\Omega(\zeta \cdot x)}$ since Y_A^x is always non-negative. Next, we can apply a Union bound over all $\binom{n}{h} = \binom{cm}{h}$ choices of rows to obtain a bound on the number of null space vectors in $\mathbf{M}$ denoted by Y^x :

$$\Pr[Y^x \geq 1 | E] \leq \binom{cm}{h} \cdot 2^{-\Omega(\zeta \cdot x)} \leq \binom{cm}{O(x)} \cdot 2^{-\Omega(\zeta \cdot x)} \leq 2^{O(x \cdot \log \zeta)} \cdot 2^{-\Omega(\zeta \cdot x)}$$

where we note that $h = O(x \cdot \zeta^{1/\zeta}) = O(x \cdot 2^{\log \zeta / \zeta}) = O(x)$ and use that $x \geq \beta \cdot m/\zeta = \Omega(m/\zeta)$. We also use Stirling's approximation of binomial coefficients such that $\binom{a}{b} \leq (ea/b)^b$. For sufficiently large $\zeta = O(\lambda + \log m)$, we

see that $\Pr[Y^x \geq 1 \mid E] \leq 2^{-4\lambda - \log m}$. Finally, we apply a Union bound over all possible choices of x and remove the conditioning on the event E to get that

$$\sum_{x=\beta \cdot m/\zeta}^{\eta \cdot m} \Pr[Y^x \geq 1 \mid E] + \Pr[\neg E] \leq m \cdot 2^{-4\lambda - \log m} + 2^{-4\lambda} = 2^{-2\lambda}$$

where we used Lemma 8 to get that $\Pr[\neg E] \leq 2^{-4\lambda}$. $\square$

The above (combined with the second step of the proof) shows that, even in the binary field $\mathbb{F}_2$, a random matrix generated by $\mathcal{M}_{\zeta \text{col}}$ will not contain null space vectors larger than some constant c_x . With this fact, the final step of the proof proceeds identically.

Proof of Theorem 8. Follows identically to the proof of Theorem 4 where we replace the usage of Lemma 4 with the above Lemma 10 to rule out the existence of large null space vectors. $\square$

B.3 Hyper-Invertible Matrices

We present the definition of hyper-invertible matrices here that strengthens super-invertible matrices.

Definition 5 (Hyper-Invertible Matrices [BTH08]). *Let $\mathbf{M}$ be a matrix of dimension $m \times n$ with $n \geq m$. $\mathbf{M}$ is a hyper-invertible matrix if, for every choice of $1 \leq z \leq m$ and for any two subsets $R = \{r_1, \dots, r_z\} \subseteq [m]$ and $C = \{c_1, \dots, c_z\} \subseteq [n]$ both of size exactly z , the $z \times z$ sub-matrix defined as*

$$\mathbf{M}_R^C = \begin{bmatrix} \mathbf{M}_{r_1}[c_1] & \mathbf{M}_{r_1}[c_2] & \dots & \mathbf{M}_{r_1}[c_z] \\ \mathbf{M}_{r_2}[c_1] & \mathbf{M}_{r_2}[c_2] & \dots & \mathbf{M}_{r_2}[c_z] \\ \dots & \dots & \dots & \dots \\ \mathbf{M}_{r_z}[c_1] & \mathbf{M}_{r_z}[c_2] & \dots & \mathbf{M}_{r_z}[c_z] \end{bmatrix}$$

consisting of all entries in $\mathbf{M}$ that appears in a row specified by R and a column defined by the subset C must be invertible.

In our definition, we choose to only focus on the setting with at least as many columns as rows, $n \geq m$. One could also consider a less restrictive definition of hyper-invertible matrices where there are no requirements on the dimensions of the matrix. In that case, one could define hyper-invertibility as any $m \times n$ matrix $\mathbf{M}$ such that any $z \times z$ sub-matrix is invertible where $1 \leq z \leq \min(m, n)$. We do not consider such matrices as the level of generality is unnecessary for MPC.

B.4 Impossibility for Weakly Hyper-Invertible Matrices

Previously, we defined the notion of weakly super-invertible matrices that extended the notion of super-invertibility. A natural question may be to also consider the same weakening for hyper-invertible matrices. We will show that there is

no advantage for considering this weakening by proving a lower bound on the L_0 norm required for weakly hyper-invertible matrices that are also required by hyper-invertible matrices.

We can consider a matrix family $\mathcal{M}$ that generates a random $m \times n$ matrix $\mathbf{M}$. Then, for every sufficiently large $z \geq 1/\gamma$, every $z \times z$ sub-matrix of $\mathbf{M}$ should have rank $\gamma \cdot z$ for some constant $\gamma > 0$ except with negligible probability over the random choice of $\mathbf{M}$. This is same generalization for weakening hyper-invertible matrices.

Definition 6 (Weakly Hyper-Invertible Matrix Family). Fix constants $0 < \delta, \gamma < 1$. Let $\mathcal{M} = (\mathcal{S}, \text{Gen})$ be a matrix family over $m \times n$ matrices where $n \geq m$. $\mathcal{M}$ is (δ, γ) -weakly hyper-invertible if, for every $1/\gamma \leq z \leq m$ and every choice of z rows $R \subseteq [m]$ and z columns $C \subseteq [n]$, the following holds:

$$\Pr_{\mathbf{R}}[\mathbf{M}_R^C \text{ has rank at least } \gamma \cdot z \mid \mathbf{M} \leftarrow \text{Gen}(\mathbf{R})] \geq 1 - \delta$$

where $\mathbf{R}$ is a uniformly random bit string.

Note that we consider $z \geq 1/\gamma$ to ensure that the requirement forces the $z \times z$ sub-matrix to have rank at least $\gamma \cdot z \geq 1$. One could also consider more stringent requirements where $z \times z$ sub-matrices for $z < 1/\gamma$ still require rank at least one. As we are proving lower bounds, weaker restrictions imply stronger results.

We present the following lower bound on the number of $\Omega(n)$ non-zero entries per row required in a (δ, γ) -weakly hyper-invertible matrix. In particular, we prove that, if δ and γ are constants, then the expected L_0 norm of any row in a weakly hyper-invertible matrix family must be $\Omega(n)$.

Theorem 9. Suppose that matrix family $\mathcal{M} = (\mathcal{S}, \text{Gen})$ is (δ, γ) -weakly hyper-invertible for $\delta \leq 2/3$ and $\gamma = \Theta(1)$. Then, the expected L_0 norm for every row $i \in [m]$ satisfies $\mathbb{E}_{\mathbf{R}}[L_0(\mathbf{M}_i) \mid \mathbf{M} \leftarrow \text{Gen}(\mathbf{R})] = \Omega(n)$.

Proof. Consider any matrix $\mathbf{M} \in \mathcal{S}$ that satisfies the requirements for weak hyper-invertibility. That is, every $z \times z$ sub-matrix of $\mathbf{M}$ has rank at least $\gamma \cdot z$. Pick the smallest integer $t \geq 1/\gamma$. We can partition any $m \times n$ matrix into disjoint $t \times t$ matrices. Thus, we can obtain $\lfloor m/t \rfloor \cdot \lfloor n/t \rfloor = \Omega(\gamma^2 \cdot m \cdot n) = \Omega(m \cdot n)$ disjoint $t \times t$ sub-matrices using $\gamma = \Theta(1)$. By the weakly hyper-invertibility requirement, each $t \times t$ matrix must have rank at least $\gamma \cdot t \geq 1$. Therefore, each disjoint $t \times t$ sub-matrix must contain at least one non-zero entry. Altogether, $\mathbf{M}$ must have L_0 norm at least $\Omega(m \cdot n)$. Finally, consider the family $\mathcal{M}$ that must produce matrices satisfying the hyper-invertibility requirement with probability at least $1 - \delta$. Therefore, the expected L_0 norm of randomly generated matrices by $\mathcal{M}$ must be at least $\Omega((1 - \delta) \cdot m \cdot n) = \Omega(m \cdot n)$ since $\delta \leq 2/3$. Note each of the m rows can have at most n non-zero entries. Therefore, each row must have expectation of at least $\Omega(n)$ non-zero entries. $\square$

From the above, we can immediately observe that Vandermonde matrices are optimal weakly hyper-invertible matrices with respect to L_0 norm. In other words, there is no real advantage for considering weaker notions of hyper-invertibility.

Our above lower bound immediately implies the same norm lower bound for hyper-invertible matrices. In other words, every hyper-invertible matrix must have $\Omega(m \cdot n)$ non-zero entries and the Vandermonde matrix is an optimal hyper-invertible matrix with respect to the number of non-zero entries.

B.5 Supplementary Material for Practical Instantiations

We start by proving Theorem 6 that relates robust cuckoo hashing to weakly super-invertible guarantees.

Proof of Theorem 6. We note there is a bijection between each ζ -way robust cuckoo hashing instance and each instance of $\mathcal{M}_{\zeta\text{col}}$ when looking at the locations of the non-zero entries and ignoring the contents of non-zero entries. Take any ζ -way cuckoo hashing instance that successfully allocates every subset of ηn items into the m entries. The allocation of each subset of ηn items directly translates this into ηn pivots for Gaussian elimination in the corresponding $m \times (\eta n)$ sub-matrix of $\mathcal{M}_{\zeta\text{col}}$ where the non-zero entries correspond to the cuckoo hashing instance. In particular, suppose the j -th item is allocated to the i -th row in the cuckoo hashing instance. Then, we will use the j -th column as the pivot for the i -th row when executing Gaussian elimination.

Consider executing Gaussian elimination in any arbitrary order using the ηn pivots associated with ηn successfully allocated items. We note that one of the ηn pivots may not be used if prior row eliminations cause the corresponding entry to become 0. This probability is at most $1/(|\mathbb{F}| - 1)$ as the entry in the pivot location is a uniformly random non-zero entry in $\mathbb{F}$. Therefore, the probability that any of the ηn pivots is unusable is at most $\eta n/(|\mathbb{F}| - 1)$ by a union bound. Finally, we apply a union bound over all $\binom{n}{\eta n}$ subsets as well as the probability p that the cuckoo hashing scheme fails to be robust to obtain the final failure probability of $p + (\eta n \binom{n}{\eta n})/(|\mathbb{F}| - 1) = p + 2^{\Omega(\eta n - \log |\mathbb{F}|)}$. $\square$

Next, we present empirical evaluation of our constructions $\mathcal{M}_{\zeta\text{col}}$ for various $\zeta \in \{3, 4, 5\}$ to derive concrete guarantees of weak super-invertibility. In particular, we consider square sub-matrices of the form $m \times m$ and their rank when each column has ζ non-zero elements from $\mathbb{F}$ placed into ζ random rows. For each data point, we executed at least 2^{25} experiments consisting of sampling a random matrix and computing its rank. We consider different choices of $n \in \{40, 60, 80, 100\}$ and field sizes of $|\mathbb{F}| = 101$ and $|\mathbb{F}| \approx 2^{20}$ where we pick the smallest 20-bit prime as the field size, 1048583.

Frequency of Ranks. We first plot the frequency of different ranks for $m \times m$ matrices with $\zeta \in \{3, 4, 5\}$ where $m \in \{40, 60, 80, 100\}$ with field sizes of $|\mathbb{F}| = 101$ and $|\mathbb{F}| \approx 2^{20}$ in Figure 4. In general, we note that the graphs are very skewed towards high rank. In particular, it is much more likely to get a full rank matrix than one with the minimum observed rank. As one picks larger ζ , we see that the ranks tend to become larger as expected. A similar phenomenon can be observed for the larger field choice, but the effect is much smaller compared to picking larger ζ . In general, we see that the rank is typically way larger than $2m/3$ as

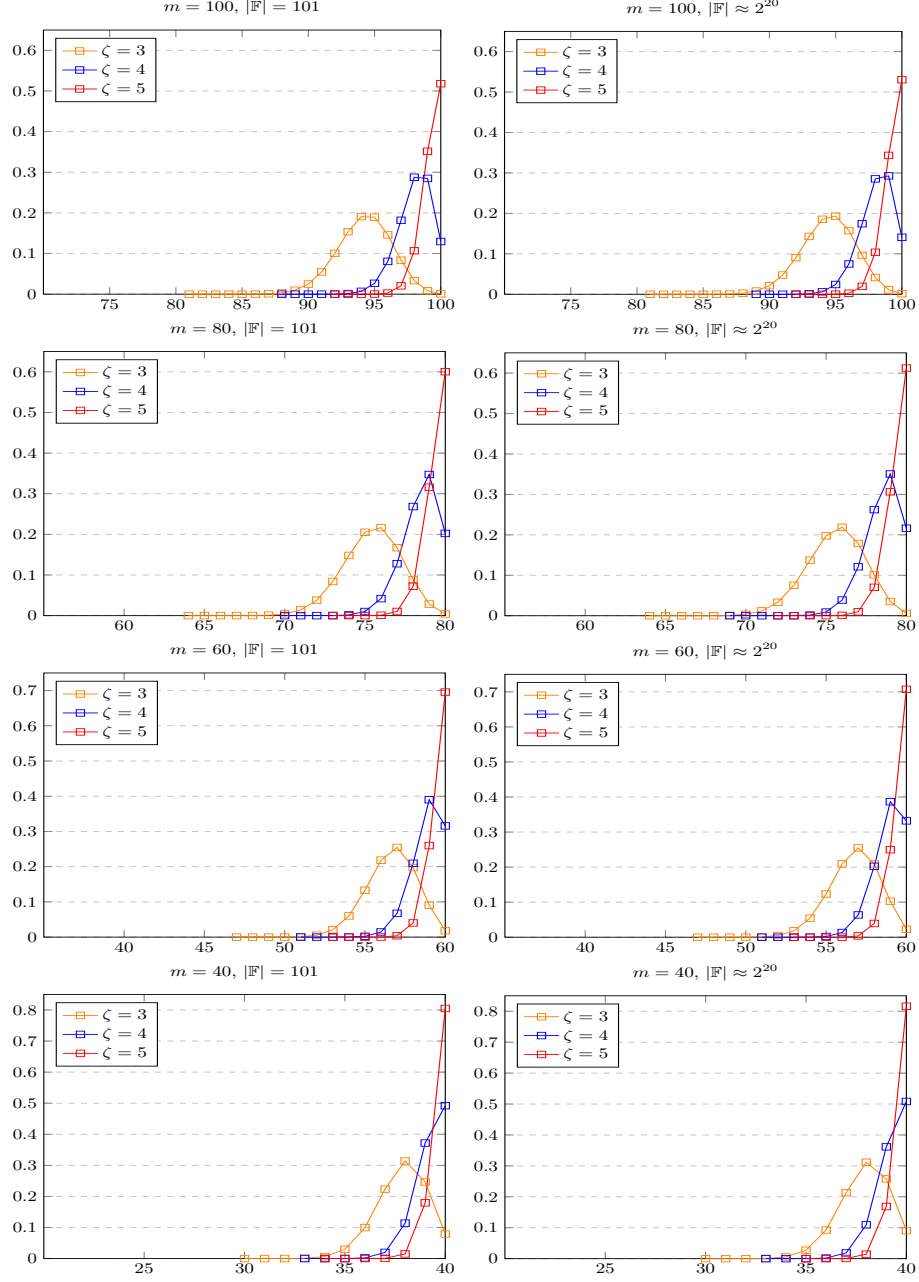


Fig. 4. Plots of the frequency of observed ranks for different choices of $m \in \{40, 60, 80, 100\}$, $\zeta \in \{3, 4, 5\}$ as well as $|\mathbb{F}| = 101$ and $|\mathbb{F}| \approx 2^{20}$. The x-axis is the rank and y-axis is the percentage of times that rank was observed.

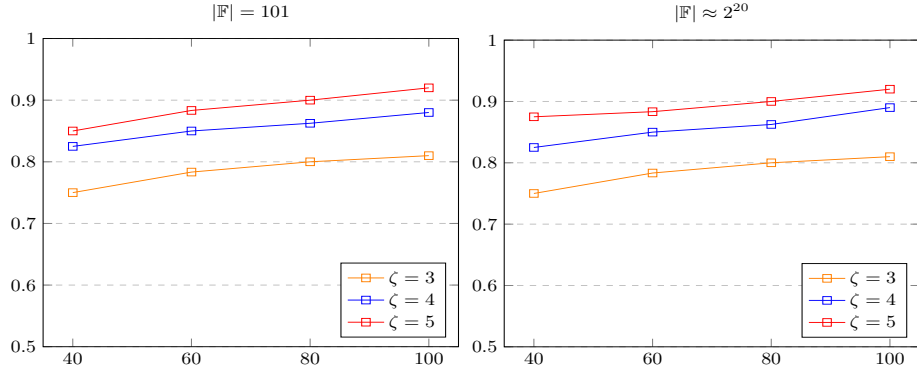


Fig. 5. Plots of the minimum observed rank as a ratio over the total number of rows across all experiments for different choices of $m \in \{40, 60, 80, 100\}$, $\zeta \in \{3, 4, 5\}$ as well as $|\mathbb{F}| = 101$ and $|\mathbb{F}| \approx 2^{20}$. The x-axis is different choices of m and the y-axis is the minimum observed rank divided by the total number of rows m .

required when choosing $\gamma = 2/3$. This experimentally supports our relations with robust cuckoo hashing. Furthermore, one could even conjecture that it may be safe to choose even larger $\gamma \geq 2/3$. In fact, we barely observe any instances where the rank was approaching even $3/4$ -fraction of the total number of rows.

Smallest Observed Rank. Next, we examine the smallest observed rank over the at least 2^{25} experiments ran for each of the parameter settings considered. In Figure 5, we plot the minimum rank divided by the total number of rows m across different values of m . This helps determine the choice of γ for $\mathcal{M}_{\zeta \text{col}}$. We see that the smallest rank observed for the smallest $m = 40$ and $\zeta = 3$ is 30 that is $3/4$ -fraction of all m rows. As m grows but ζ stays the same, we also observe the smallest observed rank increases as a fraction of the number of total rows. For example, when $m = 100$, we see that the minimum observed rank is 80 that is $4/5$ -fraction of all m rows. We conjecture that as we consider larger m , the weakly super-invertible properties of $\mathcal{M}_{\zeta \text{col}}$ get stronger. That is, one could consider larger choices of γ for $\mathcal{M}_{\zeta \text{col}}$ when m is larger.

Similarly, as ζ increases, we also observe that the smallest rank observed (and the value of γ) also increase. Fixing the setting of $m = 100$, we observe that the minimum rank observed for $\zeta = 3$ is 80. For $\zeta = 4$, this increases to 89. When considering $\zeta = 5$, this becomes 92. Therefore, considering larger ζ seems to also increase the weakly super-invertible properties.

We note that the field size does not seem to have any substantial impact on γ and the minimum observed rank is essentially the same (or differing by just one) for either field size choice of $|\mathbb{F}| = 101$ and $|\mathbb{F}| \approx 2^{20}$ across the different choices of m and ζ .

Finally, as a remark, we note that our experiments showed that the smallest observed rank was only $3/4$ -fraction of the total number of rows. In our constructions, we will choose $1/2 < \gamma \leq 2/3$ that is much smaller and, thus, an attempt at a safe choice to ensure weak super-invertibility of $\mathcal{M}_{\zeta \text{col}}$.

Robust Cuckoo Hashing Attacks [Yeo23a]. We also analyze the effectiveness of the attacks presented in [Yeo23a] for robust cuckoo hashing applied to our practical instantiations of $\mathcal{M}_{\zeta\text{col}}$. Translating the attack, the goal would be to pick some strictly less than $\gamma \cdot m$ rows and try to find a subset of m columns (amongst the n total columns) whose non-zero entries appear only in the chosen $\gamma \cdot m$ rows.

We can analyze the attack’s success probability in the following way. For any column, the probability that it satisfies the requirement is $((\gamma m)/m)^\zeta = \gamma^\zeta$. In our MPC applications, we choose m to be the set of honest parties, $\epsilon \cdot n$. From a theoretical perspective, we only require that $\gamma^\zeta < \epsilon$ to ensure that the probability that the attack success probability is negligible in n using Chernoff’s bound as the expected number of columns satisfying the constraint is $\gamma^\zeta \cdot n$ that is smaller than $\epsilon \cdot n$. We note that there are ways that one may attempt to expand the attacks in [Yeo23a] such as repeating the attack on random choices of $\gamma \cdot m$ rows or attempting to change rows from a prior attempt. To our knowledge, we are unaware of modifications that increase success probability substantially. We conjecture that, as long as γ and ζ are chosen such that γ^ζ is sufficiently smaller than ϵ for large enough n , then our practical instantiations are weakly super-invertible against efficient adversaries. We calculate the exact probability that the attack in [Yeo23a] succeeds using the Bernoulli distribution in Figure 6.

Summary and Different Instantiations. Using the experimental analysis, we note there are several potential constructions. First, we should pick $\zeta \geq 3$ and $\gamma \leq 2/3$ using the guidance from robust cuckoo hashing that is supported by our experimental analysis of random matrices and their ranks. Next, we should pick γ and ζ such that $\gamma^\zeta < \epsilon$ to avoid robust cuckoo hashing attacks from [Yeo23a] where we rely on our analysis in Figure 6. If one wishes to be more risk-tolerant, one could also pick larger ζ and/or smaller $1/2 < \gamma < 2/3$ to be safer. For our MPC constructions, we will consider larger fields $|\mathbb{F}| \approx 2^{60}$ and $|\mathbb{F}| \approx 2^{128}$ that are inherited from prior constructions. Again, this also provides more safety as larger fields have shown to provide better weakly super-invertible guarantees compared to smaller fields in our experiments.

B.6 Weakly Super-Invertible Matrices from Cuckoo Hashing

As shown in Section 5.3, we may obtain concrete instantiations of weakly super-invertible matrices with $\mathcal{M}_{\zeta\text{col}}$ with $\zeta = 3$ using experimental analysis of three-way cuckoo hashing. We note one could also attempt to obtain asymptotic constructions from provable guarantees of cuckoo hashing. Here, we explain why this approach does not enable constructions of weakly super-invertible matrices with $\gamma > 1/2$ that are necessary for our MPC applications.

We note that all current cuckoo hashing analysis [MP20, Nob21, Yeo23a] that obtain provable negligible failure probabilities require at least twice as many entries as allocated items. This is even true for those that consider non-robust schemes and/or obtain non-negligible failure probabilities. That is, at least $m \geq 2n$ entries are required for allocating n items. In fact, only the work

	$\zeta = 3$	$\zeta = 4$	$\zeta = 5$	$\zeta = 6$	$\zeta = 7$	$\zeta = 8$
$\epsilon = 1/2$						
$n = 256$	0.660	0.667	0.667	0.667	0.667	0.667
$n = 512$	0.667	0.667	0.667	0.667	0.667	0.667
$n = 1024$	0.667	0.667	0.667	0.667	0.667	0.667
$n = 2048$	0.667	0.667	0.667	0.667	0.667	0.667
$n = 4096$	0.667	0.667	0.667	0.667	0.667	0.667
$n = 8192$	0.667	0.667	0.667	0.667	0.667	0.667
$n = 16384$	0.667	0.667	0.667	0.667	0.667	0.667
$\epsilon = 1/4$						
$n = 256$	-	0.557	0.626	0.667	0.667	0.667
$n = 512$	0.510	0.603	0.667	0.667	0.667	0.667
$n = 1024$	0.546	0.635	0.667	0.667	0.667	0.667
$n = 2048$	0.571	0.657	0.667	0.667	0.667	0.667
$n = 4096$	0.589	0.667	0.667	0.667	0.667	0.667
$n = 8192$	0.601	0.667	0.667	0.667	0.667	0.667
$n = 16384$	0.609	0.667	0.667	0.667	0.667	0.667
$\epsilon = 1/8$						
$n = 256$	-	-	-	0.547	0.596	0.636
$n = 512$	-	-	0.540	0.597	0.644	0.667
$n = 1024$	-	-	0.577	0.632	0.667	0.667
$n = 2048$	-	0.530	0.602	0.655	0.667	0.667
$n = 4096$	-	0.550	0.619	0.667	0.667	0.667
$n = 8192$	-	0.563	0.631	0.667	0.667	0.667
$n = 16384$	-	0.572	0.640	0.667	0.667	0.667

Fig. 6. Table providing largest choice of $1/2 < \gamma \leq 2/3$ (rounded to three decimal places) such that the success probability of the robust cuckoo hashing attack in [Yeo23a] is smaller than 2^{-40} for various choices of ζ as well as number of parties n and honest fraction of parties ϵ . We denote no such choice of γ existing by ‘-’.

of Noble [Nob21] can obtain $m = 2n$ whereas the other works [MP20, Yeo23a] require $m > 2n$. Applying Theorem 6 with these cuckoo hashing analysis will always result in a weakly super-invertible matrix where $\gamma \leq n/m \leq 1/2$ that is insufficient for our MPC applications.

Furthermore, even if future work obtains cuckoo hashing analysis where $m < 2n$, we note the result will be weaker than our analysis in Section 5.2. In particular, Theorem 6 only results in reasonable failure probabilities for exponentially large fields. In contrast, our analysis enables constructing weakly super-invertible matrices for all finite field sizes $|\mathbb{F}| \geq 2$.

C Using Pseudorandom Generators to Reduce Communication of Secret Sharing

In this section, we adapt the common approach of using pseudorandom generators (PRGs) to reduce the communication complexity when preparing secret shar-

ings [LN17, NV18, BBC⁺19, GLO⁺21, GSY21, RS22, EGP⁺23]. Concretely, each pair of parties agree to a PRG seed at the beginning of the protocol. Whenever a party needs to send a random element to another party, the latter invokes the PRG on their shared seed to obtain this value. In the following, we discuss how to prepare certain sharings that we need in our construction and analyze the communication complexity.

Preparing Random Degree- $(n - k)$ Packed Shamir Sharings $[\mathbf{r}]_{n-k}$. We use such sharings in many places in our construction, including Π_{triple} . We generate such sharings by the following:

1. The dealer sets the shares of $[\mathbf{r}]_{n-k}$ of the first $n - k + 1$ parties to be random values.
2. For the underlying polynomial of $[\mathbf{r}]_{n-k}$, $n - k + 1$ evaluation points are fixed. The dealer reconstructs the whole sharing by using the shares of the first $n - k + 1$ parties.
3. Finally, the dealer distributes the shares to all other parties.

Observe that all shares except those of the last $k - 1$ parties are uniform elements. Thus, when using a PRG, the dealer only needs to send $k - 1 \leq (n - t)/2$ elements to distribute this sharing.

Preparing Degree- $(n - k)$ Packed Shamir Sharing $[\mathbf{x}]_{n-k}$, for Fixed $\mathbf{x}$. We generate such sharings by the following:

1. The dealer sets the secret to be $\mathbf{x}$.
2. The dealer sets the shares of the first t parties to be random values.
3. For the underlying polynomial of $[\mathbf{x}]_{n-k}$, $k + t = n - k + 1$ evaluation points are fixed. The dealer reconstructs the whole sharing by using the shares of the first t parties and the secrets $\mathbf{x}$.
4. Finally, the dealer distributes the shares to all other parties.

Observe that all shares except those of the last $n - t$ parties are uniform elements. Thus, when using a PRG, the dealer only needs to send $n - t$ elements to distribute this sharing.

Preparing Random Degree- $(n - 1)$ Packed Shamir Sharings of $\mathbf{0}$. We generate such sharings by the following:

1. The dealer sets the secret to be $\mathbf{0}$.
2. The dealer sets the shares of the first $n - k$ parties to be random values.
3. For the underlying polynomial of $[\mathbf{0}]_{n-1}$, $n - k + k = n$ evaluation points are fixed. The dealer reconstructs the whole sharing by using the shares of the first $n - k$ parties and the secrets $\mathbf{0}$.
4. Finally, the dealer distributes the shares to all other parties.

Observe that all shares except those of the last k parties are uniform elements. Thus, when using a PRG, the dealer only needs to send k elements to distribute this sharing.

Preparing Random Additive Sharings of 0. In our construction, we will need to refresh additive sharings by adding a random additive sharing of 0. We generate such sharings by the following:

1. The dealer first samples random values as the shares of $\langle 0 \rangle$ of the rest $n - 1$ parties.
2. The dealer locally computes his own share such that all shares together form an additive sharing of 0.
3. Finally, the dealer distributes the shares to all other parties.

Observe that all shares except of the dealer are random. Thus, when using a PRG, the dealer does not need to send any elements to distribute this sharing.

Preparing Random Degree- $(n - 1)$ Packed Shamir Sharings $[\mathbf{r}]_{n-1}$. We generate such sharings by the following:

1. The dealer sets the shares of $[\mathbf{r}]_{n-1}$ of all parties to be random values.
2. For the underlying polynomial of $[\mathbf{r}]_{n-1}$, all evaluation points are fixed. The dealer reconstructs the whole sharing by using the shares of the parties.
3. Finally, the dealer distributes the shares to all other parties.

Observe that all shares are uniform elements. Thus, when using a PRG, the dealer does not need to send any elements to distribute this sharing.

D Instantiating SuperPack's $\Pi_{\text{prep-sup-mal}}$

Here we present SuperPack's preprocessing functionality $\mathcal{F}_{\text{prep-sup-mal}}$ and our instantiation of it, $\Pi_{\text{prep-sup-mal}}$.

D.1 The Preprocessing Functionality

Below, we present the SuperPack preprocessing Functionality $\mathcal{F}_{\text{prep-sup-mal}}$ verbatim from [EGP⁺23]. The functionality begins by sampling a random $\Delta \leftarrow_{\S} \mathbb{F}$. This Δ serves as the MAC key in the online phase, and ensures that shared values are opened correctly. Indeed, in the online phase, all circuit values will be shared using degree- $(n-k)$ *authenticated* packed Shamir sharings $\llbracket \mathbf{v} \rrbracket_{n-k} := ([\mathbf{v}]_{n-k}, [\Delta \cdot \mathbf{v}]_{n-k})$, where the latter sharing is referred to as the ‘MAC’ on the former sharing. Intuitively, if $\mathbf{v}$ is opened then it will be checked by opening $\Delta \cdot \mathbf{v}$ —since Δ is random, with all-but-negligible probability, the adversary will be unable to force an opening to some $\mathbf{v}' \neq \mathbf{v}$.¹⁰ $\mathcal{F}_{\text{prep-sup-mal}}$ then samples k random degree- t Shamir sharings $([\Delta|_1]_t, \dots, [\Delta|_k]_t)$, one for each of the k slots in the secret space of packed sharings. Note that for these sharings (and all that follow), the adversary is allowed to choose shares for the corrupted parties; $\mathcal{F}_{\text{prep-sup-mal}}$ samples the rest of the sharing randomly based on the corrupted parties shares and the underlying

¹⁰ This is done once for many such openings at a time using random linear combinations.

secrets. Next, $\mathcal{F}_{\text{prep-sup-mal}}$ computes random degree- $(n-k)$ packed Shamir sharings $[\lambda_\alpha \cdot \mathbf{1}]_{n-k}$, where $\mathbf{1} = (1, \dots, 1) \in \mathbb{F}^k$, along with random additive sharings $\langle \Delta \cdot \lambda_\alpha \rangle$, for each output wire α of an input gate or a multiplication gate in the circuit C . While these sharings are expensive to generate, they are needed by SuperPack to obtain concrete efficiency in the online phase (related to permuting size- k groups of wire values, which is hard without such sharings).

For each group of k multiplication gates, $\mathcal{F}_{\text{prep-sup-mal}}$ prepares packed beaver triples as follows. It samples two random $\mathbf{a}, \mathbf{b} \in \mathbb{F}^k$, computes $\mathbf{c} \leftarrow \mathbf{a} * \mathbf{b}$, and then additionally prepares authenticated $([\mathbf{a}]_{n-k}, [\mathbf{b}]_{n-k}, ([\mathbf{c}]_{n-1}, \{\langle \Delta \cdot c_i \rangle\}_{i=1}^k))$. $\mathcal{F}_{\text{prep-sup-mal}}$ also prepares such random packed beaver triples for each group of k input gates or output gates, which are used to securely share and reconstruct the inputs and outputs, respectively, in the presence of a malicious adversary.

Finally, $\mathcal{F}_{\text{prep-sup-mal}}$ also prepares several random packed sharings of the form $[\mathbf{o}]_{n-1}$, where $\mathbf{o} = \mathbf{0}$, for each group of k multiplication gates, input gates, and output gates, which will be needed for securely reconstructing certain values in the online phase.

Functionality 3: $\mathcal{F}_{\text{prep-sup-mal}}$

1. **Setting Authentication Keys:** $\mathcal{F}_{\text{prep-sup-mal}}$ samples a random value Δ . Then $\mathcal{F}_{\text{prep-sup-mal}}$ samples k random degree- t Shamir sharings $([\Delta]_1, \dots, [\Delta]_k)$ and distributes the shares to all parties.
2. **Preparing Random Packed Sharings:** For each output wire α of an input gate or a multiplication gate in the circuit C , $\mathcal{F}_{\text{prep-sup-mal}}$ samples a random value λ_α and computes $\lambda_\alpha \cdot \mathbf{1}$, where $\mathbf{1} = (1, \dots, 1) \in \mathbb{F}^k$. Then $\mathcal{F}_{\text{prep-sup-mal}}$ samples
 - a random degree- $(n-k)$ packed Shamir sharing $[\lambda_\alpha \cdot \mathbf{1}]_{n-k}$,
 - and a random additive sharing $\langle \Delta \cdot \lambda_\alpha \rangle$,
 and distributes the shares to all parties.
3. **Preparing Packed Beaver Triples with Authentications:** For each group of k multiplication gates, $\mathcal{F}_{\text{prep-sup-mal}}$ samples a random packed Beaver triple with authentications as follows:
 - (a) $\mathcal{F}_{\text{prep-sup-mal}}$ samples two random vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}^k$ and computes $\Delta \cdot \mathbf{a}$, $\Delta \cdot \mathbf{b}$. Then $\mathcal{F}_{\text{prep-sup-mal}}$ samples two pairs of random degree- $(n-k)$ packed Shamir sharings $[\mathbf{a}]_{n-k} = ([a]_{n-k}, [\Delta \cdot \mathbf{a}]_{n-k})$, $[\mathbf{b}]_{n-k} = ([b]_{n-k}, [\Delta \cdot \mathbf{b}]_{n-k})$.
 - (b) $\mathcal{F}_{\text{prep-sup-mal}}$ computes $\mathbf{c} = \mathbf{a} * \mathbf{b}$ and $\Delta \cdot \mathbf{c}$. Then $\mathcal{F}_{\text{prep-sup-mal}}$ samples a random degree- $(n-1)$ packed Shamir sharing $[\mathbf{c}]_{n-1}$. For all $i \in \{1, \dots, k\}$, $\mathcal{F}_{\text{prep-sup-mal}}$ samples a random additive sharing $\langle \Delta \cdot c_i \rangle$. $\mathcal{F}_{\text{prep-sup-mal}}$ distributes the shares of $([\mathbf{a}]_{n-k}, [\mathbf{b}]_{n-k}, ([\mathbf{c}]_{n-1}, \{\langle \Delta \cdot c_i \rangle\}_{i=1}^k))$ to all parties.
4. **Preparing Random Masked Sharings for Multiplication Gates:** For each group of k multiplication gates $\mathcal{F}_{\text{prep-sup-mal}}$ sets $\mathbf{o}^{(1)} = \mathbf{o}^{(2)} = \mathbf{o}^{(3)} = \mathbf{0} \in \mathbb{F}^k$. Then $\mathcal{F}_{\text{prep-sup-mal}}$ samples three random degree- $(n-1)$ packed Shamir sharings $[\mathbf{o}^{(1)}]_{n-1}, [\mathbf{o}^{(2)}]_{n-1}, [\mathbf{o}^{(3)}]_{n-1}$ and distributes the shares to all parties.

5. **Preparing Random Sharings for Input and Output Gates:** For each group of k input gates or output gates, $\mathcal{F}_{\text{prep-sup-mal}}$ prepares the following random sharings.
- (a) $\mathcal{F}_{\text{prep-sup-mal}}$ prepares a random degree- $(n-1)$ packed Shamir sharing of $\mathbf{0} \in \mathbb{F}^k$, denoted by $[\mathbf{o}]_{n-1}$, in the same way as Step 4.
 - (b) $\mathcal{F}_{\text{prep-sup-mal}}$ also prepares a random packed Beaver triple with authentications $([\mathbf{a}]_{n-k}, [\mathbf{b}]_{n-k}, ([\mathbf{c}]_{n-1}, \{\langle \Delta \cdot c_i \rangle\}_{i=1}^k))$ in the same way as Step 3. Later we will view $\mathbf{b}$ as an authentication key and $\mathbf{c}$ as the MAC of $\mathbf{a}$. This allows the input holder to verify the correctness of $\mathbf{a}$.

Corrupted Parties: When $\mathcal{F}_{\text{prep-sup-mal}}$ prepares random sharings, corrupted parties can choose their shares. $\mathcal{F}_{\text{prep-sup-mal}}$ then samples the random sharings based on the secret it generated and the shares chosen by the corrupted parties.

Note that for semi-honest security the preprocessing is the same, except no MAC key or authentications are needed.

D.2 Useful Sub-Functionalities and Sub-Procedures

We first describe some important Sub-Functionalities and Sub-Procedures which we make use of in $\Pi_{\text{prep-st-mal}}$.

OLE $\mathcal{F}_{\text{OLE}}^{\text{prog}}$. $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ is a two party functionality that generates several random correlations between the two parties, known as oblivious linear evaluation (OLE). More specifically, the two parties P_A and P_B input $\mathbf{u} \in \mathbb{F}^\sigma$ and $\mathbf{x} \in \mathbb{F}^\sigma$, respectively, to $\mathcal{F}_{\text{OLE}}^{\text{prog}}$, which samples random $\mathbf{v} \leftarrow_{\mathbb{S}} \mathbb{F}^\sigma$, and outputs $\mathbf{w} \leftarrow \mathbf{u} * \mathbf{x} - \mathbf{v}$ to P_A and $\mathbf{v}$ to P_B . Observe that $\mathbf{x}$ remains random to P_A and $\mathbf{u}$ remains random to P_B .

Functionality 4: $\mathcal{F}_{\text{OLE}}^{\text{prog}}$

Parameters: The functionality runs between parties P_A and P_B .

On receiving $\mathbf{u} \in \mathbb{F}^\sigma$ from P_A and $\mathbf{x} \in \mathbb{F}^\sigma$ from P_B , sample $\mathbf{v} \leftarrow_{\mathbb{S}} \mathbb{F}_p^\sigma$ then output $\mathbf{w} \leftarrow \mathbf{u} * \mathbf{x} - \mathbf{v}$ to P_A and $\mathbf{v}$ to P_B .

Corrupted Party: If P_B is corrupted, $\mathbf{v}$ may be chosen by $\mathcal{A}$. If P_A is corrupted, $\mathbf{w}$ can be chosen by $\mathcal{A}$ (and $\mathbf{v}$ is computed by $\mathbf{u} * \mathbf{x} - \mathbf{w}$).

We assume the existence of a protocol that instantiates $\mathcal{F}_{\text{OLE}}^{\text{prog}}$. A number of such protocols exist [BCG⁺19b, BCG⁺20, BCGI18, WYKW21, BCG⁺19a, BDOZ11, DPSZ12, DKL⁺13].

Vector OLE $\mathcal{F}_{\text{nVOLE}}$. $\mathcal{F}_{\text{nVOLE}}$ is a functionality introduced in [RS22]. $\mathcal{F}_{\text{nVOLE}}$ provides n parties with authenticated random additive sharings. First, it gives the parties random Δ^i , which are viewed as an additive sharing of the MAC key $\Delta = \sum_{i=1}^n \Delta^i$. Then, for each party P_i , the entries of $\mathbf{u}^i = \text{Expand}(\text{seed}^i)$ which are output to P^i are viewed as additive shares held by P_i . $\mathcal{F}_{\text{nVOLE}}$ finally gives

each pair of parties P_i, P_j an additive sharing of $\mathbf{u}^i \cdot \Delta^j$, which all parties can use to form additive sharings such that the secrets are $(\sum_{i=1}^n \mathbf{u}^i) \cdot \Delta$. These are interpreted as the MACs of the additive sharings from the $\mathbf{u}^i$.

Functionality 5: $\mathcal{F}_{\text{nVOLE}}$

Parameters: An expansion function $\text{Expand} : S \rightarrow \mathbb{F}^\sigma$ with seed space S and output length σ . The functionality runs between parties $P_1, P_2, \dots, P_n$.

Initialize: On receiving **Init** from P_i for $i \in \{1, \dots, n\}$, sample $\Delta^i \leftarrow \mathbb{F}$, send it to P_i , and ignore all subsequent **Init** commands from P_i .

Extend: On receiving **Extend** from every $P_i \in \{P_1, \dots, P_n\}$:

1. Sample seed $\text{seed}^i \leftarrow S$ for all P_i .
2. Compute $\mathbf{u}^i = \text{Expand}(\text{seed}^i)$.
3. Sample $\mathbf{v}_i^j \leftarrow \mathbb{F}_p^\sigma$ for all P_i and $j \neq i$. Retrieve Δ^j and compute $\mathbf{w}_j^i = \mathbf{u}^i \cdot \Delta^j - \mathbf{v}_i^j$.
4. Output $(\text{seed}^i, (\mathbf{w}_j^i, \mathbf{v}_j^i)_{j \neq i})$ to P_i for all $P_i \in \{P_1, \dots, P_n\}$.

Corrupted Party: A corrupted party P_i can choose Δ^i and seed^i . It can also choose $\mathbf{w}_j^i$ (and $\mathbf{v}_i^j$ is computed by $\mathbf{u}^i \cdot \Delta^j - \mathbf{w}_j^i$). **Global Key Query:** If P_i is corrupted, receive (Query, Δ') from $\mathcal{A}$ with $\Delta' \in \mathbb{F}^n$. If $\Delta' = \Delta$, where $\Delta = (\Delta^1, \dots, \Delta^n)$, send **Success** to P_i and ignore any subsequent global key query. Else, send (abort, Δ) to P_i , **abort** to P_j and abort.

Packed Beaver Multiplication. Here we present the standard packed beaver multiplication procedure which we use in Π_{triple} .

Procedure 4: π_{beaver}

1. All parties hold shares of a packed beaver triple $([\mathbf{a}]_{n-k}, [\mathbf{b}]_{n-k}, [\mathbf{c}]_{n-k})$ and shares of packed inputs $([\mathbf{x}]_{n-k}, [\mathbf{y}]_{n-k})$.
2. The parties first locally compute $[\mathbf{x} - \mathbf{a}]_{n-k}$ and $[\mathbf{y} - \mathbf{b}]_{n-k}$ and open them to P_1 .
3. P_1 then reconstructs $\mathbf{d} \leftarrow \mathbf{x} - \mathbf{a}$, $\mathbf{e} \leftarrow \mathbf{y} - \mathbf{b}$ and computes then distributes sharings $[\mathbf{d}]_{k-1}, [\mathbf{e}]_{k-1}$.
4. Finally, the parties locally compute $[\mathbf{x} * \mathbf{y}]_{n-1} \leftarrow [\mathbf{d}]_{k-1} * [\mathbf{e}]_{k-1} + [\mathbf{d}]_{k-1} * [\mathbf{b}]_{n-k} + [\mathbf{e}]_{k-1} * [\mathbf{a}]_{n-k} + [\mathbf{c}]_{n-k}$.

Verifying Degrees of Sharings. Next, we have $\mathcal{F}_{\text{verify-deg}}$. This functionality takes in the honest parties' shares of M packed Shamir sharings of unknown degrees, $[\mathbf{x}_1]_{d_1}, \dots, [\mathbf{x}_M]_{d_M}$. It checks that all of their degrees are $\leq k-1$ and if not, sends **abort** to the honest parties.

Functionality 6: $\mathcal{F}_{\text{verify-deg}}$

1. $\mathcal{F}_{\text{verify-deg}}$ receives from the honest parties their shares of $[\mathbf{x}_1]_{d_1}, \dots, [\mathbf{x}_M]_{d_M}$.
2. $\mathcal{F}_{\text{verify-deg}}$ then reconstructs the sharings and sends the whole sharings $[\mathbf{x}_1]_{d_1}, \dots, [\mathbf{x}_M]_{d_M}$ to the adversary.
3. If $\exists l \in [M]$ such that $d_l > k - 1$ or $\mathcal{F}_{\text{verify-deg}}$ receives from the adversary **abort**, $\mathcal{F}_{\text{verify-deg}}$ sends **abort** to the honest parties.

We provide the following standard protocol $\Pi_{\text{verify-deg}}$ to instantiate $\mathcal{F}_{\text{verify-deg}}$. In $\Pi_{\text{verify-deg}}$, the parties simply use $\mathcal{F}_{\text{coin}}$ to sample several random values, then take a random linear combination of the $[\mathbf{x}_l]_{d_l}$ defined by the sampled values, and open the resulting sharing, $[\mathbf{x}]_d$, to each other. Since $k < n - t$, the number of honest parties and thus honest shares, the honest parties can verify that $[\mathbf{x}]_d$ is of degree- $k - 1$, even if the corrupted parties sends incorrect shares. Indeed, since the random values are sampled after the sharings are formed, with all-but-negligible probability, if any of the original sharings are not degree- $(k - 1)$, then nor will $[\mathbf{x}]_d$.

Protocol 5: $\Pi_{\text{verify-deg}}$

1. All parties take as input M packed Shamir sharings distributed by P_1 , denoted by $[\mathbf{x}_1]_{d_1}, \dots, [\mathbf{x}_M]_{d_M}$.
2. All parties invoke $\mathcal{F}_{\text{coin}}$ and receive random values $\chi_1, \dots, \chi_M$.
3. All parties locally compute $[\mathbf{x}]_d \leftarrow \sum_{l=1}^M \chi_l \cdot [\mathbf{x}_l]_{d_l}$, where $d = \max_l \{d_l\}$.
4. All parties open their shares of $[\mathbf{x}]_d$ to one another and check that the shares of $[\mathbf{x}]_d$ lie on a degree- $(k - 1)$ polynomial (i.e., $d = k - 1$). If not, all parties abort.

The communication complexity of $\Pi_{\text{verify-deg}}$ is n^2 and computation is $\tilde{O}(n^2)$. If $M = \Omega(n)$, then the complexity per underlying shared value is $\tilde{O}(1)$.

Lemma 11. $\Pi_{\text{verify-deg}}$ UC-realizes $\mathcal{F}_{\text{verify-deg}}$ in the $\mathcal{F}_{\text{coin}}$ -hybrid model.

Proof. We begin by defining the simulator $\mathcal{S}$.

1. $\mathcal{S}$ first receives from $\mathcal{F}_{\text{verify-deg}}$ the whole sharings $[\mathbf{x}_1]_{d_1}, \dots, [\mathbf{x}_M]_{d_M}$.
2. $\mathcal{S}$ then emulates $\mathcal{F}_{\text{coin}}$ by sampling random $\chi_1, \dots, \chi_M$ and sending them to the adversary.
3. Then, $\mathcal{S}$ computes the honest parties' shares of $[\mathbf{x}]_{d_{\max}} \leftarrow \sum_{l=1}^M \chi_l [\mathbf{x}]_{d_l}$, where $d_{\max} \leftarrow \max_l d_l$.
4. Finally, $\mathcal{S}$ sends to the adversary the honest parties' shares of $[\mathbf{x}]_{d_{\max}}$. Then $\mathcal{S}$ receives from the adversary its shares, and if all of the shares (including the honest parties' shares) do not lie on a polynomial of degree at most $k - 1$, then $\mathcal{S}$ sends $\mathcal{F}_{\text{verify-deg}}$, **abort**.

Now we argue that the real and ideal worlds are indistinguishable. It is clear that from the honest parties' shares, $\mathcal{F}_{\text{verify-deg}}$ will always be able to reconstruct $[\mathbf{x}_1]_{d_1}, \dots, [\mathbf{x}_M]_{d_M}$ and send them to $\mathcal{S}$, which uses the honest parties' shares.

Therefore, it is clear that the real and ideal worlds are identical, up until the point of aborting or not.

Now, we will prove that with all-but-negligible probability, an honest party aborts in the ideal world if and only if they do in the real world. $\mathcal{F}_{\text{verify-deg}}$ aborts if $\exists l^* \in [M]$ such that $d_{l^*} > k - 1$. In this case, we will show that the honest parties will also abort in the real world in $\Pi_{\text{verify-deg}}$. First, note that $n - t > k - 1$ and assume w.l.o.g., that the $(k + 1)$ -st honest party's share of $[\mathbf{x}_{l^*}]_{k-1}$ is nonzero (if all of the honest parties shares are zero, then they define the zero polynomial of degree $0 \leq k - 1$). Consider the first k honest parties' shares of $[\mathbf{x}]_{k-1}$ computed in $\Pi_{\text{verify-deg}}$, $x^{i_1}, \dots, x^{i_k}$. In order for the shares of $[\mathbf{x}]_{k-1}$ to lie on a polynomial of degree $k - 1$, it must be that the $(k + 1)$ -st honest party's share, $x^{i_{k+1}} = \sum_{j=1}^k \lambda_j \cdot x^{i_j}$ (where the λ_j are Lagrange coefficients). Letting $x_1^{i_{k+1}}, \dots, x_M^{i_{k+1}}$ be the $(k + 1)$ -st honest party's shares of $[\mathbf{x}_1]_{k-1}, \dots, [\mathbf{x}_M]_{k-1}$, respectively, we have that $x^{i_{k+1}} = \sum_{l=1}^M \chi_l \cdot x_l^{i_{k+1}}$. Thus, it must be that $\sum_{l=1}^M \chi_l \cdot x_l^{i_{k+1}} = \sum_{j=1}^k \lambda_j \cdot x^{i_j}$, which in turn means that $\chi_{l^*} = (\sum_{j=1}^k \lambda_j \cdot x^{i_j} - \sum_{l \in [M] \setminus \{l^*\}} \chi_l \cdot x_l^{i_{k+1}}) / x_{l^*}^{i_{k+1}}$. However, since χ_{l^*} is chosen independently of the honest parties' shares, this only happens with negligible probability, and so with all-but-negligible probability, the honest parties abort in the real world. $\mathcal{S}$ might also send **abort** to $\mathcal{F}_{\text{verify-deg}}$ if in $\Pi_{\text{verify-deg}}$, the adversary does not send to $\mathcal{S}$ shares that lie on the same degree- $(k - 1)$ polynomial as the honest parties' shares. However, since the honest parties' shares are identical to what they would be in the real world, the honest parties would also abort in this case in the real world. It is also clear that if honest parties do not abort in the ideal world, they also do not abort in the real world. $\square$

Generating Random Sharings. The next functionality is $\mathcal{F}_{\text{rand}}$. For any linear secret sharing scheme in $\mathbb{F}$, Σ , $\mathcal{F}_{\text{rand}}$ samples $n - t$ random Σ -sharings, each of which are consistent with shares for the corrupted parties that are input by the adversary.

Functionality 7: $\mathcal{F}_{\text{rand}}(\Sigma)$

1. $\mathcal{F}_{\text{rand}}$ receives from the adversary the corrupted parties' shares, $\{(r_j^{(1)}, \dots, r_j^{(n-t)})\}_{j \in \text{Corr}}$.
2. $\mathcal{F}_{\text{rand}}$ samples random $r^{(1)}, \dots, r^{(n-t)}$.
3. $\mathcal{F}_{\text{rand}}$ samples random sharings $(\mathbf{R}^{(1)}, \dots, \mathbf{R}^{(n-t)})$ with secrets $r^{(1)}, \dots, r^{(n-t)}$ such that for all $j \in \text{Corr}$, the j -th shares are $(r_j^{(1)}, \dots, r_j^{(n-t)})$.
4. $\mathcal{F}_{\text{rand}}$ then distributes to the honest parties their shares.

We provide the following standard protocol Π_{rand} to instantiate $\mathcal{F}_{\text{rand}}$. In Π_{rand} , each party P_i first samples their own random Σ -sharing $\mathbf{S}^{(i)}$ and distributed their shares to other parties. Then, using some agreed upon $(n - t) \times n$ super-invertible matrix $\mathbf{M}$ (for example, the transpose of a Vandermonde matrix), the parties multiply the sharings by $\mathbf{M}$ to obtain $n - t$ sharings $\mathbf{R}^{(1)}, \dots, \mathbf{R}^{(n-t)}$,

which they output. Intuitively, there are at least $n - t$ honest parties, and $\mathbf{M}$ is super-invertible, so the randomness of the $n - t$ output sharings has a one-to-one relationship with the randomness of any $n - t$ honest parties' original sharings.

Protocol 6: $\Pi_{\text{rand}}(\Sigma)$

1. All parties agree on $(n - t) \times n$ super-invertible matrix $\mathbf{M}$.
2. Each party P_i samples a random Σ -sharing $\mathbf{S}^{(i)}$ and distributes the shares to other parties.
3. All parties locally compute $(\mathbf{R}^{(1)}, \dots, \mathbf{R}^{(n-t)})^\top \leftarrow \mathbf{M} \cdot (\mathbf{S}^{(1)}, \dots, \mathbf{S}^{(n)})^\top$ and output $(\mathbf{R}^{(1)}, \dots, \mathbf{R}^{(n-t)})$.

Complexity of Π_{rand} . We let $|\Sigma|$ be the total size of sharings of Σ . Then, Π_{rand} costs $n \cdot |\Sigma|$ communication and $\tilde{O}(n \cdot |\Sigma|)$ computation, or $|\Sigma|/\varepsilon$ communication and $\tilde{O}(|\Sigma|)$ per output sharing, since $n - t > \varepsilon \cdot n = \Theta(n)$.

Lemma 12. Π_{rand} UC-realizes $\mathcal{F}_{\text{rand}}$.

Proof. First recall that we only use sharings which satisfy that there exists some $d \geq t$ such that any t shares are independent of any other set of $d - t$ shares. Now we define the simulator $\mathcal{S}$:

1. First, for each honest party P_i , $\mathcal{S}$ samples random sharing $\mathbf{S}^{(i)}$, then sends the corrupted parties their shares of the sharings.
2. $\mathcal{S}$ then receives from the adversary the honest parties' shares of the corrupted parties' sharings $\mathbf{S}^{(j)}$. $\mathcal{S}$ then for $j \in \text{Corr}$ samples a random Σ -sharing based on the honest parties shares and views it as the sharing generated by P_j .
3. $\mathcal{S}$ then computes $(\mathbf{R}^{(1)}, \dots, \mathbf{R}^{(n-t)})^\top \leftarrow \mathbf{M} \cdot (\mathbf{S}^{(1)}, \dots, \mathbf{S}^{(n)})^\top$ for the corrupted parties, and sends their shares to $\mathcal{F}_{\text{rand}}$.

Next we argue that the real and ideal worlds are distributed identically. It is clear that the honest parties distributing their random Σ -sharings $\mathbf{S}^{(i)}$ is identical in both worlds. All that remains is to argue that the shares that the honest parties output in the ideal world are identically distributed to that of the real world. For this we will use the power of super-invertible matrices.

Let t' be the number of corrupted parties after the sharing step, $\{i_1, \dots, i_{n-t}\} = H \subseteq \text{Hon}$ be any subset of Hon of size $|H| = n - t$ and $\{j_1, \dots, j_t\} = \overline{H} = [n] \setminus H$. Let $\mathbf{M}^H$ be the $(n - t) \times (n - t)$ sub-matrix of $\mathbf{M}$ whose columns correspond to honest parties' indices in H . We know that $\mathbf{M}^H$ is invertible. Thus, we can write

$$(\mathbf{S}^{(i_1)}, \dots, \mathbf{S}^{(i_{n-t})})^\top = (\mathbf{M}^H)^{-1} \cdot ((\mathbf{R}^{(1)}, \dots, \mathbf{R}^{(n-t)})^\top - \mathbf{M}^{\overline{H}} \cdot (\mathbf{S}^{(j_1)}, \dots, \mathbf{S}^{(j_t)})^\top).$$

This means that the distribution of $(\mathbf{R}^{(1)}, \dots, \mathbf{R}^{(n-t)})$ is equivalent to that of $(\mathbf{S}^{(i_1)}, \dots, \mathbf{S}^{(i_{n-t})})$, given any fixed $\mathbf{S}^{(j_1)}, \dots, \mathbf{S}^{(j_t)}$. So, for any fixed sharings $\mathbf{S}^{(j_1)}, \dots, \mathbf{S}^{(j_t)}$, the distribution of $(\mathbf{R}^{(1)}, \dots, \mathbf{R}^{(n-t)})$ is random given the shares of corrupted parties. This means that in the real world, the shares that honest parties output are random given those of the corrupted parties. In the ideal

world, $\mathcal{S}$ generates the shares of each $\mathbf{R}^{(i)}$ of the corrupted parties by randomly sampling their shares of the $\mathbf{S}^{(j_1)}, \dots, \mathbf{S}^{(j_t)}$ based on the shares of honest parties. Then, the shares of each $\mathbf{R}^{(i)}$ of the honest parties are randomly sampled based on the shares of the corrupted parties. Thus, the distribution of the shares of the honest parties in both worlds is identical. $\square$

Transforming Packed Shamir Sharings to Additive Sharings. Note that if the parties hold some packed Shamir secret sharing $[\mathbf{r}]_{n-k} = (w_1, \dots, w_n)$, then they can locally convert it to an additive sharing $\langle r_i \rangle = (v_1, \dots, v_n)$ of the i -th element of $\mathbf{r}$. Indeed, letting $L_j(-i+1)$ be the j -th Lagrange coefficient, party P_j sets their share of $\langle r_i \rangle$ to be $v_j \leftarrow L_j(-i+1) \cdot w_j$. This works since $\sum_{j=1}^n L_j(-i+1) \cdot w_j = r_i$, by definition. However, the sharing $\langle r_i \rangle$ is not uniformly random.

To get a uniformly random sharing, the parties can invoke $\mathcal{F}_{\text{rand}}$ to prepare a random sharing of the form $\langle 0 \rangle$, then reset $\langle r_i \rangle \leftarrow \langle r_i \rangle + \langle 0 \rangle$.

Transforming Additive Sharing to Packed Shamir Sharing. We now present $\pi_{\text{MAC-key}}$, which allows the parties to use some random sharings to convert their additive sharing of $\langle \Delta \rangle$ to sharings $\{[\Delta]_t\}_{t=1}^k$ and $[\Delta \cdot \mathbf{1}]_{n-k}$ which are needed for $\Pi_{\text{prep-sup-mal}}$. This is based on the standard “mask-then-open” paradigm.

Procedure 7: $\pi_{\text{MAC-key}}$

Input: All parties hold an additive sharing $\langle \Delta \rangle$, a tuple of random sharings $\{[r]_t\}_{t=1}^k$, and a random sharing $[\rho \cdot \mathbf{1}]_{n-k}$.

1. All parties locally transform $[r]_t$ to an additive sharing $\langle r \rangle$ and locally refresh the obtained additive sharing. They also locally transform $[\rho \cdot \mathbf{1}]_{n-k}$ to an additive sharing $\langle \rho \rangle$ and locally refresh the obtained additive sharing.
2. All parties locally compute $\langle \Delta + r \rangle = \langle \Delta \rangle + \langle r \rangle$ and send their shares to P_1 . They also locally compute $\langle \Delta + \rho \rangle = \langle \Delta \rangle + \langle \rho \rangle$ and send their shares to P_1 .
3. P_1 reconstructs the secret $\Delta + r$ and reshares the secret by $[(\Delta + r) \cdot \mathbf{1}]_{2k-2}$. Note that $[(\Delta + r) \cdot \mathbf{1}]_{2k-2}$ can be viewed as $[\Delta + r]_{2k-2}$ for all $i \in \{1, \dots, k\}$. P_1 also reconstructs $\Delta + \rho$ and reshares the secret by $[(\Delta + \rho) \cdot \mathbf{1}]_{2k-2}$.
4. For all $i \in \{1, \dots, k\}$, all parties locally compute $[\Delta]_t = [(\Delta + r) \cdot \mathbf{1}]_{2k-2} - [r]_t$. They also locally compute $[\Delta \cdot \mathbf{1}]_{n-k} = [(\Delta + \rho) \cdot \mathbf{1}]_{2k-2} - [\rho \cdot \mathbf{1}]_{n-k}$.

Preparing Random Redundant Sharings with Authentications We now present a procedure from SuperPack, which uses $\mathcal{F}_{\text{nVOLE}}$ and random packed sharings to generate random redundant authenticated sharings of the form $([r \cdot \mathbf{1}]_{n-k}, \langle \Delta \cdot r \rangle)$. This again uses the standard “mask-then-open” paradigm.

The parties use $\mathcal{F}_{\text{nVOLE}}$ in a clever way, introduced by [EGP⁺23], to actually interpret each element of $\mathbf{u}^i$ output by $\mathcal{F}_{\text{nVOLE}}$ as degree- $(n-1)$ random packed Shamir sharings. This works as such degree- $(n-1)$ random packed Shamir sharings have the same distribution as additive sharings.

Procedure 8: $\pi_{\text{rand-red-auth}}$

1. Each party P_i calls $\mathcal{F}_{\text{nvole}}$ with input **Extend** and receives the seed seed^i . They use the outputs to define degree- $(n-1)$ packed Shamir sharings $\{[r_\ell]_{n-1}\}_{\ell=1}^\sigma$, where σ is the output length of the expansion function defined in $\mathcal{F}_{\text{nvole}}$, such that the i -th shares of $\{[r_\ell]_{n-1}\}_{\ell=1}^\sigma$ are $\text{Expand}(\text{seed}^i)$. All parties locally compute and refresh $\{(\langle \Delta \cdot r_{\ell,1} \rangle, \dots, \langle \Delta \cdot r_{\ell,k} \rangle)\}_{\ell=1}^\sigma$.
2. All parties invoke $\mathcal{F}_{\text{rand}}$ to prepare $k \cdot \sigma$ sharings in the form of $[\rho_i \cdot \mathbf{1}]_{n-k}$. For all $\ell \in \{1, \dots, \sigma\}$ consume k fresh random sharings $\{[\rho_i \cdot \mathbf{1}]_{n-k}\}_{i=1}^k$ and run the following steps.
 - (a) All parties locally compute $[r_\ell + \rho]_{n-1} = [r_\ell]_{n-1} + \sum_{i=1}^k \mathbf{e}_i * [\rho_i \cdot \mathbf{1}]_{n-k}$, where $\rho = (\rho_1, \dots, \rho_k)$. Then all parties send their shares of $[r_\ell + \rho]_{n-1}$ to P_1 .
 - (b) P_1 reconstructs the secrets $r_\ell + \rho$ and sends the secrets to all parties.
 - (c) For all $i \in \{1, \dots, k\}$, all parties locally compute $[r_{\ell,i} \cdot \mathbf{1}]_{n-k} = (r_{\ell,i} + \rho_i) - [\rho_i \cdot \mathbf{1}]_{n-k}$. Recall that all parties hold $\langle \Delta \cdot r_{\ell,i} \rangle$. Therefore all parties hold $([r_{\ell,i} \cdot \mathbf{1}]_{n-k}, \langle \Delta \cdot r_{\ell,i} \rangle)$.
3. Finally all parties check whether P_1 sends the same values to all parties:
 - (a) Suppose P_1 distributes $\tau_1, \dots, \tau_\sigma$ to all parties.
 - (b) All parties call $\mathcal{F}_{\text{coin}}$ to obtain random values $\chi_1, \dots, \chi_\sigma \in \mathbb{F}$.
 - (c) All parties locally compute $\tau = \sum_{\ell=1}^\sigma \chi_\ell \cdot \tau_\ell$.
 - (d) All parties exchange their results τ and check whether all results are identical. If not, all parties abort.

Complexity of $\pi_{\text{rand-red-auth}}$. We have that

1. Step 1 costs $O(n)$ computation per sharing $\langle \Delta \cdot r_{\ell,i} \rangle$.
2. Step 2 costs n elements and $\tilde{O}(n)$ computation per sharing $[\rho_i \cdot \mathbf{1}]_{n-k}$.
3. Step 2a costs $(n-1)/k$ elements and $O(1)$ computation per slot.
4. Step 2b costs $(n-1)$ elements and $O(n)$ computation per slot.

The above totals $2n + 2/\varepsilon - 2/\varepsilon n - 1$ elements and $\tilde{O}(n)$ computation for each $\ell \in \sigma$ and $i \in [k]$.

Degree Reduction. Functionality $\mathcal{F}_{\text{deg-reduce}}$ takes in $n-t$ degree- $(n-1)$ packed Shamir sharings $[\mathbf{u}_i]_{n-1}$ from the parties and samples $n-t$ degree- $(n-k)$ random packed Shamir sharings of the same underlying secrets, each of which are consistent with corrupted parties' shares that are input by the adversary.

Functionality 8: $\mathcal{F}_{\text{deg-reduce}}$

1. $\mathcal{F}_{\text{deg-reduce}}$ first receives from all parties their shares of the sharings $\{[\mathbf{u}_1]_{n-1}, \dots, [\mathbf{u}_{n-t}]_{n-1}\}$ and reconstructs $\mathbf{u}_1, \dots, \mathbf{u}_{n-t}$.
2. $\mathcal{F}_{\text{deg-reduce}}$ then receives from the adversary the corrupted parties' shares $\{u_1^j, \dots, u_{n-t}^j\}_{j \in \text{Corr}}$ of the new sharings.
3. Finally, $\mathcal{F}_{\text{deg-reduce}}$ samples random sharings $[\mathbf{u}_1]_{n-k}, \dots, [\mathbf{u}_{n-t}]_{n-k}$ based on the corrupted parties' shares and distributes to the honest parties their shares.

To instantiate $\mathcal{F}_{\text{deg-reduce}}$, we use the following standard technique. First, the parties invoke $\mathcal{F}_{\text{rand}}$ to prepare $n-t$ random sharings of the form $([r_l]_{n-k}, [r_l]_{n-1})$, for $l \in [n-t]$. Then, the parties for each l compute $[u_l + r_l]_{n-1}$ and open the sharings to P_1 . Since the sharing $[r_l]_{n-1}$ is random, so too is $[u_l + r_l]_{n-1}$, thus nothing is revealed about u_l . Then P_1 reconstructs each $u_l + r_l$ and distributes sharings $[u_l + r_l]_{k-1}$ to the other parties, from which they can compute $[u_l]_{n-k}$ by subtracting $[r_l]_{n-k}$. We use $\mathcal{F}_{\text{verify-deg}}$ to ensure that P_1 did not cheat by sending a sharing of a higher degree. We present the protocol $\Pi_{\text{deg-reduce}}$ below.

Protocol 9: $\Pi_{\text{deg-reduce}}$

1. All parties hold degree- $(n-1)$ packed Shamir sharings $[u_1]_{n-1}, \dots, [u_{n-t}]_{n-1}$.
2. They first invoke $\mathcal{F}_{\text{rand}}$ to prepare random sharings $([r_1]_{n-k}, [r_1]_{n-1}), \dots, ([r_{n-t}]_{n-k}, [r_{n-t}]_{n-1})$.
3. For all $l \in [n-t]$, all parties locally compute $[u_l + r_l]_{n-1} \leftarrow [u_l]_{n-1} + [r_l]_{n-1}$ and open it to P_1 .
4. P_1 locally reconstructs each $u_l + r_l$ and then distributes sharings $[u_l + r_l]_{2(k-1)}$.
5. Next, the parties invoke $\mathcal{F}_{\text{verify-deg}}$ on the $[u_l + r_l]_{k-1}$ sharings.
6. Finally, all parties locally compute for each $l \in [n-t]$, $[u_l]_{n-k} \leftarrow [u_l + r_l]_{2k-2} - [r_l]_{n-k}$.

Complexity of $\Pi_{\text{deg-reduce}}$. We have that

1. Step 2 costs n elements and $\tilde{O}(n)$ computation per sharing.
2. Step 3 costs $n-1$ elements per sharing.
3. Step 4 costs $n-(k-1)$ elements (the first $k-1$ parties can have shares that are 0) and $\tilde{O}(n)$ computation per sharing.

The above totals $3n-k$ communication and $\tilde{O}(n)$ computation per sharing. This is $6/\varepsilon - 1$ communication and $\tilde{O}(1)$ computation per underlying slot.

Lemma 13. $\Pi_{\text{deg-reduce}}$ UC-realizes $\mathcal{F}_{\text{deg-reduce}}$ in the $\mathcal{F}_{\text{rand}}$ -hybrid model.

Proof. First we define the simulator $\mathcal{S}$:

1. $\mathcal{S}$ first emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary $\{(s_j^1, r_j^1), \dots, (s_j^{n-t}, r_j^{n-t})\}$ and sampling random sharings $([r_1]_{n-k}, [r_1]_{n-1}), \dots, ([r_{n-t}]_{n-k}, [r_{n-t}]_{n-1})$ consistent with the above as the corrupt parties' shares. Note that if the adversary corrupts additional parties after the invocation of $\mathcal{F}_{\text{rand}}$, $\mathcal{S}$ can just send them their shares.
2. Next:
 - (a) If P_1 is corrupt, for $l \in [n-t]$, $\mathcal{S}$ sends the adversary random shares t_l^i on behalf of each honest party.
 - (b) If P_1 is honest, $\mathcal{S}$ receives from the adversary the corrupted parties' shares of $[u_l + r_l]_{n-1}$. Using the corrupt parties' shares of $[r_l]_{n-1}$ received from the adversary above, $\mathcal{S}$ computes their shares of each $[u_l]_{n-1}$, then sends them to $\mathcal{F}_{\text{deg-reduce}}$.

3. Then:

- (a) If P_1 is corrupt, then $\mathcal{S}$ receives from the adversary the honest parties' shares of each $[\mathbf{u}_l + \mathbf{r}_l]_{k-1}$. $\mathcal{S}$ can then reconstruct $\mathbf{u}_l + \mathbf{r}_l$ using the first k honest parties' shares and since it sampled $\mathbf{r}_l$ above, compute $\mathbf{u}_l$. Then, $\mathcal{S}$ can compute for each honest party $u_l^i \leftarrow t_l^i - r_l^i$ and based on these and $\mathbf{u}_l$, samples the corrupt parties' shares of each $[\mathbf{u}_l]_{n-1}$ and sends them to $\mathcal{F}_{\text{deg-reduce}}$.
 - (b) If P_1 is honest, $\mathcal{S}$ samples random sharings $[\mathbf{t}_l]_{k-1}$ and sends the corrupted parties their shares.
4. Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{verify-deg}}$ by first sending the adversary the corrupted parties' shares of the sharings $[\mathbf{u}_l + \mathbf{r}_l]_{k-1}$ then aborting if P_1 is corrupted and sent sharings of degree greater than $k - 1$.
 5. Lastly, using $[\mathbf{u}_l + \mathbf{r}_l]_{k-1}$ in the former case and $[\mathbf{t}_l]_{k-1}$ in the latter case, as well as the shares of the corrupted parties of $[\mathbf{r}_l]_{n-k}$ received from the adversary above, $\mathcal{S}$ can compute the corrupted parties' shares of each $[\mathbf{u}_l]_{n-k}$ and send them to $\mathcal{F}_{\text{deg-reduce}}$.

Now we argue that the real and ideal worlds are distributed identically. Let us begin with the case in which P_1 is corrupt. Since each $[\mathbf{r}_l]_{n-1}$ is random of degree- $(n - 1)$, so too is $[\mathbf{u}_l + \mathbf{r}_l]_{n-1}$, and therefore, the honest parties' shares in the real world are random, as the t_l^i in the ideal world are. Since honest parties always send $t_l^i = u_l^i + r_l^i$ in Step 3, it must be that $u_l^i = t_l^i - r_l^i$. Therefore, the $\mathbf{u}_1, \dots, \mathbf{u}_{n-t}$ that $\mathcal{F}_{\text{deg-reduce}}$ reconstructs is the same as that defined by the corrupt P_1 in the real world. Finally, since $\mathcal{F}_{\text{deg-reduce}}$ samples the sharings $[\mathbf{u}_l]_{n-k}$ based on the corrupted parties' shares, the output honest parties' shares are distributed identically in the two worlds.

Now, if P_1 is honest, it is clear that $\mathcal{S}$ computes the correct corrupted parties' shares of each $[\mathbf{u}_l]_{n-1}$ and thus the $\mathbf{u}_1, \dots, \mathbf{u}_{n-t}$ that $\mathcal{F}_{\text{deg-reduce}}$ reconstructs is the same as that defined by the corrupt parties' shares in the real world. Now, it is clear that the underlying $\mathbf{r}_l$ are unknown and random to the adversary. Thus, so too is the underlying $\mathbf{u}_l + \mathbf{r}_l$ and so $[\mathbf{t}_l]_{k-1}$ in the ideal world is distributed identically to that of $[\mathbf{u}_l + \mathbf{r}_l]_{k-1}$ in the real world. Finally, as above, since $\mathcal{F}_{\text{deg-reduce}}$ samples the sharings $[\mathbf{u}_l]_{n-k}$ based on the corrupted parties' shares, the output honest parties' shares are distributed identically in the two worlds. $\square$

Triple sacrificing. The triples produced by $\mathcal{F}_{\text{triple}}$ may have some error. Thus, we present the following standard procedure $\pi_{\text{sacrifice}}$, which takes two packed triples output from $\mathcal{F}_{\text{triple}}$ that have been authenticated and sacrifices one of them to ensure that both have no error (including the other which is output). Note that although corrupted parties can change their shares of authenticated packed triples to change the underlying secrets (and thus add back error), they cannot do so without getting caught in the online phase, since the MAC key Δ is random and unknown to them.

Procedure 10: $\pi_{\text{sacrifice}}$

1. All parties take as input packed triple sharings $(\llbracket \mathbf{a}_1 \rrbracket_{n-k}, \llbracket \mathbf{b}_1 \rrbracket_{n-k}, \llbracket \mathbf{c}_1 \rrbracket_{n-k})$ and $(\llbracket \mathbf{a}_2 \rrbracket_{n-k}, \llbracket \mathbf{b}_2 \rrbracket_{n-k}, \llbracket \mathbf{c}_2 \rrbracket_{n-k})$.
2. All parties invoke $\mathcal{F}_{\text{coin}}$ and receive a random element $\rho \in \mathbb{F}$.
3. Then they compute $\llbracket \mathbf{a}_2 - \rho \cdot \mathbf{a}_1 \rrbracket_{n-k}$ and $\llbracket \mathbf{b}_2 - \mathbf{b}_1 \rrbracket_{n-k}$ and open them to P_1 .
4. P_1 next reconstructs $\mathbf{a}_2 - \rho \cdot \mathbf{a}_1$ and $\mathbf{b}_2 - \mathbf{b}_1$, then distributes packed Shamir sharings $\llbracket \mathbf{a}_2 - \rho \cdot \mathbf{a}_1 \rrbracket_{k-1}$, $\llbracket \mathbf{b}_2 - \mathbf{b}_1 \rrbracket_{k-1}$, and $\llbracket (\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1) \rrbracket_{k-1}$.
5. Then, the parties compute $\llbracket \Delta \cdot \boldsymbol{\theta} \rrbracket_{n-1} \leftarrow \llbracket \Delta \cdot \mathbf{1} \rrbracket_{n-k} * \llbracket (\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1) \rrbracket_{k-1} + \llbracket \Delta \cdot \mathbf{b}_1 \rrbracket_{n-k} * \llbracket \mathbf{a}_2 - \rho \cdot \mathbf{a}_1 \rrbracket_{k-1} + \rho \cdot \llbracket \Delta \cdot \mathbf{a}_1 \rrbracket_{n-k} * \llbracket \mathbf{b}_2 - \mathbf{b}_1 \rrbracket_{k-1} + \rho \cdot \llbracket \Delta \cdot \mathbf{c}_1 \rrbracket_{n-k} - \llbracket \Delta \cdot \mathbf{c}_2 \rrbracket_{n-k}$.

Complexity of $\pi_{\text{sacrifice}}$. We ignore calls to $\mathcal{F}_{\text{coin}}$.

1. Step 3 costs $2(n-1)$ elements.
2. Step 4 costs $3(n-1)$ elements and $\tilde{O}(n)$ computation.

The above totals $5(n-1)$ communication and $\tilde{O}(n)$ computation per sharing, or $10/\varepsilon - 10/(\varepsilon n)$ communication and $\tilde{O}(1)$ computation per individual slot.

Checking that Many Sharings Share 0. Lastly, we present a procedure $\pi_{\text{check-zero}}$, which takes as input M degree- $(n-1)$ packed Shamir sharings, $[\boldsymbol{\theta}_1]_{n-1}, \dots, [\boldsymbol{\theta}_M]_{n-1}$ and ensures that each of them share underlying value $\mathbf{0}$. To do so, the parties use $\mathcal{F}_{\text{coin}}$ to generate several random values, then take a random linear combination of the $[\boldsymbol{\theta}_i]_{n-1}$ defined by these values to obtain $[\boldsymbol{\theta}]_{n-1}$. The parties then use $\mathcal{F}_{\text{commit}}$ to commit to their shares, then open the shares to each other and check whether the underlying secret is $\mathbf{0}$ or not. A minor subtlety is that the input sharings $[\boldsymbol{\theta}_i]_{n-1}$ may not be randomly distributed degree- $(n-1)$ packed Shamir sharings (if for example they are the multiplication of degree- $(n-k)$ and- $(k-1)$ packed Shamir sharings). Thus, the parties re-randomize $[\boldsymbol{\theta}]_{n-1}$ with a random degree- $(n-1)$ packed Shamir sharing $[\mathbf{0}]_{n-1}$, obtained from $\mathcal{F}_{\text{rand}}$.

Procedure 11: $\pi_{\text{check-zero}}$

1. All parties take as input M degree- $(n-1)$ packed Shamir sharings, denoted by $[\boldsymbol{\theta}_1]_{n-1}, \dots, [\boldsymbol{\theta}_M]_{n-1}$.
2. All parties invoke $\mathcal{F}_{\text{coin}}$ and receive random values $\chi_1, \dots, \chi_M$.
3. All parties invoke $\mathcal{F}_{\text{rand}}$ to prepare a random degree- $(n-1)$ packed Shamir sharing of $\mathbf{0}$, denoted by $[\mathbf{0}]_{n-1}$.
4. All parties locally compute $[\boldsymbol{\theta}]_{n-1} \leftarrow [\mathbf{0}]_{n-1} + \sum_{i=1}^M \chi_i \cdot [\boldsymbol{\theta}_i]_{n-1}$.
5. All parties invoke $\mathcal{F}_{\text{commit}}$ to commit their shares of $[\boldsymbol{\theta}]_{n-1}$ to one another.
6. All parties open the commitments of the shares of $[\boldsymbol{\theta}]_{n-1}$ to one another and check whether it is a degree- $(n-1)$ packed Shamir sharing of $\mathbf{0}$. If not, all parties abort. If not, all parties abort.

The communication complexity of $\pi_{\text{check-zero}}$ is $kn/(n-t) + n^2$ and computational complexity is $\tilde{O}(n^2)$. If $M = \Omega(n)$, this is $\tilde{O}(1)$ per underlying shared value.

Assume that for at least one of the input sharings $[\theta_l]_{n-1}$ such that the underlying vector $\theta_l \neq \mathbf{0}$, this underlying vector is random and unknown to the adversary. We now provide the following lemma which shows that if the above holds, then the adversary cannot force $[\theta]_{n-1}$ to open to $\mathbf{0}$.

Lemma 14. *If $\exists l^* \in [M]$ such that $\theta_{l^*} \neq \mathbf{0}$ and is random and unknown to the adversary, then the honest parties will abort with all-but-negligible probability at the end of $\pi_{\text{check-zero}}$.*

Proof. First note that since the adversary uses $\mathcal{F}_{\text{commit}}$ to commit to its shares, the shares are independent of the underlying value $\theta = \sum_{l=1}^M \chi_l \cdot \theta_l$. If the parties do not abort, then by definition $\theta = \mathbf{0}$, so we have that $\mathbf{0} = \sum_{l=1}^M \chi_l \cdot \theta_l$. Thus, we have that $\theta_{l^*} = -(\sum_{l \in [M] \setminus \{l^*\}} \chi_l \cdot \theta_l) / \chi_{l^*}$, which can only happen with negligible probability, since θ_{l^*} is random and unknown to the adversary. $\square$

D.3 $\Pi_{\text{prep-sup-mal}}$ Protocol

Finally, we describe the rest of $\Pi_{\text{prep-sup-mal}}$. First, $\Pi_{\text{prep-sup-mal}}$ uses $\mathcal{F}_{\text{nVOLE}}$ along with $\mathcal{F}_{\text{rand}}$ and $\pi_{\text{MAC-key}}$ to obtain the relevant sharings of the MAC key. $\Pi_{\text{prep-sup-mal}}$ then uses $\pi_{\text{rand-red-auth}}$ to prepare the random sharings $([r \cdot \mathbf{1}]_{n-k}, \langle \Delta \cdot r \rangle)$ for each multiplication and input gate. Most notably, for packed, authenticated beaver triples, $\Pi_{\text{prep-sup-mal}}$ first invokes $\mathcal{F}_{\text{triple}}$ to receive (unauthenticated) packed beaver triples. Then, after degree-reducing the ‘c components’, $\Pi_{\text{prep-sup-mal}}$ uses π_{auth} to authenticate each component of the triples, followed by running $\pi_{\text{sacrifice}}$ on them, along with the associated checks of $\pi_{\text{check-zero}}$ and $\mathcal{F}_{\text{verify-deg}}$, to ensure that none of them have any error. The parties also use $\mathcal{F}_{\text{rand}}$ to sample some random sharings needed throughout.

Protocol 12: $\Pi_{\text{prep-sup-mal}}$

Initialization: For every order pair of parties (P_i, P_j) , P_i sends a random PRG seed $s_{i,j}$ to P_j . Whenever P_i needs to sample a random value and sends it to P_j , P_i and P_j locally evaluate the PRG with random seed $s_{i,j}$ to obtain the common random value.

1. **Settling Authentication Keys:** Each P_i calls $\mathcal{F}_{\text{nVOLE}}$ with input Init and receives Δ^i . Next, all parties invoke $\mathcal{F}_{\text{rand}}$ to prepare random sharings $\{[r|_i]_t\}_{i=1}^k$ and $[\rho \cdot \mathbf{1}]_{n-k}$, then invoke $\pi_{\text{MAC-key}}$ to obtain $\{[\Delta|_i]_t\}_{i=1}^k [\Delta \cdot \mathbf{1}]_{n-k}$.
2. **Preparing Random Packed Sharings:** Let N_1 be the number of input gates and multiplication gates. All parties invoke $\pi_{\text{rand-red-auth}}$ to prepare N_1 random sharings in the form of $([r \cdot \mathbf{1}]_{n-k}, \langle \Delta \cdot r \rangle)$.
3. **Preparing Packed Beaver Triples with Authentications:** Let N_2 denote the number of multiplication/input/output gates. Repeat the following, until $\geq N_2$ triples are generated:
 - (a) All parties invoke $\mathcal{F}_{\text{triple}}$ two times to generate 2μ packed Beaver triples $[\mathbf{a}_l]_{n-k}, [\mathbf{b}_l]_{n-k}, [\mathbf{c}_l + \delta_l]_{n-1}$ for $l \in [2\mu]$.
 - (b) Next, they invoke $\mathcal{F}_{\text{deg-reduce}}$ on input $[\mathbf{c}_l + \delta_l]_{n-1}$ ($n-t$ at a time, in parallel) to obtain $[\mathbf{c}_l + \delta_l]_{n-k}$.

- (c) The parties input each component of $[a_l]_{n-k}, [b_l]_{n-k}, [c_l + \delta_l]_{n-k}$, for $l \in [2\mu]$, $(n-t)$ at a time to π_{auth} to obtain $\llbracket a_l \rrbracket_{n-k}, \llbracket b_l \rrbracket_{n-k}, \llbracket c_l + \delta_l \rrbracket_{n-k}$.
 - (d) Then, all parties run $\pi_{\text{sacrifice}}$ on $\llbracket a_l \rrbracket_{n-k}, \llbracket b_l \rrbracket_{n-k}, \llbracket c_l + \delta_l \rrbracket_{n-k}$ and $\llbracket a_{\mu+l} \rrbracket_{n-k}, \llbracket b_{\mu+l} \rrbracket_{n-k}, \llbracket c_{\mu+l} + \delta_{\mu+l} \rrbracket_{n-k}$ to compute $\llbracket \Delta \cdot \theta_l \rrbracket_{n-1}$, for $l \in [\mu]$, then input these to $\pi_{\text{check-zero}}$ to verify the correctness of the triples, and finally invoke $\mathcal{F}_{\text{verify-deg}}$ on input all of the degree- $(k-1)$ sharings from $\pi_{\text{sacrifice}}$.
 - (e) If the parties do not abort, this must mean that each $\delta_l = \mathbf{0}$, and so the parties have $\llbracket a_l \rrbracket_{n-k}, \llbracket b_l \rrbracket_{n-k}, \llbracket c_l \rrbracket_{n-k}$ for $l \in [\mu]$. All parties locally compute and refresh $\{\langle \Delta \cdot c_{l,i} \rangle\}_{i=1}^k$ from $\llbracket \Delta \cdot c_l \rrbracket_{n-k}$, for $l \in [\mu]$.
4. **Preparing Random Masked Sharings for Multiplication Gates:** Let N_3 denote the number of groups of multiplication gates. All parties invoke $\mathcal{F}_{\text{rand}}$ to prepare $3N_3$ random sharings in the form $[0]_{n-1}$.
5. **Preparing Random Masked Sharings for Input and Output Gates:** Let N_4 be the number of groups of input gates and output gates. We have that $N_2 = N_3 + N_4$. All parties invoke $\mathcal{F}_{\text{rand}}$ to prepare N_4 random sharings in the form of $[0]_{n-1}$.

Note that for semi-honest security, we essentially just need $\mathcal{F}_{\text{triple}}$ for the multiplication triples (nothing else from Step 3), and no MAC key or MACs in all of $\Pi_{\text{prep-sup-mal}}$.

Complexity of $\Pi_{\text{prep-sup-mal}}$. We will focus on the complexity needed to generate the preprocessing required for multiplication gates: Step 2 runs $\pi_{\text{rand-red-auth}}$, which costs $2n + 2/\varepsilon - 2/\varepsilon n - 1$ elements and $\tilde{O}(n)$ computation per sharing. Step 3a invokes $\mathcal{F}_{\text{triple}}$, which costs $2/(\varepsilon(\gamma - 1/2)) \cdot (4 + \zeta - 2/n)$ communication and $\tilde{O}(1)$ computation and OLE correlations per underlying packed triple (which is run twice per multiplication gate). Step 3b invokes $\mathcal{F}_{\text{deg-reduce}}$, which costs $6/\varepsilon - 1$ communication and $\tilde{O}(1)$ computation and OLE correlations per slot (which is run twice per multiplication gate). Step 3c runs π_{auth} , which costs $2/(\varepsilon(\gamma - 1/2)) \cdot (4 + \zeta - 2/n) + 20/\varepsilon - 8/(\varepsilon n) - 2$ communication and $\tilde{O}(1)$ computation and OLE correlations per slot (which is run six times per multiplication gate). Step 3d runs $\pi_{\text{sacrifice}}$, which costs $10/\varepsilon - 10/(\varepsilon n)$ communication and $\tilde{O}(1)$ computation per pair of underlying triples. We ignore the costs of $\mathcal{F}_{\text{deg-reduce}}$ and $\pi_{\text{check-zero}}$, since they are independent of $\gamma \cdot m$. Steps 4 invokes $\mathcal{F}_{\text{rand}}$ for Σ such that $|\Sigma| = 1$, which costs $1/\varepsilon$ communication and $\tilde{O}(1)$ computation per slot (which is run thrice per multiplication gate).

This totals $2n + 16/(\varepsilon(\gamma - 1/2)) \cdot (4 + \zeta - 2/n) + 145/\varepsilon - 60/(\varepsilon n) - 15$ communication and $\tilde{O}(1)$ computation and OLE correlations for every multiplication gate. For constants $\zeta > 0, \gamma > 1/2$ from Section B.5, this gives $2n + O(1)$ communication per gate.

Theorem 10. $\Pi_{\text{prep-sup-mal}}$ UC-realizes $\mathcal{F}_{\text{prep-sup-mal}}$ in the $(\mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{triple}}, \mathcal{F}_{\text{auth-rand}}, \mathcal{F}_{\text{deg-reduce}}, \mathcal{F}_{\text{verify-deg}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{commit}})$ -hybrid model against a malicious, adaptive adversary who controls $t < (1 - \varepsilon)n$ parties.

Proof. We construct a simulator $\mathcal{S}$:

1. **Initialization:** $\mathcal{S}$ just sends on behalf of every honest party P_i a random PRG seed to every corrupted party P_j . Note that if the adversary later corrupts more parties, $\mathcal{S}$ can just simulate their PRG seeds for other honest parties randomly. We later omit the simple hybrids which transform prg outputs to random values.
2. **Settling Authentication Keys:** $\mathcal{S}$ emulates $\mathcal{F}_{\text{nVOLE}}$ (by collecting Δ^i from corrupted parties and just internally sampling Δ^i for honest parties). Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary the corrupted parties' shares of $\{[r|_i]_t\}_{i=1}^k$ and $[\rho \cdot \mathbf{1}]_{n-k}$ and randomly sampling the rest of the sharings based on the corrupted parties' shares. Based on this $\mathcal{S}$ simulates $\pi_{\text{MAC-key}}$ as follows:
 - (a) $\mathcal{S}$ first computes all shares of $\langle r \rangle$ and $\langle \rho \rangle$.
 - (b) Then if P_1 is corrupted, $\mathcal{S}$ sends to the adversary the honest parties' shares of $\langle \Delta + r \rangle$ and $\langle \Delta + \rho \rangle$. Otherwise, it receives from the adversary the corrupted parties' shares of $\langle \Delta + r + \varepsilon \rangle$ and $\langle \Delta + \rho + \eta \rangle$, where ε and η are errors added by the adversary.
 - (c) Then if P_1 is corrupted, $\mathcal{S}$ receives from the adversary $[(\Delta + r + \varepsilon) \cdot \mathbf{1}]_{2k-2}$ and $[(\Delta + \rho + \eta) \cdot \mathbf{1}]_{2k-2}$, where ε and η are errors added by the adversary. If P_1 is honest, $\mathcal{S}$ computes sharings $[(\Delta + r + \varepsilon) \cdot \mathbf{1}]_{2k-2}$ and $[(\Delta + \rho + \eta) \cdot \mathbf{1}]_{2k-2}$ and sends the corrupted parties their shares.
 - (d) In either case, $\mathcal{S}$ can compute the sharings $[\Delta + \varepsilon|_i]_t = [(\Delta + r + \varepsilon) \cdot \mathbf{1}]_{2k-2} - [r|_i]_t$ for all $i \in \{1, \dots, k\}$ and $[(\Delta + \eta) \cdot \mathbf{1}]_{n-k} = [(\Delta + \rho + \eta) \cdot \mathbf{1}]_{2k-2} - [\rho \cdot \mathbf{1}]_{n-k}$.
 - (e) Finally, $\mathcal{S}$ sends to $\mathcal{F}_{\text{prep-sup-mal}}$ the corrupted parties' shares.
3. **Preparing Random Packed Sharings:** To simulate $\pi_{\text{rand-red-auth}}$, $\mathcal{S}$ starts by setting $\text{bad-distr} \leftarrow 0$, then:
 - (a) $\mathcal{S}$ first emulates $\mathcal{F}_{\text{nVOLE}}$ and uses this emulation to compute all shares of $\{[r_\ell]_{n-1}\}_{\ell=1}^\sigma$ and $\{\langle \Delta \cdot r_{\ell,1} \rangle, \dots, \langle \Delta \cdot r_{\ell,k} \rangle\}_{\ell=1}^\sigma$.
 - (b) Next $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ to compute sharings $[\rho_i \cdot \mathbf{1}]_{n-k}$, then:
 - i. If P_1 is corrupted, $\mathcal{S}$ sends to the adversary the honest parties' shares of $[r_\ell + \rho]_{n-1}$. Otherwise, it receives from the adversary the corrupted parties' shares of $[r_\ell + \rho + \varepsilon_\ell]_{n-1}$, where ε_ℓ is error added by the adversary.
 - ii. Then if P_1 is corrupted, $\mathcal{S}$ receives $\{r_\ell + \rho + \varepsilon_\ell^j\}_{j \in \text{Hon}}$ from the adversary, where ε_ℓ^j are errors added by the adversary; if these values are not all the same, $\mathcal{S}$ sets $\text{bad-distr} \leftarrow 1$. If P_1 is honest, $\mathcal{S}$ sends $r_\ell + \rho + \varepsilon_\ell$ to the corrupted parties.
 - iii. In either case, $\mathcal{S}$ can compute the sharings $[(r_{\ell,i} + \varepsilon_{\ell,i}^j) \cdot \mathbf{1}]_{n-k} = (r_{\ell,i} + \rho_i + \varepsilon_{\ell,i}^j) - [\rho_i \cdot \mathbf{1}]_{n-k}$ for all $i \in \{1, \dots, k\}$.
 - (c) Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{coin}}$ and samples random values $\chi_1, \dots, \chi_c$, computes $\tau^j = \sum_{\ell=1}^c \chi_\ell \cdot \tau_\ell^j$ on behalf of each honest party P_j , where τ_ℓ^j are the values sent by P_1 above, and finally sends these values to the corrupted parties. If $\text{bad-distr} = 1$ or $\mathcal{S}$ receives from the corrupted parties any value different from the τ it computed, then $\mathcal{S}$ tells the honest parties to abort.

(d) Finally, $\mathcal{S}$ sends to $\mathcal{F}_{\text{prep-sup-mal}}$ the corrupted parties' shares.

4. Preparing Packed, Authenticated Beaver Triples:

- (a) $\mathcal{S}$ first emulates $\mathcal{F}_{\text{triple}}$ by receiving from the adversary the corrupted parties' shares of each $[\mathbf{a}_l]_{n-k}$, $[\mathbf{b}_l]_{n-k}$, and $[\mathbf{c}_l + \delta_l]_{n-k}$, and each δ_l , and then sampling the rest of the sharings randomly, consistent with the corrupted parties' shares and each δ_l .
- (b) Then, $\mathcal{S}$ emulates $\mathcal{F}_{\text{deg-reduce}}$ by receiving from the adversary the corrupted parties' shares of each $[\mathbf{c}_l * + \delta_l]_{n-1}$ and the new sharings $[\mathbf{c}_l + \delta_l]_{n-k}$, and then randomly sampling the latter consistent with the corrupted parties' share and the underlying values.
- (c) To simulate π_{auth} , $\mathcal{S}$ first sets $\text{bad-shr} \leftarrow 0$, then:
 - i. $\mathcal{S}$ first emulates $\mathcal{F}_{\text{auth-rand}}$ by receiving from the adversary the corrupted parties' shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ and then either **abort** or the corrupted parties' shares of each $[\mathbf{r}_l]_{n-k}$. In the former case, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{prep-sup-mal}}$; otherwise, it samples random sharings $[\mathbf{r}_l]_{n-k} = ([\mathbf{r}_l]_{n-k}, [(\Delta \cdot \mathbf{1} + \varepsilon) * \mathbf{r}_l]_{n-k})$ consistent with the corrupted parties' shares, where ε is the difference between $\Delta \cdot \mathbf{1}$ and that which is defined by the honest parties' shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ and those input by the adversary above. Note, if the adversary corrupts more parties later, it can just send their shares to the adversary.
 - ii. Then for $l \in [n - t]$:
 - A. If P_l is corrupted, $\mathcal{S}$ sends to the adversary the honest parties' shares of $[\mathbf{u}_l + \mathbf{r}_l]_{n-k}$. Otherwise, it receives from the adversary the corrupted parties' shares of $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{n-k}$, where $\boldsymbol{\eta}_l$ is error added by the adversary.
 - B. Then, if P_l is corrupted, $\mathcal{S}$ receives from the adversary $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1}$, where $\boldsymbol{\eta}_l$ is error added by the adversary. If these shares do not lie on a polynomial of degree $k - 1$, $\mathcal{S}$ sets $\text{bad-shr} \leftarrow 1$. If P_l is honest, $\mathcal{S}$ computes sharing $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1}$ and sends the corrupted parties their shares.
 - C. In either case, $\mathcal{S}$ can compute the sharing $[\Delta \cdot \mathbf{1} * (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l]_{n-1} \leftarrow [\Delta \cdot \mathbf{1}] * [\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1} - [(\Delta \cdot \mathbf{1} + \varepsilon) * \mathbf{r}_l]_{n-k}$
 - iii. Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{deg-reduce}}$ by receiving from the adversary the corrupted parties' shares of each $[\Delta \cdot \mathbf{1} * (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l]_{n-1}$ and then of each $[\Delta \cdot \mathbf{1} * (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l]_{n-k}$, and then randomly sampling the rest of the latter sharings, consistent with the corrupted parties' shares and the values $\Delta \cdot \mathbf{1} * (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l$.
 - iv. Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{verify-deg}}$ by sending the adversary the corrupted parties' shares of each $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1}$ and then sending $\mathcal{F}_{\text{prep-sup-mal}}$ **abort** if $\text{bad-shr} = 1$ or the adversary sends $\mathcal{S}$ **abort**.
 - v. If $\mathcal{S}$ does not abort, it outputs the sharings $([\mathbf{u}_l]_{n-k}, [\Delta \cdot \mathbf{1} * (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l]_{n-k})$.
- (d) To simulate $\pi_{\text{sacrifice}}$:
 - i. $\mathcal{S}$ first emulates $\mathcal{F}_{\text{coin}}$ by sampling random ρ and sending it to the adversary.

- ii. If P_1 is corrupted, $\mathcal{S}$ sends to the adversary the honest parties' shares of $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{n-k}$ and $[\mathbf{b}_2 - \mathbf{b}_1]_{n-k}$. Otherwise, it receives from the adversary the corrupted parties' shares of $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{n-k}$ and $[\mathbf{b}_2 - \mathbf{b}_1]_{n-k}$.
- iii. Then, if P_1 is corrupted, $\mathcal{S}$ receives from the adversary $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{k-1}$, $[\mathbf{b}_2 - \mathbf{b}_1]_{k-1}$, and $[(\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1)]_{k-1}$. If the shares do not lie on a polynomial of degree $k-1$ for any of the sharings, $\mathcal{S}$ sets $\text{bad-shr} \leftarrow 1$. If P_1 is honest, $\mathcal{S}$ computes the sharings themselves, and sends the corrupted parties their shares.
- iv. In either case, $\mathcal{S}$ computes the sharing

$$\begin{aligned} [\Delta \cdot \boldsymbol{\theta}]_{n-1} &\leftarrow [\Delta \cdot \mathbf{1}]_{n-k} * [(\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1)]_{k-1} \\ &+ [\Delta \cdot \mathbf{b}_1]_{n-k} * [\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{k-1} + \rho \cdot [\Delta \cdot \mathbf{a}_1]_{n-k} * [\mathbf{b}_2 - \mathbf{b}_1]_{k-1} + \\ &\rho \cdot [\Delta \cdot \mathbf{c}_1]_{n-k} - [\Delta \cdot \mathbf{c}_2]_{n-k}. \end{aligned}$$

- (e) To simulate $\pi_{\text{check-zero}}$:
 - i. $\mathcal{S}$ first emulates $\mathcal{F}_{\text{coin}}$ by sampling random $\chi_1, \dots, \chi_M$ and sending them to the adversary.
 - ii. Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary the corrupted parties' shares of $[\mathbf{0}]_{n-1}$, and then sampling the rest of the sharing randomly, based on the corrupted parties' shares and $\mathbf{0}$.
 - iii. Then $\mathcal{S}$ computes $[\boldsymbol{\theta}]_{n-1} \leftarrow [\mathbf{0}]_{n-1} + \sum_{l=1}^M \chi_l \cdot [\boldsymbol{\theta}_l]_{n-1}$.
 - iv. Next $\mathcal{S}$ emulates $\mathcal{F}_{\text{commit}}$ by receiving from the adversary the corrupted parties' shares of $[\boldsymbol{\theta}]_{n-1}$.
 - v. Finally, $\mathcal{S}$ sends the adversary the honest parties' shares of $[\boldsymbol{\theta}]_{n-1}$ and receives the corrupted parties' shares from the adversary. If the corrupted parties' shares are different from the committed values or the computed shared value $\boldsymbol{\theta} \neq \mathbf{0}$, then $\mathcal{S}$ sends to $\mathcal{F}_{\text{prep-sup-mal}}$, **abort**.
- (f) Finally, $\mathcal{S}$ emulates $\mathcal{F}_{\text{verify-deg}}$ by sending to the adversary the corrupted parties' shares of each $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{k-1}$, $[\mathbf{b}_2 - \mathbf{b}_1]_{k-1}$, and $[(\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1)]_{k-1}$, from $\pi_{\text{sacrifice}}$, then sending **abort** to $\mathcal{F}_{\text{prep-sup-mal}}$ if $\text{bad-shr} = 1$ or the adversary sends **abort**.

$\mathcal{S}$ then sends to $\mathcal{F}_{\text{prep-sup-mal}}$ the corrupted parties' shares of each $[\mathbf{a}_l]_{n-k}$, $[\mathbf{b}_l]_{n-k}$, $[\mathbf{c}_l]_{n-k}$

5. **Preparing Random Masked Sharings for Multiplicaiton Gates:** $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary the corrupted parties' shares of $[\mathbf{0}]_{n-1}$ and then sampling the rest of the sharings randomly, based on the corrupted parties' shares and $\mathbf{0}$. $\mathcal{S}$ then sends to $\mathcal{F}_{\text{prep-sup-mal}}$ the corrupted parties' shares.
6. **Preparing Random Masked Sharings for Iput and Output Gates:** Same as above.

Now we argue that the real and ideal worlds are indistinguishable. For initialization, it is clear that $\mathcal{S}$ simulates the PRG seeds randomly, as in the real world. For settling authentication keys, It is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{nVole}}$ and $\mathcal{F}_{\text{rand}}$ as in the real world. Also, the shares of $\{[r|_i]_t\}_{i=1}^k$ and $[\rho \cdot \mathbf{1}]_{n-k}$ that

the honest parties output in the real world are distributed randomly given the corrupted parties' shares, by definition of $\mathcal{F}_{\text{rand}}$. This is exactly the same as how $\mathcal{F}_{\text{prep-sup-mal}}$ samples the honest parties' shares. For the simulation of $\pi_{\text{MAC-key}}$, it is clear that $\mathcal{S}$ simulates the opening and redistribution of $[(\Delta + r + \varepsilon) \cdot \mathbf{1}]_{2k-2}$ and $[(\Delta + \rho + \eta) \cdot \mathbf{1}]_{2k-2}$. Now we also note that the corrupted parties can locally change their shares to change the secrets from $\Delta + \varepsilon$ and $(\Delta + \eta) \cdot \mathbf{1}$ to Δ and $\Delta \cdot \mathbf{1}$. We may equivalently think that the shares of corrupted parties are changed so that the secrets are Δ and $\Delta \cdot \mathbf{1}$ (and they can then change their shares to any arbitrary values). Thus, $\mathcal{S}$ adjusts the shares of corrupted parties as follows: $\mathcal{S}$ generates degree- t Shamir sharings $[\varepsilon|_i]_t$ for all $i \in \{1, \dots, k\}$ and degree- $(n-k)$ packed Shamir sharing $[\eta \cdot \mathbf{1}]_{n-k}$ such that the honest parties' shares are 0's. Then $\mathcal{S}$ computes the sharings $[\Delta]_t = [\Delta + \varepsilon|_i]_t - [\varepsilon|_i]_t$ for all $i \in \{1, \dots, k\}$ and $[\Delta \cdot \mathbf{1}]_{n-k} = [(\Delta + \eta) \cdot \mathbf{1}]_{n-k} - [\eta \cdot \mathbf{1}]_{n-k}$ and uses this sharing in the rest of the simulation. Now we just need to argue that the shares that the honest parties output in the real world are identical to that of the ideal world. In both worlds, it is clear that all of the sharings are randomly distributed given the corrupted parties' shares.

For the simulation of $\pi_{\text{rand-red-auth}}$, it is clear that $\mathcal{S}$ simulates the opening of $[\mathbf{r}_\ell + \boldsymbol{\rho}]_{n-1}$ and the redistribution of its secrets perfectly. It is also clear that $\mathcal{S}$ simulates the exchange of the $\boldsymbol{\tau}^j$'s correctly. Now suppose that $\text{bad-distr} = 1$, meaning that $\varepsilon_\ell^{j_1} \neq \varepsilon_\ell^{j_2}$ for some ℓ for two honest parties P_{j_1} and P_{j_2} , so that the parties abort in the ideal world. Assume w.l.o.g., that $\ell = 1$. In the real world, the only way these parties do not abort is if $\chi_1 \cdot \boldsymbol{\tau}_1^{j_1} + \sum_{\ell=2}^c \chi_\ell \boldsymbol{\tau}_\ell^{j_1} = \chi_1 \cdot \boldsymbol{\tau}_1^{j_2} + \sum_{\ell=2}^c \chi_\ell \boldsymbol{\tau}_\ell^{j_2}$, which can only happen if $\chi_1 = (\sum_{\ell=2}^c \chi_\ell \boldsymbol{\tau}_\ell^{j_2} - \sum_{\ell=2}^c \chi_\ell \boldsymbol{\tau}_\ell^{j_1}) / (\boldsymbol{\tau}_1^{j_1} - \boldsymbol{\tau}_1^{j_2})$, which can only happen with probability $1/|\mathbb{F}|$. So the simulation is identical to the real world, with all-but-negligible probability in this case. Now we also note that if $\mathcal{S}$ does not abort, the corrupted parties can locally change their shares to change the secrets from $(r_{\ell,i} + \varepsilon_{\ell,i}) \cdot \mathbf{1}$ to $r_{\ell,i} \cdot \mathbf{1}$. We may equivalently think that the shares of corrupted parties are changed so that the secrets are $r_{\ell,i} \cdot \mathbf{1}$ (and they can then change their shares to any arbitrary values). Thus $\mathcal{S}$ adjusts the shares of corrupted parties as follows: $\mathcal{S}$ generates degree- $(n-k)$ Shamir sharings $[\varepsilon_{\ell,i} \cdot \mathbf{1}]_{n-k}$ such that the honest parties' shares are 0's. Then $\mathcal{S}$ computes the sharings $[r_{\ell,i} \cdot \mathbf{1}]_{n-k} = [(r_{\ell,i} + \varepsilon_{\ell,i}) \cdot \mathbf{1}]_{n-k} - [\varepsilon_{\ell,i} \cdot \mathbf{1}]_{n-k}$ and uses this sharing in the rest of the simulation. Now we just need to argue that the shares that the honest parties output in the real world are identical to that of the ideal world. In both worlds, it is clear that the sharings are randomly distributed given the corrupted parties' shares.

For the packed, authenticated beaver triples: It is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{triple}}$ and $\mathcal{F}_{\text{deg-reduce}}$ as in the real world. For $\mathcal{S}$'s simulation of π_{auth} , it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{auth-rand}}$ exactly as in the real world. It is also clear that $\mathcal{S}$ simulates the opening and redistribution of $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1}$ as in the real world. Finally, it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{deg-reduce}}$ and $\mathcal{F}_{\text{verify-deg}}$ as in the real world. Now, we also note that the corrupted parties can locally change their shares to change the secret from $\Delta \cdot \mathbf{1} * (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l$ to $\Delta \cdot \mathbf{1} * \mathbf{u}_l$. We may equivalently think that the shares of corrupted parties are changed

so that the secret is $\Delta \cdot \mathbf{1} * \mathbf{u}_l$ (and they can then change their shares to any arbitrary values). Thus, $\mathcal{S}$ adjusts the shares of corrupted parties as follows: $\mathcal{S}$ generates a degree- $(n - k)$ packed Shamir sharing $[\Delta \cdot \mathbf{1} * \boldsymbol{\eta}_l - \boldsymbol{\varepsilon} * \mathbf{r}_l]_{n-k}$ such that the shares of honest parties are 0's. Then $\mathcal{S}$ computes the sharing $[\Delta \cdot \mathbf{1} * \mathbf{u}_l]_{n-k} \leftarrow [\Delta \cdot \mathbf{1} * (\mathbf{u}_l + \boldsymbol{\eta}_l) - \boldsymbol{\varepsilon} * \mathbf{r}_l]_{n-k} - [\Delta \cdot \mathbf{1} * \boldsymbol{\eta}_l - \boldsymbol{\varepsilon} * \mathbf{r}_l]_{n-k}$ and uses this sharing in the rest of the simulation. In the simulation of $\pi_{\text{sacrifice}}$, it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{coin}}$ as in the real world. It is also clear that the opening and redistribution of $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{k-1}$, $[\mathbf{b}_2 - \mathbf{b}_1]_{k-1}$, and $[(\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1)]_{k-1}$ is as in the real world. In the simulation of $\pi_{\text{check-zero}}$ it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{coin}}$, $\mathcal{F}_{\text{rand}}$, and $\mathcal{F}_{\text{commit}}$ as in the real world. It is also clear that it computes $[\boldsymbol{\theta}]_{n-1}$ as in the real world. So, the honest parties abort in the ideal world if and only if they do in the real world. Finally, it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{verify-deg}}$ and $\mathcal{F}_{\text{rand}}$ as in the real world. Now, from Lemma 14, we know that if any $\Delta \cdot \boldsymbol{\theta}_{l^*}$ is nonzero, and random and unknown to the adversary, then the honest parties will abort with all-but-negligible probability. In particular, this means that each $[\mathbf{c}_1]_{n-k}$ cannot have any error, for otherwise, the corresponding $\Delta \cdot \boldsymbol{\theta}_{l^*}$ would contain $\rho \cdot \Delta \cdot \boldsymbol{\delta}_1$ and thus be random and unknown to the adversary. Observe also that in both worlds, if the adversary does not introduce any errors, then $\Delta \cdot \boldsymbol{\theta} = \mathbf{0}$. Now we just need to argue that the shares that the honest parties output in the real world are identical to that of the ideal world. In both worlds, it is clear that $[\mathbf{a}_1]_{n-k}$, $[\mathbf{b}_1]_{n-k}$ are randomly distributed given the corrupted parties' shares. Each $[\mathbf{c}_l]_{n-k}$ is distributed randomly given the corrupted parties' shares and $\mathbf{a}_l * \mathbf{b}_l$. The same can be said of $[\Delta \cdot \mathbf{a}_l]_{n-k}$, $[\Delta \cdot \mathbf{b}_l]_{n-k}$, $[\Delta \cdot \mathbf{c}_l]_{n-k}$, given the corrupted parties' shares and $\Delta \cdot \mathbf{a}_l$, $\Delta \cdot \mathbf{b}_l$, and $\Delta \cdot \mathbf{c}_l$, respectively.

For the random masked sharings for multiplication, input, and output gates: it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ as in the real world. It is also clear that in both worlds, sharings $[\mathbf{0}]_{n-1}$ are distributed randomly given the corrupted parties' shares and $\mathbf{0}$. Also, the honest parties' shares of $\langle r \rangle$, $\langle \Delta \cdot r \rangle$ are distributed randomly.

Thus, we have argued that the real and ideal worlds are indistinguishable with all-but-negligible probability. $\square$

E More Concrete Communication Costs and Experimental Results

Here we provide the full versions of Figures 1, 2, and 3 from Section 8. For each setting, we test every value of $\zeta \in [3, 8]$, with the corresponding γ from Section B.5. We generally see that once γ reaches 0.667 for a given setting, the communication and runtime increases with ζ . This makes sense, since the communication is proportional to ζ . We also see that for lower γ , communication and computation is worse. This also makes sense, since the number of triples we can extract from Π_{triple} is proportional to $\gamma - 1/2$, so if γ is lower, we have to run Π_{triple} more times to account for all $|C|$ multiplication gates.

We see in Figures 7 and 9, that we obtain communication gains with respect to SuperPack for all such ζ , albeit starting at higher n for lower ε . Notably, for

	SuperPack	TwoLevel, $\zeta = 3$	TwoLevel, $\zeta = 4$	TwoLevel, $\zeta = 5$	TwoLevel, $\zeta = 6$	TwoLevel, $\zeta = 7$	TwoLevel, $\zeta = 8$
$\epsilon = 1/2$							
$n = 256$	284.0	174.8 (0.660)	191.4 (0.667)	215.4 (0.667)	239.3 (0.667)	263.3 (0.667)	287.2 (0.667)
$n = 512$	540.0	167.6 (0.667)	191.5 (0.667)	215.5 (0.667)	239.4 (0.667)	263.4 (0.667)	287.3 (0.667)
$n = 1024$	1052	167.6 (0.667)	191.6 (0.667)	215.5 (0.667)	239.5 (0.667)	263.4 (0.667)	287.4 (0.667)
$n = 2048$	2076	167.6 (0.667)	191.6 (0.667)	215.5 (0.667)	239.5 (0.667)	263.4 (0.667)	287.4 (0.667)
$n = 4096$	4124	167.7 (0.667)	191.6 (0.667)	215.6 (0.667)	239.5 (0.667)	263.5 (0.667)	287.4 (0.667)
$n = 8192$	8220	167.7 (0.667)	191.6 (0.667)	215.6 (0.667)	239.5 (0.667)	263.5 (0.667)	287.4 (0.667)
$\epsilon = 1/4$							
$n = 256$	312.0	-	1121.7 (0.557)	570.9 (0.626)	478.7 (0.667)	526.6 (0.667)	574.5 (0.667)
$n = 512$	568.0	5596.9 (0.510)	621.1 (0.603)	431.0 (0.667)	478.9 (0.667)	526.8 (0.667)	574.7 (0.667)
$n = 1024$	1080	1271.1 (0.546)	474.0 (0.635)	431.0 (0.667)	478.9 (0.667)	526.9 (0.667)	574.8 (0.667)
$n = 2048$	2104	788.6 (0.571)	407.6 (0.657)	431.1 (0.667)	479.0 (0.667)	526.9 (0.667)	574.8 (0.667)
$n = 4096$	4152	629.2 (0.589)	383.2 (0.667)	431.1 (0.667)	479.0 (0.667)	526.9 (0.667)	574.8 (0.667)
$n = 8192$	8248	554.4 (0.601)	383.2 (0.667)	431.1 (0.667)	479.0 (0.667)	526.9 (0.667)	574.8 (0.667)
$\epsilon = 1/8$							
$n = 256$	368	-	-	-	3402 (0.547)	1832 (0.596)	1411 (0.636)
$n = 512$	624	-	-	3590 (0.540)	1649 (0.597)	1222 (0.644)	1151 (0.667)
$n = 1024$	1136	-	-	1870 (0.577)	1212 (0.632)	1054 (0.667)	1150 (0.667)
$n = 2048$	2160	-	4266 (0.530)	1412 (0.602)	969.6 (0.665)	1054 (0.667)	1150 (0.667)
$n = 4096$	4208	-	2560 (0.550)	1210 (0.619)	958.0 (0.667)	1054 (0.667)	1150 (0.667)
$n = 8192$	8304	-	2032 (0.563)	1099 (0.631)	958.1 (0.667)	1054 (0.667)	1150 (0.667)

Fig. 7. Number of communicated field elements per individual multiplication triple generated in SuperPack’s and our TwoLevel packed beaver triple protocol for different values of ζ . We report for n that are powers of 2 starting with 256 and $\epsilon \in \{1/2, 1/4, 1/8\}$. For our protocol, in parentheses we provide the used γ value from Section B.5. We denote no such choice of γ existing by ‘-’. **Bold** values are those in which our protocol has lower communication than SuperPack’s.

Figure 7, our communication does not increase with n , unlike SuperPacks. For Figure 9, our communication increases with n at a slower rate than SuperPack. Figure 2 shows that for $\epsilon = 1/2$, $|C| = 10K$, and network bandwidth 10 mbps, our protocol is faster than SuperPack’s for $\zeta \in \{3, 4, 5\}$. For $\zeta \geq 6$ in this setting, SuperPack’s protocol is faster. We also see that when $|C| = 100K$, we can run our protocol for bandwidth 10 mbps and 50 mbps, $\epsilon \in \{1/4, 1/2\}$, and several settings of ζ . SuperPack’s protocol never finishes for any of these settings.

F Instantiating the Preprocessing of [GPS22]

We now instantiate the preprocessing of [GPS22, Functionality 15] with $\tilde{O}(|C|)$ communication and adaptive security, in the $t < (1 - \epsilon)n$ setting. Assuming this preprocessing, [GPS22] obtain an MPC protocol with $O(|C|)$ online communication in this setting. In this section, we present the preprocessing functionality $\mathcal{F}_{\text{prep-st-mal}}$ and our protocol with $\tilde{O}(|C|)$ communication.

F.1 The Preprocessing Functionality

We now present the preprocessing Functionality verbatim from [GPS22]. The functionality begins by generating a random $\Delta \leftarrow_{\$} \mathbb{F}$. This Δ serves as the MAC key in the online phase, and ensures that shared values are opened correctly.

	SuperPack	TwoLevel, $\zeta = 3$	TwoLevel, $\zeta = 4$	TwoLevel, $\zeta = 5$	TwoLevel, $\zeta = 6$	TwoLevel, $\zeta = 7$	TwoLevel, $\zeta = 8$
$\epsilon = 1/2$; 10 mbps							
$ C = 10K$	56.89	52.02 (0.660)	52.63 (0.667)	55.79 (0.667)	58.79 (0.667)	61.59 (0.667)	64.7 (0.667)
$ C = 100K$	-	288.9 (0.660)	314.9 (0.667)	380.8 (0.667)	448.3 (0.667)	513.5 (0.667)	518.0 (0.667)
$\epsilon = 1/2$; 50 mbps							
$ C = 10K$	23.35	36.01 (0.660)	34.82 (0.667)	35.80 (0.667)	36.59 (0.667)	36.66 (0.667)	37.52 (0.667)
$ C = 100K$	-	233.0 (0.660)	222.3 (0.667)	228.2 (0.667)	231.7 (0.667)	236.6 (0.667)	241.8 (0.667)
$\epsilon = 1/4$; 10mbps							
$ C = 10K$	73.71	No such γ	-	-	152.8 (0.667)	175.2 (0.667)	199.7 (0.667)
$ C = 100K$	-	No such γ	-	-	1524 (0.667)	1675 (0.667)	1816 (0.667)
$\epsilon = 1/4$; 50mbps							
$ C = 10K$	22.72	No such γ	-	-	61.32 (0.667)	64.10 (0.667)	66.62 (0.667)
$ C = 100K$	-	No such γ	-	-	544.9 (0.667)	572.6 (0.667)	599.4 (0.667)

Fig. 8. Running time in seconds of SuperPack’s and our TwoLevel packed beaver triple protocol. We report for $n = 256$ and $\epsilon \in \{1/2, 1/4\}$. We specify the chosen ζ and γ in parentheses taken from Appendix B.5. We denote by ‘-’ settings in which we could not obtain complete experiment runs. For $\zeta = 3, \epsilon = 1/4$, there is no existing choice of γ in Appendix B.5. **Bold** values indicate settings where our protocol has lower computation compared to SuperPack’s protocol.

Indeed, in the online phase, all circuit values will be shared using degree- $(n - k)$ *authenticated* packed Shamir sharings $\llbracket \mathbf{v} \rrbracket_{n-k} := ([\mathbf{v}]_{n-k}, [\Delta \cdot \mathbf{v}]_{n-k})$, where the latter sharing is referred to as the ‘MAC’ on the former sharing. Intuitively, if $\mathbf{v}$ is opened then it will be checked by opening $\Delta \cdot \mathbf{v}$ —since Δ is random, with all-but-negligible probability, the adversary will be unable to force an opening to some $\mathbf{v}' \neq \mathbf{v}$.¹¹ $\mathcal{F}_{\text{prep-st-mal}}$ then has two procedures:

1. **rand-sharing**($\mathbf{r}, d$): Samples a random degree- d packed Shamir sharing $[\mathbf{r}]_d$ that is consistent with shares input by the adversary for corrupt parties, then sends the honest parties their shares.
2. **auth-sharing**($\mathbf{r}$): Samples degree- $(n - k)$ authenticated packed Shamir sharings $\llbracket \mathbf{r} \rrbracket_{n-k}$ consistent with input corrupt parties’ shares as above, then sends the honest parties their shares.

$\mathcal{F}_{\text{prep-st-mal}}$ uses the procedures as follows. First, it uses **rand-sharing** to prepare $[\Delta \cdot \mathbf{1}]_{n-k}$. Then, for every group of k input and output gates, it uses **auth-sharing** to prepare $\llbracket \mathbf{r} \rrbracket_{n-k}$ for random $\mathbf{r} \in \mathbb{F}^k$, then **rand-sharing** to prepare $[\mathbf{r}]_{n-k}$ and $[\mathbf{r} * \mathbf{r}]_{n-k}$ for random $\mathbf{r} \in \mathbb{F}^k$, and finally **rand-sharing** to prepare random $[\mathbf{o}]_{n-1}$, where $\mathbf{o} = \mathbf{0}$. The random sharings are used to securely share and reconstruct inputs and outputs, respectively, in the presence of a malicious adversary.

For each group of k multiplication gates, $\mathcal{F}_{\text{prep-st-mal}}$ prepares packed beaver triples as follows. It samples two random $\mathbf{a}, \mathbf{b} \in \mathbb{F}^k$, computes $\mathbf{c} \leftarrow \mathbf{a} * \mathbf{b}$, and finally inputs them to **auth-sharing** to prepare $(\llbracket \mathbf{a} \rrbracket_{n-k}, \llbracket \mathbf{b} \rrbracket_{n-k}, \llbracket \mathbf{c} \rrbracket_{n-k})$. It also prepares random sharings $[\mathbf{o}^{(1)}]_{n-1}, [\mathbf{o}^{(2)}]_{n-1}$, where $\mathbf{o}^{(1)} = \mathbf{o}^{(2)} = \mathbf{0}$, using **rand-sharing**.

Finally, for a verification phase at the end of the online phase which is used to catch a cheating adversary, $\mathcal{F}_{\text{prep-st-mal}}$ prepares additional random sharings $[\mathbf{o}^{(1)}]_{n-1}, [\mathbf{o}^{(2)}]_{n-1}$, where $\mathbf{o}^{(1)} = \mathbf{o}^{(2)} = \mathbf{0}$, using **rand-sharing**. It also samples random $r \in \mathbb{F}$, computes $\Delta \cdot r$, and generates a pair of additive sharings $(\langle r \rangle, \langle \Delta \cdot r \rangle)$,

¹¹ This is done once for many such openings at a time using random linear combinations.

	SuperPack	TwoLevel, $\zeta = 3$	TwoLevel, $\zeta = 4$	TwoLevel, $\zeta = 5$	TwoLevel, $\zeta = 6$	TwoLevel, $\zeta = 7$	TwoLevel, $\zeta = 8$
$\epsilon = 1/2$							
$n = 256$	1606	2185 (0.660)	2318 (0.667)	2506 (0.667)	2701 (0.667)	2893 (0.667)	3084 (0.667)
$n = 512$	3142	2639 (0.667)	2831 (0.667)	3023 (0.667)	3214 (0.667)	3406 (0.667)	3597 (0.667)
$n = 1024$	6214	3664 (0.667)	3855 (0.667)	4047 (0.667)	4239 (0.667)	4430 (0.667)	4622 (0.667)
$n = 2048$	12358	5712 (0.667)	5904 (0.667)	6095 (0.667)	6287 (0.667)	6479 (0.667)	6670 (0.667)
$n = 4096$	24650	9808 (0.667)	10000 (0.667)	10190 (0.667)	10390 (0.667)	10570 (0.667)	10770 (0.667)
$n = 8192$	49220	18000 (0.667)	18190 (0.667)	18380 (0.667)	18580 (0.667)	18770 (0.667)	18960 (0.667)
$\epsilon = 1/4$							
$n = 256$	1676	-	10050 (0.557)	5644 (0.626)	4905 (0.667)	5289 (0.667)	5672 (0.667)
$n = 512$	3212	46360 (0.510)	6557 (0.603)	5036 (0.667)	5419 (0.667)	5803 (0.667)	6186 (0.667)
$n = 1024$	6284	12350 (0.546)	6404 (0.635)	6061 (0.667)	6444 (0.667)	6827 (0.667)	7211 (0.667)
$n = 2048$	12428	10970 (0.571)	7922 (0.657)	8110 (0.667)	8493 (0.667)	8876 (0.667)	9259 (0.667)
$n = 4096$	24716	13790 (0.589)	11820 (0.667)	12210 (0.667)	12590 (0.667)	12970 (0.667)	13360 (0.667)
$n = 8192$	49290	21380 (0.601)	20010 (0.667)	20400 (0.667)	20780 (0.667)	21160 (0.667)	21550 (0.667)
$\epsilon = 1/8$							
$n = 256$	1826	-	-	-	28870 (0.547)	16310 (0.596)	12940 (0.636)
$n = 512$	3352	-	-	30960 (0.540)	15360 (0.597)	11940 (0.644)	11360 (0.667)
$n = 1024$	6424	-	-	18150 (0.577)	12890 (0.632)	11620 (0.667)	12390 (0.667)
$n = 2048$	12568	-	39370 (0.530)	16530 (0.602)	12300 (0.665)	13670 (0.667)	14440 (0.667)
$n = 4096$	24856	-	29820 (0.550)	19020 (0.619)	17001 (0.667)	17770 (0.667)	18530 (0.667)
$n = 8192$	49430	-	33780 (0.563)	26320 (0.631)	25190 (0.667)	29960 (0.667)	26730 (0.667)

Fig. 9. Number of communicated field elements per individual multiplication gate in SuperPack’s and our TwoLevel input-independent preprocessing protocol for different values of ζ . We report for n that are powers of 2 starting with 256 and $\epsilon \in \{1/2, 1/4, 1/8\}$. For our protocol, in parentheses we provide the used γ value from Section B.5. We denote no such choice of γ existing by ‘-’. **Bold** values are those in which our protocol has lower communication than SuperPack’s.

where the shares for corrupted parties are input by the adversary, then distributes to the honest parties their shares of these sharings.

Functionality 9: $\mathcal{F}_{\text{prep-st-mal}}$

$\mathcal{F}_{\text{prep-st-mal}}$ receives the set of corrupted parties, denoted by Corr . $\mathcal{F}_{\text{prep-st-mal}}$ samples a random field element $\Delta \in \mathbb{F}$. Let $d \in \{n-k, n-1\}$. We define the following two procedures.

- **rand-sharing**($\mathbf{r}, d$): $\mathcal{F}_{\text{prep-st-mal}}$ receives from the adversary a set of shares $\{r_j\}_{j \in \text{Corr}}$. Then $\mathcal{F}_{\text{prep-st-mal}}$ samples a random degree- d packed Shamir sharing $[\mathbf{r}]_d$ such that for all $P_j \in \text{Corr}$, the j -th share of $[\mathbf{r}]_d$ is r_j . Finally, $\mathcal{F}_{\text{prep-st-mal}}$ distributes the shares of $[\mathbf{r}]_d$ to honest parties.
- **auth-sharing**($\mathbf{r}$): $\mathcal{F}_{\text{prep-st-mal}}$ receives from the adversary a set of shares $\{(r_j, u_j)\}_{j \in \text{Corr}}$. Then $\mathcal{F}_{\text{prep-st-mal}}$ computes two degree- $(n-k)$ packed Shamir sharings $([\mathbf{r}]_{n-k}, [\Delta \cdot \mathbf{r}]_{n-k})$ such that for all $P_j \in \text{Corr}$, the j -th shares of $([\mathbf{r}]_{n-k}, [\Delta \cdot \mathbf{r}]_{n-k})$ are r_j, u_j , respectively. Finally $\mathcal{F}_{\text{prep-st-mal}}$ distributes the shares of $([\mathbf{r}]_{n-k} = ([\mathbf{r}]_{n-k}, [\Delta \cdot \mathbf{r}]_{n-k})$ to honest parties.

The ideal functionality $\mathcal{F}_{\text{prep-st-mal}}$ runs the following steps.

1. $\mathcal{F}_{\text{prep-st-mal}}$ invokes **rand-sharing**($\Delta \cdot \mathbf{1}, n-k$) to prepare $[\Delta \cdot \mathbf{1}]_{n-k}$.
2. For every group of k input gates and output gates:
 - (a) $\mathcal{F}_{\text{prep-st-mal}}$ samples a random vector $\mathbf{r} \in \mathbb{F}^k$ and invokes **auth-sharing**($\mathbf{r}$) to prepare $[\mathbf{r}]_{n-k}$.

- (b) $\mathcal{F}_{\text{prep-st-mal}}$ samples a random vector $\mathbf{I} \in \mathbb{F}^k$ and invokes $\text{rand-sharing}(\mathbf{I}, n - k)$ and $\text{rand-sharing}(\mathbf{I} * \mathbf{r}, n - k)$ to prepare $([\mathbf{I}]_{n-k}, [\mathbf{I} * \mathbf{r}]_{n-k})$.
- (c) For every group of k output gates, $\mathcal{F}_{\text{prep-st-mal}}$ invokes $\text{rand-sharing}(\mathbf{0}, n - 1)$ to prepare $[\mathbf{o}]_{n-1}$, where $\mathbf{o} \leftarrow \mathbf{0}$.
- 3. For every group of k multiplication gates:
 - (a) $\mathcal{F}_{\text{prep-st-mal}}$ samples two random vectors $\mathbf{a}, \mathbf{b} \in \mathbb{F}^k$ and computes $\mathbf{c} = \mathbf{a} * \mathbf{b}$. Then, $\mathcal{F}_{\text{prep-st-mal}}$ invokes $\text{auth-sharing}(\mathbf{a})$, $\text{auth-sharing}(\mathbf{b})$, and $\text{auth-sharing}(\mathbf{c})$ to prepare $([\mathbf{a}]_{n-k}, [\mathbf{b}]_{n-k}, [\mathbf{c}]_{n-k})$.
 - (b) $\mathcal{F}_{\text{prep-st-mal}}$ invokes two times of $\text{rand-sharing}(\mathbf{0}, n - 1)$ to prepare $[\mathbf{o}^{(1)}]_{n-1}, [\mathbf{o}^{(2)}]_{n-1}$, where $\mathbf{o}^{(1)}, \mathbf{o}^{(2)} \leftarrow \mathbf{0}$.
- 4. All parties prepare the following random sharings for the verification of the computation:
 - (a) All parties invoke two times of $\text{rand-sharing}(\mathbf{0}, n - 1)$ to prepare $[\mathbf{o}^{(1)}]_{n-1}, [\mathbf{o}^{(2)}]_{n-1}$, where $\mathbf{o}^{(1)}, \mathbf{o}^{(2)} \leftarrow \mathbf{0}$.
 - (b) $\mathcal{F}_{\text{prep-st-mal}}$ receives from the adversary a set of shares $\{(r_j, r'_j)\}_{j \in \text{Corr}}$. Then $\mathcal{F}_{\text{prep-st-mal}}$ samples a random field element r and computes $\Delta \cdot r$. $\mathcal{F}_{\text{prep-st-mal}}$ randomly generates a pair of additive sharings $(\langle r \rangle, \langle \Delta \cdot r \rangle)$ such that for all $P_j \in \text{Corr}$, the j -th shares of $(\langle r \rangle, \langle \Delta \cdot r \rangle)$ are r_j, r'_j , respectively. Finally, $\mathcal{F}_{\text{prep-st-mal}}$ distributes the shares of $(\langle r \rangle, \langle \Delta \cdot r \rangle)$ to honest parties.

F.2 The Preprocessing Protocol

Finally, we describe the rest of $\Pi_{\text{prep-st-mal}}$. Notably, for packed, authenticated beaver triples, $\Pi_{\text{prep-st-mal}}$ first invokes $\mathcal{F}_{\text{triple}}$ to receive (unauthenticated) packed beaver triples. Then, after degree-reducing the ‘c components’, $\Pi_{\text{prep-st-mal}}$ uses π_{auth} to authenticate each component of the triples, followed by running $\pi_{\text{sacrifice}}$ on them, along with the associated checks of $\pi_{\text{check-zero}}$ and $\mathcal{F}_{\text{verify-deg}}$, to ensure that none of them have any error.

The parties use a similar process to generate the random authenticated sharings for the input and output gates. Also, the parties use $\mathcal{F}_{\text{rand}}$ to sample some random sharings needed throughout, including for sampling a sharing of the MAC key, $[\Delta \cdot \mathbf{1}]_{n-k}$.

Note that since $n - k > n - t$, the honest parties’ shares do not define any given sharing, and thus the adversary can always locally change the corrupted parties’ shares to change the underlying secret. We may equivalently think that the shares of corrupted parties are changed so that the secret of the sharings $[\mathbf{I}_l * \mathbf{r}_l + \delta_l]_{n-k}$ for input gates is actually $\mathbf{I}_l * \mathbf{r}_l$ (and they can then change their shares to any arbitrary values). Thus, the output sharings satisfy those demanded by the functionality (see the proof of Theorem 11 below for more).

Protocol 13: $\Pi_{\text{prep-st-mal}}$

1. **Sampling random MAC key:** All parties invoke $\mathcal{F}_{\text{rand}}$ to prepare a random sharing of the form $[\Delta \cdot \mathbf{1}]_{n-k}$ (and they ignore all other output sharings from $\mathcal{F}_{\text{rand}}$).
2. **Preparing Random, Packed, Authenticated Sharings for Input and Output Gates:** Let N_1 denote the number of input and output gates.
 - (a) The parties first invoke $\mathcal{F}_{\text{triple}}$ to prepare N_1/k packed Beaver triples $[\mathbf{r}_l]_{n-k}, [\mathbf{r}_l]_{n-k}, [\mathbf{r}_l * \mathbf{r}_l + \delta_l]_{n-1}$, for $l \in [N_1/k]$.
 - (b) Then the parties input the $[\mathbf{r}_l]_{n-k}$ ($n-t$ at a time) to π_{auth} to obtain authenticated random packed sharings $\llbracket \mathbf{r}_l \rrbracket_{n-k}$.
 - (c) Next, they invoke $\mathcal{F}_{\text{deg-reduce}}$ on input $[\mathbf{r}_l * \mathbf{r}_l + \delta_l]_{n-1}$ ($n-t$ at a time) to obtain $[\mathbf{r}_l * \mathbf{r}_l + \delta_l]_{n-k}$.
 - (d) Additionally, letting N_o be the number of output gates, the parties invoke $\mathcal{F}_{\text{rand}}$ to prepare N_o/k random sharings of the form $[\mathbf{0}]_{n-1}$.
3. **Preparing Packed, Authenticated Beaver Triples:** Let N_2 denote the number of multiplication gates. Repeat the following, until $\geq N_2/k$ packed triples are generated:
 - (a) All parties invoke $\mathcal{F}_{\text{triple}}$ twice times to generate 2μ packed Beaver triples $[\mathbf{a}_l]_{n-k}, [\mathbf{b}_l]_{n-k}, [\mathbf{c}_l + \delta_l]_{n-1}$ for $l \in [2\mu]$.
 - (b) Next, they invoke $\mathcal{F}_{\text{deg-reduce}}$ on input $[\mathbf{c}_l + \delta_l]_{n-1}$ ($n-t$ at a time, in parallel) to obtain $[\mathbf{c}_l + \delta_l]_{n-k}$.
 - (c) The parties input each component of $[\mathbf{a}_l]_{n-k}, [\mathbf{b}_l]_{n-k}, [\mathbf{c}_l + \delta_l]_{n-k}$, for $l \in [2\mu]$, ($n-t$ at a time) to π_{auth} to obtain $\llbracket \mathbf{a}_l \rrbracket_{n-k}, \llbracket \mathbf{b}_l \rrbracket_{n-k}, \llbracket \mathbf{c}_l + \delta_l \rrbracket_{n-k}$.
 - (d) Then, all parties run $\pi_{\text{sacrifice}}$ on $\llbracket \mathbf{a}_l \rrbracket_{n-k}, \llbracket \mathbf{b}_l \rrbracket_{n-k}, \llbracket \mathbf{c}_l + \delta_l \rrbracket_{n-k}$ and $\llbracket \mathbf{a}_{\mu+l} \rrbracket_{n-k}, \llbracket \mathbf{b}_{\mu+l} \rrbracket_{n-k}, \llbracket \mathbf{c}_{\mu+l} + \delta_{\mu+l} \rrbracket_{n-k}$ to compute $\llbracket \Delta \cdot \theta_l \rrbracket_{n-1}$, for $l \in [\mu]$, then input these to $\pi_{\text{check-zero}}$ to verify the correctness of the triples, and finally invoke $\mathcal{F}_{\text{verify-deg}}$ on input all of the degree- $(k-1)$ sharings from $\pi_{\text{sacrifice}}$.
 - (e) If the parties do not abort, this must mean that each $\delta_l = \mathbf{0}$, and the parties output $\llbracket \mathbf{a}_l \rrbracket_{n-k}, \llbracket \mathbf{b}_l \rrbracket_{n-k}, \llbracket \mathbf{c}_l \rrbracket_{n-k}$ for $l \in [\mu]$.
 - (f) Additionally, the parties invoke $\mathcal{F}_{\text{rand}}$ to prepare sharings $[\mathbf{0}_1]_{n-1}$ and $[\mathbf{0}_2]_{n-1}$, for each group of $k = \Omega(n)$ multiplication gates.
4. **Preparing Random, Packed Sharings for the Computation Verification:**
 - (a) The parties first invoke $\mathcal{F}_{\text{rand}}$ to prepare two random sharings $[\mathbf{0}_1]_{n-1}$ and $[\mathbf{0}_2]_{n-1}$.
 - (b) Next, they invoke $\mathcal{F}_{\text{auth-rand}}$ on input $[\Delta \cdot \mathbf{1}]_{n-k}$ to obtain a random authenticated packed sharing $\llbracket \mathbf{r} \rrbracket_{n-k}$ (and they ignore the rest).
 - (c) Finally, they locally convert $[\mathbf{r}]_{n-k}$ and $[\Delta \cdot \mathbf{r}]_{n-k}$ into additive sharings $\langle r \rangle, \langle \Delta \cdot r \rangle$ of the respective first packed secrets, and use $\mathcal{F}_{\text{rand}}$ to refresh them.^a

^a We show how to do this in Appendix D.

Note that for semi-honest security, we essentially just need $\mathcal{F}_{\text{triple}}$ for the multiplication triples, and nothing else at all (see [GPS22]).

Complexity of $\Pi_{\text{prep-st-mal}}$. We will focus on the complexity needed to generate the preprocessing required for multiplication gates:

1. Step 3a invokes $\mathcal{F}_{\text{triple}}$, which costs $2/(\varepsilon(\gamma-1/2)) \cdot (4+\zeta-2/n)$ communication and $\tilde{O}(1)$ computation and OLE correlations per underlying packed triple (which is run twice per multiplication gate).
2. Step 3b invokes $\mathcal{F}_{\text{deg-reduce}}$, which costs $6/\varepsilon - 1$ communication and $\tilde{O}(1)$ computation and OLE correlations per slot (which is run twice per multiplication gate).
3. Step 3c runs π_{auth} , which costs $2/(\varepsilon(\gamma-1/2)) \cdot (4+\zeta-2/n) + 20/\varepsilon - 8/(\varepsilon n) - 2$ communication and $\tilde{O}(1)$ computation and OLE correlations per slot (which is run six times per multiplication gate).
4. Step 3d runs $\pi_{\text{sacrifice}}$, which costs $10/\varepsilon - 10/(\varepsilon n)$ communication and $\tilde{O}(1)$ computation per pair of underlying triples. We ignore the costs of $\mathcal{F}_{\text{deg-reduce}}$ and $\pi_{\text{check-zero}}$, since they are independent of $\gamma \cdot m$.
5. Steps 4 invokes $\mathcal{F}_{\text{rand}}$ for Σ such that $|\Sigma| = 1$, which costs $1/\varepsilon$ communication and $\tilde{O}(1)$ computation per slot (which is run twice per multiplication gate).

This totals $16/(\varepsilon(\gamma-1/2)) \cdot (4+\zeta-2/n) + 144/\varepsilon - (58/(\varepsilon n) + 14)$ communication and $\tilde{O}(1)$ computation and OLE correlations for every multiplication gate.

Theorem 11. $\Pi_{\text{prep-st-mal}}$ UC-realizes $\mathcal{F}_{\text{prep-st-mal}}$ in the $(\mathcal{F}_{\text{rand}}, \mathcal{F}_{\text{triple}}, \mathcal{F}_{\text{auth-rand}}, \mathcal{F}_{\text{deg-reduce}}, \mathcal{F}_{\text{verify-deg}}, \mathcal{F}_{\text{coin}}, \mathcal{F}_{\text{commit}})$ -hybrid model against a malicious, adaptive adversary who controls $t < (1 - \varepsilon)n$ parties.

Proof. We will construct a simulator $\mathcal{S}$. First, we describe how $\mathcal{S}$ simulates π_{auth} , with knowledge of the whole sharing $[\Delta \cdot \mathbf{1}]_{n-k}$ and each $([\mathbf{u}_1]_{n-k}, \dots, [\mathbf{u}_{n-t}]_{n-k})$, which starts by setting $\text{bad-shr} \leftarrow 0$:

1. $\mathcal{S}$ first emulates $\mathcal{F}_{\text{auth-rand}}$ by receiving from the adversary the corrupted parties' shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ and then either **abort** or the corrupted parties' shares of each $[\mathbf{r}_l]_{n-k}$. In the former case, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{prep-st-mal}}$; otherwise, it samples random sharings $[\mathbf{r}_l]_{n-k} = ([\mathbf{r}_l]_{n-k}, [(\Delta \cdot \mathbf{1} + \varepsilon) * \mathbf{r}_l]_{n-k})$ consistent with the corrupted parties' shares, where ε is the difference between $\Delta \cdot \mathbf{1}$ and that which is defined by the honest parties' shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ and those input by the adversary above. Note, if the adversary corrupts more parties later, it can just send their shares to the adversary.
2. Then for $l \in [n - t]$:
 - (a) If P_1 is corrupted, $\mathcal{S}$ sends to the adversary the honest parties' shares of $[\mathbf{u}_l + \mathbf{r}_l]_{n-k}$. Otherwise, it receives from the adversary the corrupted parties' shares of $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{n-k}$, where $\boldsymbol{\eta}_l$ is error added by the adversary.
 - (b) Then, if P_1 is corrupted, $\mathcal{S}$ receives from the adversary $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1}$, where $\boldsymbol{\eta}_l$ is error added by the adversary. If these shares do not lie on a polynomial of degree $k - 1$, $\mathcal{S}$ sets $\text{bad-shr} \leftarrow 1$. If P_1 is honest, $\mathcal{S}$ computes sharing $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1}$ and sends the corrupted parties their shares.

- (c) In either case, $\mathcal{S}$ can compute the sharing $[\Delta \cdot (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l]_{n-1} \leftarrow [\Delta \cdot \mathbf{1}]_{n-k} * [\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1} - [(\Delta \cdot \mathbf{1} + \varepsilon) * \mathbf{r}_l]_{n-k}$
- 3. Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{deg-reduce}}$ by receiving from the adversary the corrupted parties' shares of each $[\Delta \cdot (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l]_{n-1}$ and then of each $[\Delta \cdot (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l]_{n-k}$, and then randomly sampling the rest of the latter sharings, consistent with the corrupted parties' shares and the values $\Delta \cdot (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l$.
- 4. Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{verify-deg}}$ by sending the adversary the corrupted parties' shares of each $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1}$ and then sending $\mathcal{F}_{\text{prep-st-mal}}$ **abort** if **bad-shr** = 1 or the adversary sends $\mathcal{S}$ **abort**.
- 5. If $\mathcal{S}$ does not abort, it outputs the sharings $([\mathbf{u}_l]_{n-k}, [\Delta \cdot (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l]_{n-k})$.

If in fact it is the case that the sharings $[\Delta \cdot \mathbf{1}]_{n-k}$ and each $([\mathbf{u}_1]_{n-k}, \dots, [\mathbf{u}_{n-t}]_{n-k})$ are distributed identically to the real world, then we show that $\mathcal{S}$'s simulation of π_{auth} is identical to the real world. It is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{auth-rand}}$ exactly as in the real world. It is also clear that $\mathcal{S}$ simulates the opening and redistribution of $[\mathbf{u}_l + \mathbf{r}_l + \boldsymbol{\eta}_l]_{k-1}$ as in the real world. Finally, it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{deg-reduce}}$ and $\mathcal{F}_{\text{verify-deg}}$ as in the real world.

Now, we also note that the corrupted parties can locally change their shares to change the secret from $\Delta \cdot (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l$ to $\Delta \cdot \mathbf{u}_l$. We may equivalently think that the shares of corrupted parties are changed so that the secret is $\Delta \cdot \mathbf{u}_l$ (and they can then change their shares to any arbitrary values). Thus, $\mathcal{S}$ adjusts the shares of corrupted parties as follows: $\mathcal{S}$ generates a degree- $(n-k)$ packed Shamir sharing $[\Delta \cdot \boldsymbol{\eta}_l - \varepsilon * \mathbf{r}_l]_{n-k}$ such that the shares of honest parties are 0's. Then $\mathcal{S}$ computes the sharing $[\Delta \cdot \mathbf{u}_l]_{n-k} \leftarrow [\Delta \cdot (\mathbf{u}_l + \boldsymbol{\eta}_l) - \varepsilon * \mathbf{r}_l]_{n-k} - [\Delta \cdot \boldsymbol{\eta}_l - \varepsilon * \mathbf{r}_l]_{n-k}$ and uses this sharing in the rest of the simulation.

Now, we describe the rest of $\mathcal{S}$:

1. **Initialization:** $\mathcal{S}$ just sends on behalf of every honest party P_i a random PRG seed to every corrupted party P_j . Note that if the adversary later corrupts more parties, $\mathcal{S}$ can just simulate their PRG seeds for other honest parties randomly. We later omit the simple hybrids which transform prg outputs to random values.
2. For sampling the MAC key, $\mathcal{S}$ first emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary the corrupted parties' shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ and then sampling the rest of the sharing randomly, consistent with the corrupted parties' shares. $\mathcal{S}$ then sends to $\mathcal{F}_{\text{prep-st-mal}}$ the corrupted parties' shares. Note that if the adversary later corrupts more parties, $\mathcal{S}$ can just send their shares to the adversary.
3. **Preparing Random, Authenticated Sharings for Input and Output Gates:**
 - (a) $\mathcal{S}$ first emulates $\mathcal{F}_{\text{triple}}$ by receiving from the adversary the corrupted parties' shares of each $[\mathbf{I}_l]_{n-k}$, $[\mathbf{r}_l]_{n-k}$, and $[\mathbf{I}_l * \mathbf{r}_l + \boldsymbol{\delta}_l]_{n-k}$, and each $\boldsymbol{\delta}_l$, and then sampling the rest of the sharings randomly, consistent with the corrupted parties' shares and each $\boldsymbol{\delta}_l$. Note that if the adversary later corrupts more parties, $\mathcal{S}$ can just send their shares to the adversary.
 - (b) Then, $\mathcal{S}$ simulates π_{auth} as above, using the whole sharings $[\Delta \cdot \mathbf{1}]_{n-k}$ and $[\mathbf{r}_l]_{n-k}$.

- (c) Next, $\mathcal{S}$ emulates $\mathcal{F}_{\text{deg-reduce}}$ by receiving from the adversary the corrupted parties' shares of each $[\mathbf{I}_l * \mathbf{r}_l + \delta_l]_{n-1}$ and the new sharings $[\mathbf{I}_l * \mathbf{r}_l + \delta_l]_{n-k}$, and then randomly sampling the latter consistent with the corrupted parties' share and the underlying values.
- (d) Finally, $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary the corrupted parties' shares of $[\mathbf{0}]_{n-1}$ and randomly sampling the rest of the sharing based on the corrupted parties' shares and $\mathbf{0}$.

$\mathcal{S}$ then sends to $\mathcal{F}_{\text{prep-st-mal}}$ the corrupted parties' shares of each $[\mathbf{r}_l]_{n-k}$, $[\mathbf{I}_l]_{n-k}$, and $[\mathbf{0}]_{n-1}$. As for $[\mathbf{I}_l * \mathbf{r}_l + \delta_l]_{n-k}$, first note that the corrupted parties can locally change their shares to change the secret to $\mathbf{I}_l * \mathbf{r}_l$. We may equivalently think that the shares of corrupted parties are changed so that the secret is $\mathbf{I}_l * \mathbf{r}_l$ (and they can then change their shares to any arbitrary values). Thus, $\mathcal{S}$ adjusts the shares of corrupted parties as follows: $\mathcal{S}$ generates a degree- $(n-k)$ packed Shamir sharing $[\delta_l]_{n-k}$ such that the shares of honest parties are 0's. Then $\mathcal{S}$ computes the shares of $[\mathbf{I}_l * \mathbf{r}_l]_{n-k} \leftarrow [\mathbf{I}_l * \mathbf{r}_l + \delta_l]_{n-k} - [\delta_l]_{n-k}$ of the corrupted parties and sends to $\mathcal{F}_{\text{prep-st-mal}}$ these shares.

4. **Preparing Packed, Authenticated Beaver Triples:**

- (a) $\mathcal{S}$ first emulates $\mathcal{F}_{\text{triple}}$ by receiving from the adversary the corrupted parties' shares of each $[\mathbf{a}_l]_{n-k}$, $[\mathbf{b}_l]_{n-k}$, and $[\mathbf{c}_l + \delta_l]_{n-k}$, and each δ_l , and then sampling the rest of the sharings randomly, consistent with the corrupted parties' shares and each δ_l . Note that if the adversary later corrupts more parties, $\mathcal{S}$ can just send their shares to the adversary.
- (b) Then, $\mathcal{S}$ emulates $\mathcal{F}_{\text{deg-reduce}}$ by receiving from the adversary the corrupted parties' shares of each $[\mathbf{c}_l * + \delta_l]_{n-1}$ and the new sharings $[\mathbf{c}_l + \delta_l]_{n-k}$, and then randomly sampling the latter consistent with the corrupted parties' share and the underlying values.
- (c) Then, $\mathcal{S}$ simulates π_{auth} as above, using the whole sharings $[\Delta \cdot \mathbf{1}]_{n-k}$ and $[\mathbf{a}_l]_{n-k}$, $[\mathbf{b}_l]_{n-k}$, $[\mathbf{c}_l + \delta_l]_{n-k}$
- (d) To simulate $\pi_{\text{sacrifice}}$:
 - i. $\mathcal{S}$ first emulates $\mathcal{F}_{\text{coin}}$ by sampling random ρ and sending it to the adversary.
 - ii. If P_1 is corrupted, $\mathcal{S}$ sends to the adversary the honest parties' shares of $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{n-k}$ and $[\mathbf{b}_2 - \mathbf{b}_1]_{n-k}$. Otherwise, it receives from the adversary the corrupted parties' shares of $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{n-k}$ and $[\mathbf{b}_2 - \mathbf{b}_1]_{n-k}$.
 - iii. Then, if P_1 is corrupted, $\mathcal{S}$ receives from the adversary $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{k-1}$, $[\mathbf{b}_2 - \mathbf{b}_1]_{k-1}$, and $[(\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1)]_{k-1}$. If the shares do not lie on a polynomial of degree $k-1$ for any of the sharings, $\mathcal{S}$ sets $\text{bad-shr} \leftarrow 1$. If P_1 is honest, $\mathcal{S}$ computes the sharings themselves, and sends the corrupted parties their shares.
 - iv. In either case, $\mathcal{S}$ computes the sharing

$$\begin{aligned}
 [\Delta \cdot \theta]_{n-1} &\leftarrow [\Delta \cdot \mathbf{1}]_{n-k} * [(\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1)]_{k-1} \\
 &+ [\Delta \cdot \mathbf{b}_1]_{n-k} * [\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{k-1} + \rho \cdot [\Delta \cdot \mathbf{a}_1]_{n-k} * [\mathbf{b}_2 - \mathbf{b}_1]_{k-1} + \\
 &\rho \cdot [\Delta \cdot \mathbf{c}_1]_{n-k} - [\Delta \cdot \mathbf{c}_2]_{n-k}.
 \end{aligned}$$

- (e) To simulate $\pi_{\text{check-zero}}$:
 - i. $\mathcal{S}$ first emulates $\mathcal{F}_{\text{coin}}$ by sampling random $\chi_1, \dots, \chi_M$ and sending them to the adversary.
 - ii. Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary the corrupted parties' shares of $[\mathbf{0}]_{n-1}$, and then sampling the rest of the sharing randomly, based on the corrupted parties' shares and $\mathbf{0}$.
 - iii. Then $\mathcal{S}$ computes $[\boldsymbol{\theta}]_{n-1} \leftarrow [\mathbf{0}]_{n-1} + \sum_{l=1}^M \chi_l \cdot [\boldsymbol{\theta}_l]_{n-1}$.
 - iv. Next $\mathcal{S}$ emulates $\mathcal{F}_{\text{commit}}$ by receiving from the adversary the corrupted parties' shares of $[\boldsymbol{\theta}]_{n-1}$.
 - v. Finally, $\mathcal{S}$ sends the adversary the honest parties' shares of $[\boldsymbol{\theta}]_{n-1}$ and receives the corrupted parties' shares from the adversary. If the corrupted parties' shares are different from the committed values or the computed shared value $\boldsymbol{\theta} \neq \mathbf{0}$, then $\mathcal{S}$ sends to $\mathcal{F}_{\text{prep-st-mal}}$, **abort**.
 - (f) Finally, $\mathcal{S}$ emulates $\mathcal{F}_{\text{verify-deg}}$ by sending to the adversary the corrupted parties' shares of each $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{k-1}$, $[\mathbf{b}_2 - \mathbf{b}_1]_{k-1}$, and $[(\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1)]_{k-1}$, from $\pi_{\text{sacrifice}}$, then sending **abort** to $\mathcal{F}_{\text{prep-st-mal}}$ if $\text{bad-shr} = 1$ or the adversary sends $\mathcal{S}$ **abort**.
 - (g) Additionally, $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary the corrupted parties' shares of $[\mathbf{0}_1]_{n-1}$ and $[\mathbf{0}_2]_{n-1}$, and then sampling the rest of the sharings randomly, based on the corrupted parties' shares and $\mathbf{0}$. $\mathcal{S}$ then sends to $\mathcal{F}_{\text{prep-st-mal}}$ the corrupted parties' shares of each $[\mathbf{a}_i]_{n-k}$, $[\mathbf{b}_i]_{n-k}$, $[\mathbf{c}_i]_{n-k}$, as well as $[\mathbf{0}_1]_{n-1}$ and $[\mathbf{0}_2]_{n-1}$.
5. **Preparing Random Sharings for the Computation Verification:**
- (a) $\mathcal{S}$ first emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary the corrupted parties' shares of $[\mathbf{0}_1]_{n-1}$ and $[\mathbf{0}_2]_{n-1}$, and then sampling the rest of the sharings randomly, based on the corrupted parties' shares and $\mathbf{0}$. Note that if the adversary later corrupts more parties, $\mathcal{S}$ can just send their shares to the adversary.
 - (b) Then, $\mathcal{S}$ emulates $\mathcal{F}_{\text{auth-rand}}$ by receiving from the adversary the corrupted parties' shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ and then either **abort** or the corrupted parties' shares of $[\mathbf{r}]_{n-k}$. In the former case, $\mathcal{S}$ sends **abort** to $\mathcal{F}_{\text{prep-st-mal}}$; otherwise, it samples random sharing $[\mathbf{r}]_{n-k}$ consistent with the corrupted parties' shares. Note that if the adversary later corrupts more parties, $\mathcal{S}$ can just send their shares to the adversary.
 - (c) $\mathcal{S}$ then emulates $\mathcal{F}_{\text{rand}}$ by receiving from the adversary the corrupted parties' shares of $\langle \mathbf{0}_1 \rangle$ and $\langle \mathbf{0}_2 \rangle$, and then sampling the rest of the sharings randomly, based on the corrupted parties' shares and $\mathbf{0}$. $\mathcal{S}$ then sends to $\mathcal{F}_{\text{prep-st-mal}}$ the corrupted parties' shares of $[\mathbf{0}_1]_{n-1}$ and $[\mathbf{0}_2]_{n-1}$, as well as $\langle \mathbf{r} \rangle$, $\langle \Delta \cdot \mathbf{r} \rangle$.

Now we argue that the real and ideal worlds are indistinguishable. For sampling of the MAC key, it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ as in the real world. Also, the shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ that the honest parties output in the real world are distributed randomly given the corrupted parties' shares, by definition of $\mathcal{F}_{\text{rand}}$. This is exactly the same as how $\mathcal{F}_{\text{prep-st-mal}}$ samples the honest parties' shares.

For the sharings of input and output gates: It is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{triple}}$ as in the real world. From above, we can also argue that $\mathcal{S}$'s simulation of π_{auth} is

as in the real world. It is also clear that $\mathcal{S}$'s emulation of $\mathcal{F}_{\text{deg-reduce}}$ and $\mathcal{F}_{\text{rand}}$ are identical to the real world. Now we just need to argue that the shares that the honest parties output in the real world are identical to that of the ideal world. By definition of $\mathcal{F}_{\text{triple}}$ and $\mathcal{F}_{\text{rand}}$, the shares of $[\mathbf{r}_l]_{n-k}$, $[\mathbf{I}_l]_{n-k}$, and $[\mathbf{0}]_{n-1}$ that the honest parties output in the real world are distributed randomly given the corrupted parties' shares (and $\mathbf{0}$ for $[\mathbf{0}]_{n-1}$). This holds even for $[\mathbf{r}_l]_{n-k}$, which is masked by random $[\mathbf{r}'_l]_{n-k}$ in π_{auth} . This is exactly the same as how $\mathcal{F}_{\text{prep-st-mal}}$ samples the honest parties' shares. In both the real and ideal worlds, $[\Delta \cdot \mathbf{r}_l]_{n-k}$ is distributed randomly given the corrupt parties' shares together with $\Delta \cdot \mathbf{r}_l$. The same can be said about $[\mathbf{I}_l * \mathbf{r}_l]_{n-k}$, where in the real world, the corrupted parties shares are also changed so that the secret is $\mathbf{I}_l * \mathbf{r}_l$.

For the packed, authenticated beaver triples: It is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{triple}}$ and $\mathcal{F}_{\text{deg-reduce}}$ as in the real world. From above, we can also argue that $\mathcal{S}$'s simulation of π_{auth} is as in the real world. In the simulation of $\pi_{\text{sacrifice}}$, it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{coin}}$ as in the real world. It is also clear that the opening and redistribution of $[\mathbf{a}_2 - \rho \cdot \mathbf{a}_1]_{k-1}$, $[\mathbf{b}_2 - \mathbf{b}_1]_{k-1}$, and $[(\mathbf{a}_2 - \rho \cdot \mathbf{a}_1) * (\mathbf{b}_2 - \mathbf{b}_1)]_{k-1}$ is as in the real world. In the simulation of $\pi_{\text{check-zero}}$ it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{coin}}$, $\mathcal{F}_{\text{rand}}$, and $\mathcal{F}_{\text{commit}}$ as in the real world. It is also clear that it computes $[\boldsymbol{\theta}]_{n-1}$ as in the real world. So, the honest parties abort in the ideal world if and only if they do in the real world. Finally, it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{verify-deg}}$ and $\mathcal{F}_{\text{rand}}$ as in the real world. Now, from Lemma 14, we know that if any $\Delta \cdot \boldsymbol{\theta}_{l^*}$ is nonzero, and random and unknown to the adversary, then the honest parties will abort with all-but-negligible probability. In particular, this means that each $[\mathbf{c}_1]_{n-k}$ cannot have any error, for otherwise, the corresponding $\Delta \cdot \boldsymbol{\theta}_{l^*}$ would contain $\rho \cdot \Delta \cdot \boldsymbol{\delta}_1$ and thus be random and unknown to the adversary. Observe also that in both worlds, if the adversary does not introduce any errors, then $\Delta \cdot \boldsymbol{\theta} = \mathbf{0}$. Now we just need to argue that the shares that the honest parties output in the real world are identical to that of the ideal world. In both worlds, it is clear that $[\mathbf{a}_1]_{n-k}$, $[\mathbf{b}_1]_{n-k}$, $[\mathbf{0}_1]_{n-1}$, $[\mathbf{0}_2]_{n-1}$ are randomly distributed given the corrupted parties' shares (and also $\mathbf{0}$ for the last two). Each $[\mathbf{c}_l]_{n-k}$ is distributed randomly given the corrupted parties' shares and $\mathbf{a}_l * \mathbf{b}_l$. The same can be said of $[\Delta \cdot \mathbf{a}_l]_{n-k}$, $[\Delta \cdot \mathbf{b}_l]_{n-k}$, $[\Delta \cdot \mathbf{c}_l]_{n-k}$, given the corrupted parties' shares and $\Delta \cdot \mathbf{a}_l$, $\Delta \cdot \mathbf{b}_l$, and $\Delta \cdot \mathbf{c}_l$, respectively.

For the random sharings for computation verification: it is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{rand}}$ and $\mathcal{F}_{\text{auth-rand}}$ as in the real world. It is also clear that in both worlds, $[\mathbf{0}_1]_{n-1}$, $[\mathbf{0}_2]_{n-1}$ are distributed randomly given the corrupted parties' shares and $\mathbf{0}$. Also, the honest parties' shares of $\langle r \rangle$, $\langle \Delta \cdot r \rangle$ are distributed randomly.

Thus, we have argued that the real and ideal worlds are indistinguishable with all-but-negligible probability. $\square$

G Missing Proofs

G.1 Proof of Lemma 5

Proof. First we define the simulator $\mathcal{S}$, which initializes $\text{bad-shr} \leftarrow 0$:

1. **Extraction Level 1:**

- (a) $\mathcal{S}$ first samples random $\mathbf{u}_i, \mathbf{v}_i \in \mathbb{F}^t$ on behalf of each honest party P_i . Note if P_i is late corrupted, $\mathcal{S}$ can send the adversary these random vectors.
- (b) Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{coin}}$ by sampling random $r \in \mathbb{F}$ and sending it to the adversary. It then interprets r as bit string $\mathbf{R}$, and locally computes the matrix $\mathbf{M} \leftarrow \text{Gen}(\mathbf{R})$.
- (c) Then, for every corrupted party P_j , let $\mathbf{u}_j$ be the first vector it inputs to $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ as party P_A and $\mathbf{v}_j$ be the first vector it inputs to $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ as party P_B . For each $l \in [m]$, for every ordered pair (P_i, P_j) such that $\mathbf{M}_l[i], \mathbf{M}_l[j] \neq 0$, $\mathcal{S}$ emulates $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ as follows:
 - If P_i and P_j are both corrupted, then $\mathcal{S}$ receives from the adversary α_i^j, β_j^i and returns them to the adversary.
 - If P_i is corrupted and P_j is honest, $\mathcal{S}$ first receives $\bar{\mathbf{u}}_i$ and α_i^j from the adversary, then computes $\beta_j^i \leftarrow \bar{\mathbf{u}}_i * \mathbf{v}_j - \alpha_i^j$, and returns α_i^j to the adversary.
 - If P_i is honest and P_j is corrupted, $\mathcal{S}$ first receives $\bar{\mathbf{v}}_j$ and β_j^i from the adversary, then computes $\alpha_i^j \leftarrow \mathbf{u}_i * \bar{\mathbf{v}}_j - \beta_j^i$ and returns β_j^i to the adversary.
 - If both P_i and P_j are honest, $\mathcal{S}$ samples random β_j^i and sets $\alpha_i^j \leftarrow \mathbf{u}_i * \mathbf{v}_j - \beta_j^i$.
 - In each case, $\mathcal{S}$ computes $\varepsilon^{(i,j)} \leftarrow \alpha_i^j + \beta_j^i - \mathbf{u}_i * \mathbf{v}_j$.
- (d) Then, for each honest party P_i , $\mathcal{S}$ splits $\mathbf{u}_i, \mathbf{v}_i$ and each α_i^j, β_j^i into length- k vectors $\{\mathbf{u}_{i,w}\}_{w \in [t/k]}, \{\mathbf{v}_{i,w}\}_{w \in [t/k]}, \{\alpha_{i,w}^j\}_{w \in [t/k]}, \{\beta_{i,w}^j\}_{w \in [t/k]}$, respectively. $\mathcal{S}$ also splits each $\varepsilon^{(i,j)}$ into $\{\varepsilon_w^{(i,j)}\}_{w \in [t/k]}$.
- (e) Then, for $w \in [t/k]$:
 - i. For each honest party, $\mathcal{S}$ computes random packed sharings $[\mathbf{u}_{i,w}]_{n-k}, [\mathbf{v}_{i,w}]_{n-k}$ and sends the adversary the corrupted parties' shares (Note if the adversary later corrupts more parties, $\mathcal{S}$ just sends the adversary their shares). $\mathcal{S}$ also receives from each corrupted party P_j , the honest parties' shares of $[\mathbf{u}_{j,w}]_{n-k}, [\mathbf{v}_{j,w}]_{n-k}$, and samples the rest of the sharings based on the honest parties' shares and $\mathbf{u}_{j,w}, \mathbf{v}_{j,w}$, respectively.
 - ii. $\mathcal{S}$ computes $([\mathbf{a}_{1,w}]_{n-k}, \dots, [\mathbf{a}_{m,w}]_{n-k})$ and $([\mathbf{b}_{1,w}]_{n-k}, \dots, [\mathbf{b}_{m,w}]_{n-k})$ using the above and $\mathbf{M}$.
 - iii. Then, for each $l \in [m]$:
 - A. For each honest party P_i such that $\mathbf{M}_l[i] \neq 0$, $\mathcal{S}$ computes $\mathbf{c}_{i,w}^l \leftarrow \sum_{j: \mathbf{M}_l[j] \neq 0} \alpha_{i,w}^j + \beta_{i,w}^j$ then computes degree- $(n-k)$ packed Shamir sharing $[\mathbf{c}_{i,w}^l]_{n-k}$ and sends the adversary the corrupted parties' shares (Note if the adversary later corrupts more parties, $\mathcal{S}$ just sends the adversary their shares). $\mathcal{S}$ also receives from each corrupted party P_j such that $\mathbf{M}_l[j] \neq 0$, the honest parties' shares of $[\mathbf{c}_{j,w}^l]_{n-k}$ and samples the rest of the sharing based on the honest parties' shares and $\mathbf{c}_{i,w}^j$.

- B. Finally, using the above, $\mathcal{S}$ can compute $[c_{l,w}]_{n-k}$. $\mathcal{S}$ also computes $\varepsilon_{l,w} \leftarrow \sum_{(i,j): M_l[i] \neq 0 \neq M_l[j]} \varepsilon_w^{(i,j)}$.
2. **Extraction Level 2:** For each $w \in [[t/k]]$:
- For $l \in [(m-1)/2 + 1]$, $\tau \in [k]$, $\mathcal{S}$ implicitly views $a_{l,w}^\tau, b_{l,w}^\tau, c_{l,w}^\tau$ as the l -th evaluation points of polynomials $A_{\tau,w}(\cdot)$ of degree $(m-1)/2$, $B_{\tau,w}(\cdot)$ of degree $(m-1)/2$, $C_{\tau,w}(\cdot)$ of degree $m-1$, respectively; i.e., $A_{\tau,w}(l) = a_{l,w}^\tau$, $B_{\tau,w}(l) = b_{l,w}^\tau$, and $C_{\tau,w}(l) = c_{l,w}^\tau$.
 - Letting $\mathbf{A}_w(\cdot) \leftarrow (A_{1,w}(\cdot), \dots, A_{k,w}(\cdot))$ and $\mathbf{B}_w(\cdot) \leftarrow (B_{1,w}(\cdot), \dots, B_{k,w}(\cdot))$, $\mathcal{S}$ uses $([\mathbf{A}_w(1)]_{n-k}, \dots, [\mathbf{A}_w((m-1)/2+1)]_{n-k})$ and $([\mathbf{B}_w(1)]_{n-k}, \dots, [\mathbf{B}_w((m-1)/2+1)]_{n-k})$, which implicitly define the polynomials $\mathbf{A}_w(\cdot)$ and $\mathbf{B}_w(\cdot)$, respectively, to locally compute the corrupted parties' shares of $[\mathbf{A}_w(l)]_{n-k}$ and $[\mathbf{B}_w(l)]_{n-k}$ for $l \in [(m-1)/2 + 2, m]$.
 - Then, for $l \in [(m-1)/2 + 2, m]$, to simulate π_{beaver} , $\mathcal{S}$ first sets $\eta_{l,w} \leftarrow \mathbf{0}$ and computes $\mathbf{d} \leftarrow \mathbf{A}_w(l) - \mathbf{a}_{l,w}$ and $\mathbf{e} \leftarrow \mathbf{B}_w(l) - \mathbf{b}_{l,w}$, then:
 - If P_1 is corrupt:
 - $\mathcal{S}$ first locally computes the honest parties' shares of $[\mathbf{A}_w(l)]_{n-k} - [\mathbf{a}_{l,w}]_{n-k}$ and $[\mathbf{B}_w(l)]_{n-k} - [\mathbf{b}_{l,w}]_{n-k}$ then sends them to the adversary.
 - $\mathcal{S}$ then receives from the adversary $(d_{i_1}, e_{i_1}), \dots, (d_{i_{n-t}}, e_{i_{n-t}})$ on behalf of the honest parties. If these shares do not lie on a polynomial of degree $k-1$, $\mathcal{S}$ sets $\text{bad-shr} \leftarrow 1$. Otherwise, it reconstructs $[\bar{\mathbf{d}}]_{k-1}, [\bar{\mathbf{e}}]_{k-1}$.
 - In the latter case, $\mathcal{S}$ for each corrupted party then computes its share of $[\mathbf{C}_w(l)]_{n-1}$ as $[\bar{\mathbf{d}}]_{k-1} * [\mathbf{e}]_{k-1} + [\bar{\mathbf{e}}]_{k-1} * [\mathbf{d}]_{k-1} + [\mathbf{b}_{l,w}]_{n-k} + [\mathbf{e}]_{k-1} * [\mathbf{a}_{l,w}]_{n-k} + [\mathbf{c}_{l,w}]_{n-k}$.
 - If P_1 is honest:
 - $\mathcal{S}$ receives from the adversary the corrupt parties' shares of $[\mathbf{A}_w(l)]_{n-k} - [\mathbf{a}_{l,w}]_{n-k}$ and $[\mathbf{B}_w(l)]_{n-k} - [\mathbf{b}_{l,w}]_{n-k}$.
 - Then, together with the honest parties' shares, $\mathcal{S}$ reconstructs $\bar{\mathbf{d}} \leftarrow \mathbf{A}_w(l) - \mathbf{a}_{l,w}$ and $\bar{\mathbf{e}} \leftarrow \mathbf{B}_w(l) - \mathbf{b}_{l,w}$, computes $[\bar{\mathbf{d}}]_{k-1}$ and $[\bar{\mathbf{e}}]_{k-1}$, and then sends the corrupt parties' their shares.
 - Finally, $\mathcal{S}$ computes the corrupted parties' shares of $[\mathbf{C}_w(l)]_{n-1}$ as above.
- In both cases, $\mathcal{S}$ computes $\eta_{l,w} \leftarrow (\mathbf{d}_{l,w} * \mathbf{e}_{l,w} + \mathbf{d}_{l,w} * \mathbf{b}_{l,w} + \mathbf{e}_{l,w} * \mathbf{a}_{l,w} + \mathbf{c}_{l,w}) - (\bar{\mathbf{d}}_{l,w} * \bar{\mathbf{e}}_{l,w} + \bar{\mathbf{d}}_{l,w} * \mathbf{b}_{l,w} + \bar{\mathbf{e}}_{l,w} * \mathbf{a}_{l,w} + \mathbf{c}_{l,w})$
- Note: sharings $([\mathbf{A}_w(1)]_{n-k}, \dots, [\mathbf{A}_w(m)]_{n-k})$, $([\mathbf{B}_w(1)]_{n-k}, \dots, [\mathbf{B}_w(m)]_{n-k})$, and $([\mathbf{C}_w(1)]_{n-1}, \dots, [\mathbf{C}_w(m)]_{n-1})$ implicitly define the polynomials $\mathbf{A}_w(\cdot)$, $\mathbf{B}_w(\cdot)$, and $\mathbf{C}_w(\cdot)$, respectively. Thus, for $l \in [m+1, m+\gamma \cdot m - (m+1)/2]$, $\mathcal{S}$ locally computes the corrupted parties' shares of

$$([\mathbf{a}^{l,w}]_{n-k}, [\mathbf{b}^{l,w}]_{n-k}, [\mathbf{c}^{l,w}]_{n-1}) \leftarrow ([\mathbf{A}_w(m+l)]_{n-k}, [\mathbf{B}_w(m+l)]_{n-k}, [\mathbf{C}_w(m+l)]_{n-1}).$$

The errors $\varepsilon_{1,w}, \dots, \varepsilon_{(m-1)/2+1,w}, \varepsilon_{(m-1)/2+2,w} + \eta_{(m-1)/2+2,w}, \dots, \varepsilon_m + \eta_m$ also define degree- $m-1$ polynomials $\delta_w(\cdot)$. $\mathcal{S}$ computes $\delta_w(l)$ for $l \in [m+1, m+\gamma \cdot m - (m+1)/2]$ based on these.

3. **Checking the degree of P_1 's sharings:** $\mathcal{S}$ then emulates $\mathcal{F}_{\text{verify-deg}}$ by first sending the adversary the corrupted parties' shares of the sharings $[\mathbf{d}]_{d_1}, [\mathbf{e}]_{d_2}$ from π_{beaver} , then sending **abort** to $\mathcal{F}_{\text{triple}}$ if $\text{bad-shr} = 1$ or the adversary sends **abort** to $\mathcal{S}$.
4. **Output:** If $\mathcal{S}$ has not yet aborted it sends to $\mathcal{F}_{\text{triple}}$ the corrupted parties' shares of $([\mathbf{a}^{l,w}]_{n-k}, [\mathbf{b}^{l,w}]_{n-k}, [\mathbf{c}^{l,w}]_{n-1})$ and $\delta_w(l)$ for all $w \in [t/k], l \in [m+1, m+\gamma \cdot m - (m+1)/2]$.

Now we must show that the real and ideal worlds are indistinguishable. It is clear that $\mathcal{S}$ emulates $\mathcal{F}_{\text{coin}}$ and $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ in the first extraction level as in the real world, and that $\delta^{(i,j)}$ correctly computes the difference between $\mathbf{u}_i * \mathbf{v}_j$ based on the first seeds that the corrupted parties input to $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ and those that it inputs to the $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ invocation for the ordered pair (i, j) . It is also clear that the distribution of corrupted parties' shares of honest sharings $[\mathbf{u}_{i,w}]_{n-k}, [\mathbf{v}_{i,w}]_{n-k}$ are identical in the two worlds, and same with that of $[\mathbf{c}_{l,w}^i]_{n-k}$.

For the simulation of π_{beaver} in the second extraction level, let us first handle the case in which P_1 is corrupt. Since $\mathcal{S}$ just uses the already computed honest parties' shares when sending to P_1 , $[\mathbf{A}_w(l)]_{n-k} - [\mathbf{a}_{l,w}]_{n-k}$ and $[\mathbf{B}_w(l)]_{n-k} - [\mathbf{b}_{l,w}]_{n-k}$, it is clear that this is an identical distribution to that of the real world. It is also clear that $\mathcal{S}$ correctly sets $\text{bad-shr} \leftarrow 1$ if the honest parties' shares of $[\mathbf{d}]_{k-1}, [\mathbf{e}]_{k-1}$ do not lie on a degree- $(k-1)$ polynomial. Now, if P_1 is honest, again since $\mathcal{S}$ just uses the already computed honest parties' shares, the reconstruction of $\mathbf{d}$ and $\mathbf{e}$ occur as in the real world, as does the computation of $[\mathbf{d}]_{k-1}, [\mathbf{e}]_{k-1}$. It is also clear that $\mathcal{S}$'s emulation of $\mathcal{F}_{\text{verify-deg}}$ is identical to the real world.

Now we show that if the honest parties do not abort, then their shares are distributed identically in the ideal and real worlds. Let $H \subseteq \text{Hon}$ be some subset of Hon of size $|H| = n - t$ and $\mathbf{M}^H$ be the sub-matrix of $\mathbf{M}$ that contains the columns corresponding to the indices of the honest parties in H . From Theorem 4, we have that with all-but-negligible probability, $\mathbf{M}^H$ has rank at least $\gamma \cdot m$, for $\gamma > 1/2$. This means that there must be some $(\gamma \cdot m) \times (\gamma \cdot m)$ dimensional sub-matrix $\mathbf{M}'$ of $\mathbf{M}^H$ that is invertible. Let $R = \{r_1, \dots, r_{\gamma \cdot m}\}$ be the rows of $\mathbf{M}$ of which $\mathbf{M}'$ consists, $C = \{c_1, \dots, c_{\gamma \cdot m}\}$ be those columns, and $\bar{C} = \{\bar{c}_1, \dots, \bar{c}_{n-\gamma \cdot m}\}$ be those columns of $\mathbf{M}$ that are not in $\mathbf{M}'$. Then we can write

$$([\mathbf{a}_{r_1,w}]_{n-k}, \dots, [\mathbf{a}_{r_{\gamma \cdot m},w}]_{n-k})^\top = \mathbf{M}' \cdot ([\mathbf{u}_{c_1,w}]_{n-k}, \dots, [\mathbf{u}_{c_{\gamma \cdot m},w}]_{n-k})^\top + \mathbf{M}_R^{\bar{C}} \cdot ([\mathbf{u}_{\bar{c}_1,w}]_{n-k}, \dots, [\mathbf{u}_{\bar{c}_{n-\gamma \cdot m},w}]_{n-k})^\top.$$

Since $\mathbf{M}'$ is invertible, for any fixed sharings $([\mathbf{u}_{\bar{c}_1,w}]_{n-k}, \dots, [\mathbf{u}_{\bar{c}_{n-\gamma \cdot m},w}]_{n-k})$, the distribution of $([\mathbf{a}_{r_1,w}]_{n-k}, \dots, [\mathbf{a}_{r_{\gamma \cdot m},w}]_{n-k})$ is the same as $([\mathbf{u}_{c_1,w}]_{n-k}, \dots, [\mathbf{u}_{c_{\gamma \cdot m},w}]_{n-k})$, which is random given the shares of corrupted parties. Note also that what the adversary receives from $\mathcal{S}$'s emulation of $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ is independent of the $\mathbf{u}_{c_1,w}, \dots, \mathbf{u}_{c_{\gamma \cdot m},w}$. Furthermore, the distribution of the sharings $[\mathbf{c}_{l,w}^i]$ for each honest party P_i is random given the corrupted parties' shares. So, the above still holds that the distribution of $([\mathbf{a}_{r_1,w}]_{n-k}, \dots, [\mathbf{a}_{r_{\gamma \cdot m},w}]_{n-k})$ is random given the shares of corrupted parties.

In the second level of extraction, consider the vector of polynomials $\mathbf{A}_w(\cdot)$. We have two cases. First, we have the case that $[(m-1)/2+1] \subseteq R$. In this case, we have from above that for each $r \in R \cap [(m-1)/2+1] = [(m-1)/2+1]$, the sharing $[\mathbf{A}_w(r)]_{n-k} = [\mathbf{a}_{r,w}]_{n-k}$ is distributed randomly given the shares of corrupted parties. Also, for each $r \in R \cap [(m-1)/2+2, m]$, the sharing $[\mathbf{a}_{r,w}]_{n-k}$ is distributed randomly given the shares of corrupted parties, and thus so is $[\mathbf{A}_w(r)]_{n-k} + [\mathbf{a}_{r,w}]_{n-k}$. Therefore, all but $m - \gamma \cdot m$ points of each polynomial $\mathbf{A}_w(\cdot)$ are random to the adversary. Indeed, since each polynomial in $\mathbf{A}_w$ is of degree $(m-1)/2$, the $m - \gamma \cdot m < (m-1)/2 + 1$ points of each polynomial which may not be random to the adversary are independent of the remaining $(m-1)/2 + 1 - (m - \gamma \cdot m) = \gamma \cdot m - (m+1)/2$ points, all of which are random. This means that the sharings $[\mathbf{A}_w(r)]_{n-k}$ corresponding to these points are still distributed randomly given the shares of the corrupted parties.

In the second case, we have that $[(m-1)/2+1] \not\subseteq R$, which means that $[(m-1)/2+2, m] \subseteq R$, since $|R| = \gamma \cdot m > m/2$. In this case, we have from above that for each $r \in R \cap [(m-1)/2+1]$, the sharing $[\mathbf{A}_w(r)]_{n-k} = [\mathbf{a}_{r,w}]_{n-k}$ is distributed randomly given the shares of corrupted parties. Moreover, since each polynomial in $\mathbf{A}_w$ is of degree $(m-1)/2$, the points of each polynomial corresponding to $[(m-1)/2+1] \setminus R$ are independent of those that are in $R \cap [(m-1)/2+1]$. Furthermore, for each $r \in R \cap [(m-1)/2+2, m] = [(m-1)/2+2, m]$, the sharing $[\mathbf{a}_{r,w}]_{n-k}$ is distributed randomly given the shares of corrupted parties, and thus so is $[\mathbf{A}_w(r)]_{n-k} + [\mathbf{a}_{r,w}]_{n-k}$. Therefore, the adversary learns nothing else about those $[\mathbf{A}_w(r)]_{n-k}$ for $r \in R \cap [(m-1)/2+1]$, of which there must be $(m-1)/2+1 - (m - \gamma \cdot m) = \gamma \cdot m - (m+1)/2$.

In either case, $[\mathbf{A}_w(1)]_{n-k}, \dots, [\mathbf{A}_w((m-1)/2+1)]_{n-k}$, of which $\gamma \cdot m - (m+1)/2$ are random given the corrupted parties' shares, are used to compute the output $[\mathbf{A}_w(m+l)]_{n-k}$ for $l \in [m+1, m + \gamma \cdot m - (m+1)/2]$, in a one-to-one fashion (since the computation is represented by a super-invertible matrix whose number of rows is equal to $\gamma \cdot m - (m+1)/2$), which are therefore random given the corrupted parties' shares.

In the ideal world, $\mathcal{S}$ generates the corrupted parties' shares of those $[\mathbf{u}_{j,w}]_{n-k}$ for $j \in \text{Corr}$ by randomly sampling their shares based on those of the honest parties and the underlying $\mathbf{u}$, which $\mathcal{S}$ received from its emulation of $\mathcal{F}_{\text{OLE}}^{\text{prog}}$. Using these, and the corrupted parties' shares of honest parties' sharings, $\mathcal{S}$ computes the corrupted parties' shares of the output sharings by following the protocol description. Then the shares of the honest parties are randomly sampled based on the shares of corrupted parties. Thus the distribution of the output shares of honest parties are identical in the two worlds.

Note that the same exact argument as above can be used for the output $[\mathbf{B}_w(m+l)]_{n-k}$ for $l \in [m+1, m + \gamma \cdot m - (m+1)/2]$.

For the honest parties' shares of the output $[\mathbf{c}^{l,w}]_{n-k}$, recall that the distribution of the sharings $[\mathbf{c}_{i,w}^l]$ for each honest party P_i is random given the corrupted parties' shares. Since the $[\mathbf{c}^{l,w}]_{n-k}$ are linear combinations of $(([\mathbf{a}_{1,w}]_{n-k}, [\mathbf{b}_{1,w}]_{n-k}, [\mathbf{c}_{1,w}]_{n-k}), \dots, ([\mathbf{a}_{m,w}]_{n-k}, [\mathbf{b}_{m,w}]_{n-k}, [\mathbf{c}_{m,w}]_{n-k}))$, then it must be that the shares of honest parties are random given the corrupted parties' shares and such that

they are consistent with $\mathbf{a}^{l,w} * \mathbf{b}^{l,w}$ and any errors which the adversary injected for $\mathcal{F}_{\text{OLE}}^{\text{prog}}$ and in π_{beaver} . Since $\mathcal{S}$ tracks these errors and inputs them to $\mathcal{F}_{\text{triple}}$, the ideal world is distributed identically to the real world. $\square$

G.2 Proof of Lemma 6

Proof. First we define the simulator $\mathcal{S}$, which begins by setting $\text{bad-shr} \leftarrow 0$:

1. $\mathcal{S}$ first emulates $\mathcal{F}_{\text{triple}}$ by receiving from the adversary the corrupted parties' shares and $(\delta_{l,w})_{l \in \mu, w \in [\lfloor t/k \rfloor]}$. It then samples random sharings $([\mathbf{a}_{l,w}]_{n-k}, [\mathbf{b}_{l,w}]_{n-k}, [\mathbf{a}_{l,w} * \mathbf{b}_{l,w} + \delta_{l,w}]_{n-k})_{l \in \mu, w \in [\lfloor t/k \rfloor]}$ consistent with the corrupted parties' shares. Note, if the adversary corrupts more parties after the invocation of $\mathcal{F}_{\text{triple}}$, it can just send the corresponding shares to the adversary.
2. Then for $w \in [\lfloor t/k \rfloor], l \in [\mu]$:
 - (a) If P_1 is corrupted, $\mathcal{S}$ sends the adversary random $\rho_{l,w}^i$ for $i \in \text{Hon}$. Else it receives from the adversary its shares of $[\Delta \cdot \mathbf{1} + \mathbf{s}_{l,w}]_{n-k}$ and using the corrupted parties' shares of $[\mathbf{s}_{l,w}]_{n-k}$ received from the adversary above, $\mathcal{S}$ computes their shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ and sends them to $\mathcal{F}_{\text{auth-rand}}$.
 - (b) Then, if P_1 is corrupted, $\mathcal{S}$ receives from the adversary the honest parties' shares of $[\Delta \cdot \mathbf{1} + \mathbf{s}_{l,w}]_{k-1}$. If these shares do not lie on a polynomial of degree $k-1$, $\mathcal{S}$ sets $\text{bad-shr} \leftarrow 1$. Otherwise, it reconstructs $\Delta \cdot \mathbf{1} + \mathbf{s}_{l,w}$ and since $\mathbf{s}_{l,w}$ is sampled above, computes $\Delta \cdot \mathbf{1}$. Then, $\mathcal{S}$ can compute for each honest party $(\Delta \cdot \mathbf{1})^i \leftarrow \rho_{l,w}^i - s_{l,w}^i$ and based on these and $\Delta \cdot \mathbf{1}$, sample a sharing $[\Delta \cdot \mathbf{1}]_{n-k}$ and send the corrupted parties' shares to $\mathcal{F}_{\text{auth-rand}}$. If P_1 is honest, $\mathcal{S}$ samples random sharing $[\rho_{l,w}]_{k-1}$ and sends the corrupted parties their shares.
 - (c) Then $\mathcal{S}$ emulates $\mathcal{F}_{\text{deg-reduce}}$ by receiving from the adversary the corrupted parties' shares of $[\Delta \cdot \mathbf{r}_{l,w} - \delta_{l,w}]_{n-1}$ and their shares of $[\Delta \cdot \mathbf{r}_{l,w} - \delta_{l,w}]_{n-k}$. $\mathcal{S}$ can also compute the honest parties' shares of the former and use them together with the corrupted parties' shares to reconstruct $\Delta \cdot \mathbf{r}_{l,w} - \delta_{l,w}$, then compute and sample random sharing $[\Delta \cdot \mathbf{r}_{l,w} - \delta_{l,w}]_{n-k}$ consistent with the corrupted parties' shares. Note that at this point, (if it hasn't already) $\mathcal{S}$ can also compute $\Delta \cdot \mathbf{1}$ and use it along with the corrupted parties' shares of $[\Delta \cdot \mathbf{1}]_{n-k}$ to sample the rest of the sharing.
3. Next, $\mathcal{S}$ emulates $\mathcal{F}_{\text{verify-deg}}$ by first sending the adversary the corrupted parties' shares of the sharings $[\Delta \cdot \mathbf{1} + \mathbf{s}_{l,w}]_{k-1}$ then aborting if $\text{bad-shr} = 1$.
4. Finally, $\mathcal{S}$ sends to $\mathcal{F}_{\text{auth-rand}}$ the corrupted parties' shares of $[\mathbf{r}_{l,w}]_{n-k}$. As for $[\Delta \cdot \mathbf{r}_{l,w} - \delta_{l,w}]_{n-k}$, first note that the corrupted parties can locally change their shares to change the secret to $[\Delta \cdot \mathbf{r}_{l,w}]_{n-k}$. We may equivalently think that the shares of corrupted parties are changed so that the secret is $\Delta \cdot \mathbf{r}_{l,w}$ (and they can then change their shares to any arbitrary values). Thus, $\mathcal{S}$ adjusts the shares of corrupted parties as follows: $\mathcal{S}$ generates a degree- $(n-k)$ packed Shamir sharing $[\delta_{l,w}]_{n-k}$ such that the shares of honest parties are 0's. Then $\mathcal{S}$ computes the shares of $[\Delta \cdot \mathbf{r}_{l,w}]_{n-k} \leftarrow [\Delta \cdot \mathbf{r}_{l,w} - \delta_{l,w}]_{n-k} + [\delta_{l,w}]_{n-k}$ of the corrupted parties and sends to $\mathcal{F}_{\text{prep-st-mal}}$ these shares.

Now, we show that the real and ideal worlds are distributed identically, except with all-but-negligible probability. It is clear that $\mathcal{S}$'s emulation of $\mathcal{F}_{\text{triple}}$ is identical to the real world. If P_1 is corrupted, then $\mathcal{S}$ sending the adversary random values for the opening of $[\Delta \cdot \mathbf{1} + \mathbf{s}_{l,w}]_{n-k}$ is distributed identically to the real world since $[\mathbf{s}_{l,w}]_{n-k}$ is distributed randomly, given the corrupted parties' shares. If P_1 is honest, it is clear that it can extract the correct corrupted parties' shares of $[\Delta \cdot \mathbf{1}]_{n-k}$. If P_1 is corrupted, then $\mathcal{S}$ can clearly reconstruct $\Delta \cdot \mathbf{1} + \mathbf{s}_{l,w}$ and from this extract $\Delta \cdot \mathbf{1}$ then, along with the computed honest parties' shares, sample a sharing $[\Delta \cdot \mathbf{1}]_{n-k}$ that is consistent with the corrupted parties' shares. If P_1 is honest, it is clear that simulated $[\rho]_{k-1}$ is identical to the real world, since $[\mathbf{s}_{l,w}]_{n-k}$ is distributed randomly, given the corrupted parties shares, and so then $\mathbf{s}_{l,w}$ is random to the adversary. Then, it is clear that $\mathcal{S}$'s emulation of $\mathcal{F}_{\text{deg-reduce}}$ is identical to the real world and also, that $\mathcal{S}$ can compute $\Delta \cdot \mathbf{1}$ and, along with the corrupted parties' shares, sample $[\Delta \cdot \mathbf{1}]_{n-k}$.

Lastly, we need to show that the honest parties shares of $[\mathbf{r}_{l,w}]$ are distributed identically in both worlds. We again have that the corrupted parties' shares of $[\mathbf{r}_{l,w}]_{n-k}$ are independent of the rest of the sharings. Since $\mathcal{S}$ also has the corrupted parties' shares, this means that the sharings $[\mathbf{r}_{l,w}]$ that $\mathcal{F}_{\text{auth-rand}}$ samples are distributed identically to those in the real world. $\mathcal{S}$ also gets the corrupted parties' shares of $[\Delta \cdot \mathbf{r}_{l,w}]$ from its emulation of $\mathcal{F}_{\text{deg-reduce}}$, which are also independent of the rest of the sharings by the definition of $\mathcal{F}_{\text{deg-reduce}}$. Thus, $\mathcal{F}_{\text{auth-rand}}$ samples $[\Delta \cdot \mathbf{r}_{l,w}]$ distributed identically to those in the real world, where in the real world, the corrupted parties are also changed so that the secret is $\Delta \cdot \mathbf{r}_{l,w}$. $\square$